

What is Mística?

What is Mística?

Mística is **Telefónica's Digital Design System**. An open and collaborative system for the whole ecosystem of digital products and services in Telefónica.

It is a common design language that provides tools and guidelines with compliant components to deliver your next Telefónica's digital experience.

Why a digital design system?

Is Mística for me?

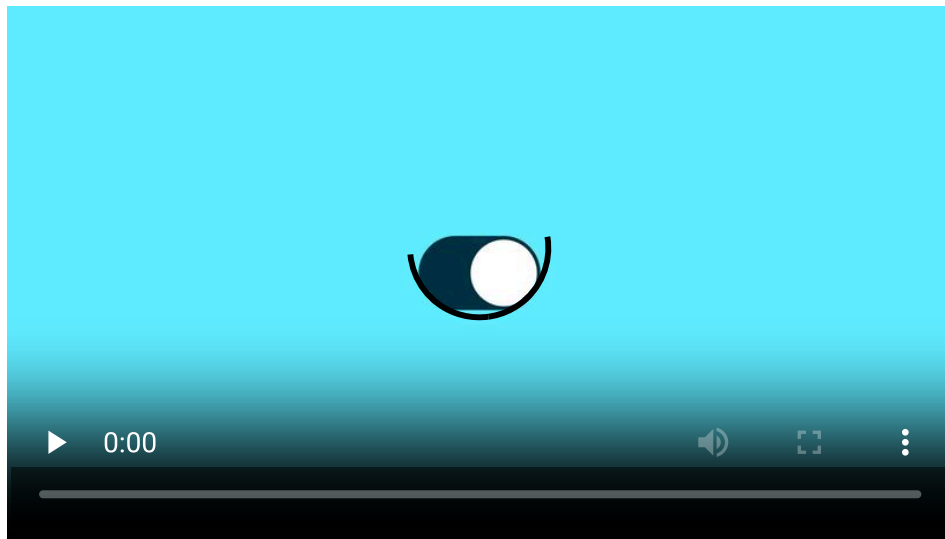
Mística is for whoever wants to use it.

You can adopt Mística all at once (Big Bang) or do it incrementally (over time). The good thing is that **each team can adopt Mística as and when it is needed.**

New products are easy since they allow full adoption from the start. A little more effort is required for existing products. If this is your case, we will advise you on how to approach it in a simple manner.

To give you an idea, this would be the **roadmap** that we will adjust to each individual case:

Contact us via **Teams** and we'll tell you how to adopt Mística on your project.



Who is using it?

Start to design

Hi, designer! 🙌

We have developed a suite of tools for the designers of different teams to achieve consistency across Telefónica products.

These include:

- Figma libraries with all available components
- Documentation related to the use of components, available under the **Components** category
- Figma materials including starting guide with Figma, UI Kits & Templates and more...
- Mística discussions to evolve components ([visit contributing guidelines](#))
- Mística team support in through **Teams**

Get Figma access

For editors

To use Mística to design products, you must have a Figma account and an Editor license.

To obtain an Editor license, please contact to epg@tid.es

For viewers (free)

If, on the other hand, you have a Figma account with your Telefonica address (@telefonica.com) and only want to have fun trying Mística in Figma or access to view projects opened by teams (viewer), fill the following form

[Get viewer access](#)



Which design library do I need?

Native OS

If your digital product is going to be developed in a native OS for iOS or Android, you must use the library corresponding to **Mística Mobile**.

Webapp

If you are going to develop a webapp that needs to have mobile and desktop views (with different breakpoints), you will need to make the mobile designs with **Mística Mobile** library and the desktop screens with **Mística Desktop** library.

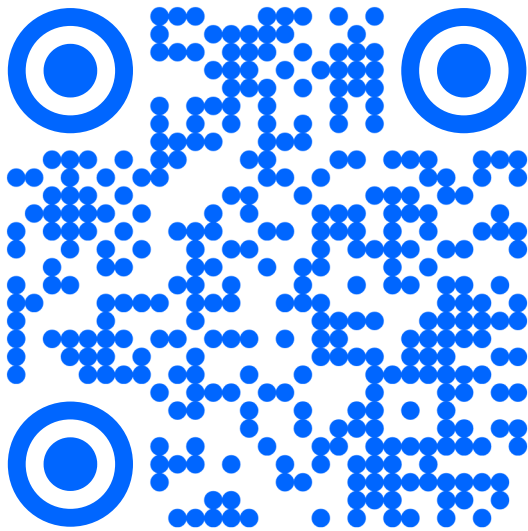
Website

To develop a **website** with Mística, you will need the **Mística Mobile** library for mobile views and **Mística Desktop** for resolutions higher than 1024px.

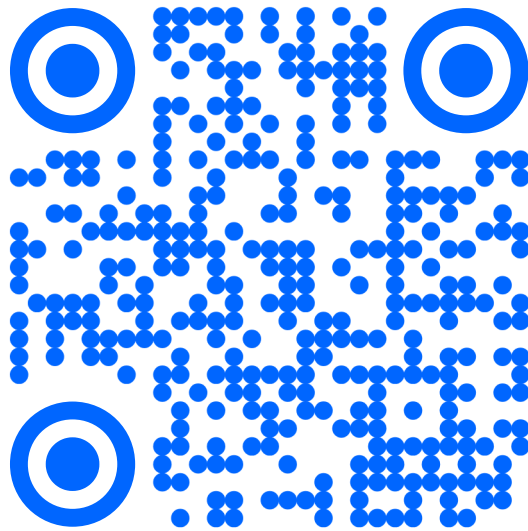
Mística design repository

Mística Catalog (native)

With Mística App (for iOS & Android) you will be able to test components directly on your phone, helping you to understand the interactions on the device where the user will consume the design.



Scan for iOS



Scan for Android

[Download for iOS !\[\]\(919a2cb85b99741a73c0c31a427236a8_img.jpg\)](#)[Download for Android](#)

Storybook (web)

With Mística Storybook you will be able to test components directly on your browser, helping you to understand the interactions on the device where the user will consume the design.

[Visit Storybook](#)

Playroom

For quick prototyping using Mística components. Using Playroom you can simultaneously design across a variety of themes and screen sizes, powered by JSX and Mística components library. It's the perfect tool to create quick mock-ups and interactive prototypes with real code. It also allows you to share your work with others by simply copying the URL.

[Try Playroom](#)

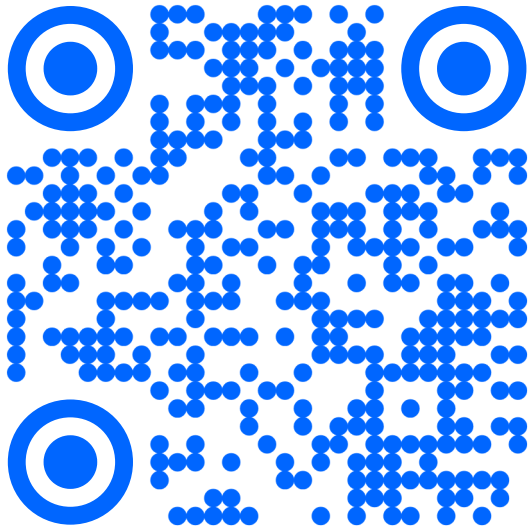
Start to develop

Hi, engineer! 🙌

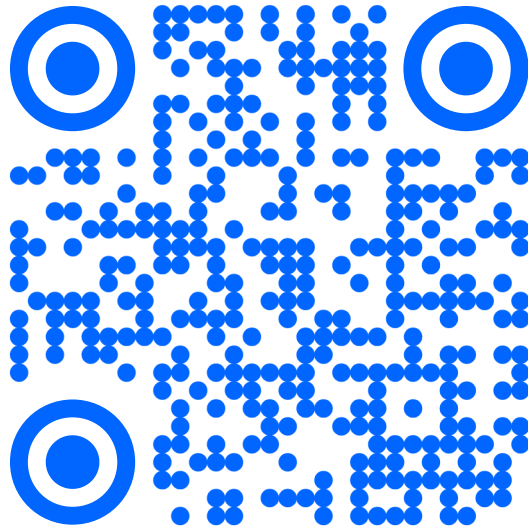
We have created all components to be used in three different environments so here we have all you need to do your development magic without losing a minute in adapting your job to meet specific OS requirements.

Mística Catalog

With Mística App (for iOS & Android) you will be able to test components directly on your phone, helping you to understand the interactions on the device where the user will consume the design.



Scan for iOS



Scan for Android

[Download for iOS !\[\]\(2e897e890e69d81eae4503a8342c36b0_img.jpg\)](#)[Download for Android](#)

Storybook (Web)

[Visit Storybook](#)

For quick prototyping using Mistica components. Using Playroom you can simultaneously design across a variety of themes and screen sizes, powered by JSX and Mistica components library. It's the perfect tool to create quick mock-ups and interactive prototypes with real code. It also allows you to share your work with others by simply copying the URL.

[Try Playroom](#)

Development Link References

Documentation & Help

Pull Requests

Bug report

Feature requests (no UI/UX changes)

Get in touch

Want to know more about Mística?

Mística is **Telefónica's Digital Design System**. An open and collaborative system for the whole ecosystem of digital products and services in Telefónica.

It is a common design language that provides tools and guidelines with compliant components to deliver your next Telefónica's digital experience.

Design Values

Mission & Purpose

The purpose is the core of the product, and it should inform design and development decisions. The effectiveness of a design system can be measured by how well its different parts work together to help achieve the purpose of the product.

Our mission is to digitize the customer relationship around the services they have signed up to with Telefónica to facilitate problem solving in all points of contact with the telco, as well as access to exclusive benefits and subscription to new services.

Design Principles

Design Principles are a set of considerations that form the basis of any good product. Our digital design principles reflect how we think about design. They provide a way for us to look at the work we create and how we create it: **building the right thing; building the thing right.**

Think global, act local

Take advantage of localization to make your global products reach specific markets.

Flexible

Our design should meet the requirements of users from different countries and with different digital knowledge and skills through the localization of our product.

Scalable

Our design consists of independent modules that can evolve, grow and be adapted to the different needs of the product and users.

Inclusive

Our products are used globally by a diverse community. Our goal is to ensure that the design of our products is welcoming and accessible for everyone.

Human

Our design should be respectful and revolve around the experience and habits of each user, communicating with them directly, closely and transparently, without withholding information.

Functional

Any design decision should have a defined, clear and tangible objective. The main function of our design is to find solutions to problems, always taking into account business needs and respecting users.

Delightful

The design of our product should be an added value to the functional part and should create a positive and valued experience for the user in addition to having its own personality that reflects the brand's values.

Innovative

Our design should reflect the image of a leading company in the Telco market and at the forefront of mobile technology and incorporate new technological and design trends into our products.

Design Languages

Tokens

Tokens are those system pieces that are more subatomic, such as colour, typography, spaces, edge width, rounded corners, shadows, animations or even media queries.

Learn to use Mística colors

There are many other used for specific things such as Buttons or Feedback but here we shared the most generic ones.

If you need to know more about color and contrast, refer to our specific [accessibility section](#).





Colours

The following list show all the colors available in Mística



Palettes



Border radius



Typography

Tokens are those system pieces that are more subatomic, such as colour, typography, spaces, edge width, rounded corners, shadows, animations or even media queries.

Mística's font stack

Brand fonts

Furthermore, if your project or product requires it to create a cohesive brand experience, you have available the following brand fonts:

What are text-presets?

The text-presets nomenclature allows us to make these styles reusable on different screens and in different contexts, regardless of their location. At the same time, they make it possible to create a relationship of sizes between the different formats and devices with which we design: mobile, desktop, TV, etc., allowing us to address the different content and requirements of a product.

There are currently two sizes of our text-preset typographic styles: **App/Mobile** and **Desktop**.

Weighing details

Mística doesn't allow the possibility of playing with different types of weights in large sizes (from text-preset-5 onwards). This is because we consider large typographic sizes as a feature that a brand can leverage to showcase its personality, therefore, the size of these larger weights is determined by each skin.

Correspondences

The sizes shown below correspond to the size and height of the line:



Combining Text-presets

Text-presets allow styles to be reused across different screens and contexts, regardless of their location. This means that, for example, a text-preset-5 (18 Light) can be used either as a Title or as a Body.

For this reason, we have defined a hierarchy that helps us combine the different styles in a logical way when designing.

Accessibility

Conversational

Conversation design is a design language based on human conversation. It's a synthesis of several design disciplines, including voice user interface design, interaction design, visual design, motion design, audio design and UX writing. The designer curates the conversation, defining the flow and its underlying logic in a detailed design specification that complete user experience.

It's a common misconception to assume that "conversation" refers only to what is spoken or heard, conversation is inherently multimodal.

What is Aura?

Layouts

In Mística we divide the layout components in two main groups:

Full page oriented

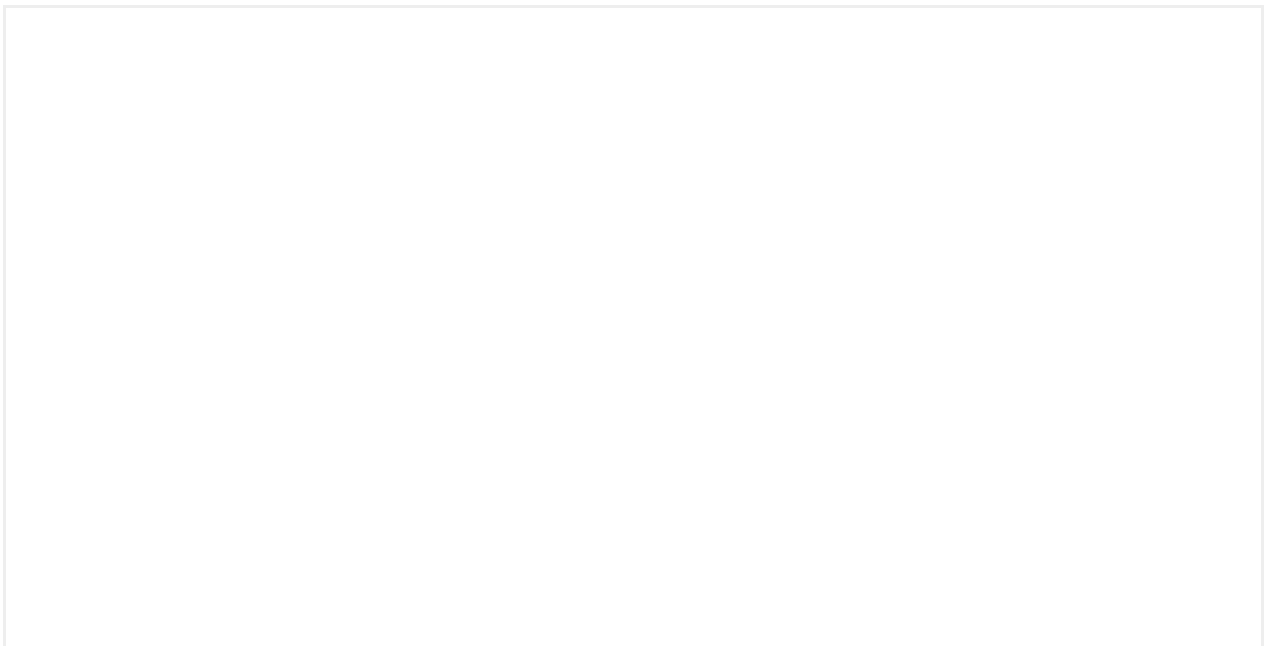
- Responsive Layout
- Grid Layout

Content oriented

- Grid
- Horizontal Scroll

Using layout components

The following example mixes multiple layout components wrapped inside a responsive layout:

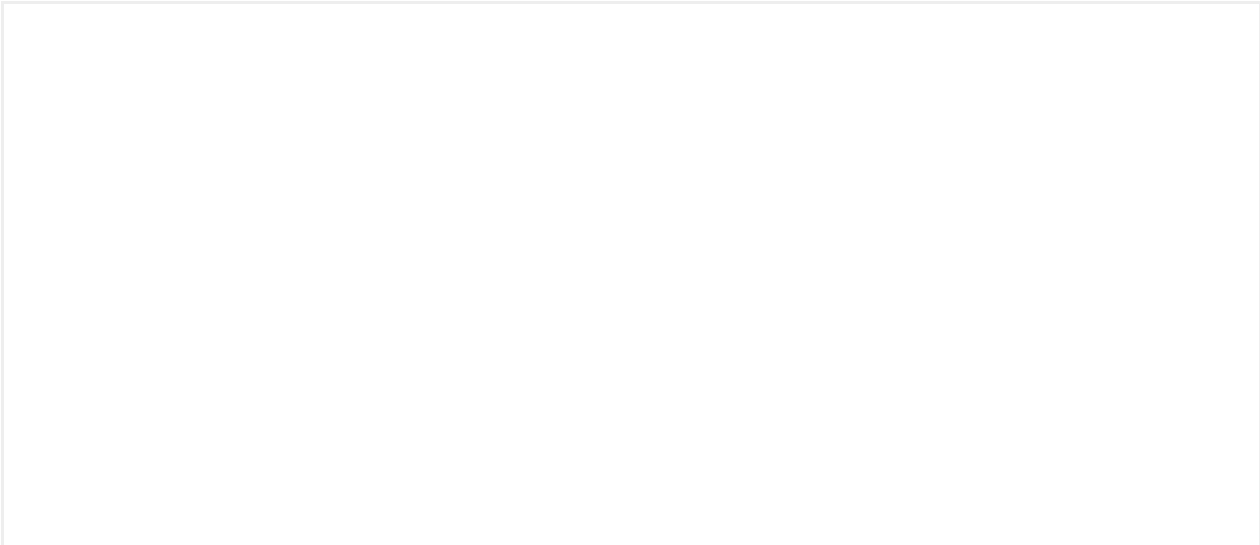




Responsive layout

The responsive layout center the content applying the correct margins in the horizontal axis for each of the different breakpoints defined.

	Mobile	Tablet	Desktop	Large Desktop
Breakpoint range	320 - 767	768 - 1023	1024 - 1511	+1512
Horizontal margin	16	32	48	auto





Align



Grid layout

Distributes the content in 12 columns with predefined gutters depending on the breakpoint. It is also possible to define the vertical spacing for tablet and mobile when the items are vertically stacked.

	Mobile	Tablet	Desktop	Large Desktop
Breakpoint range	320 - 767	768 - 1023	1024 - 1511	+1512
Columns number	1	1	12	12
Column size	Fluid	Fluid	Fluid	96
Gutters size	—	—	24	24



Grid layout templates

The grid layout provides a set of predefined templates that distribute the content in identified patterns:

- **6 + 6:** Focused in image + text distributions
- **8 + 4:** Used for a main content + aside distributions
- **4 + 6:** Used for the master detail layout
- **5 + 4:** It is exclusively used for the full screen funnel.
- **3 + 9:** Useful for compact navigation or filter left sections.

Grid

The grid component helps to distribute the content in both vertical and horizontal axis.

It allows the following configurations:

- **Columns and rows:** define the amount of columns or rows.
- **Grid item span:** define the amount of columns, rows or both a grid item can occupy.
- **Gutter:** define the space between columns or rows
- **Alignment:** define how items are aligned inside each cell of the grid.

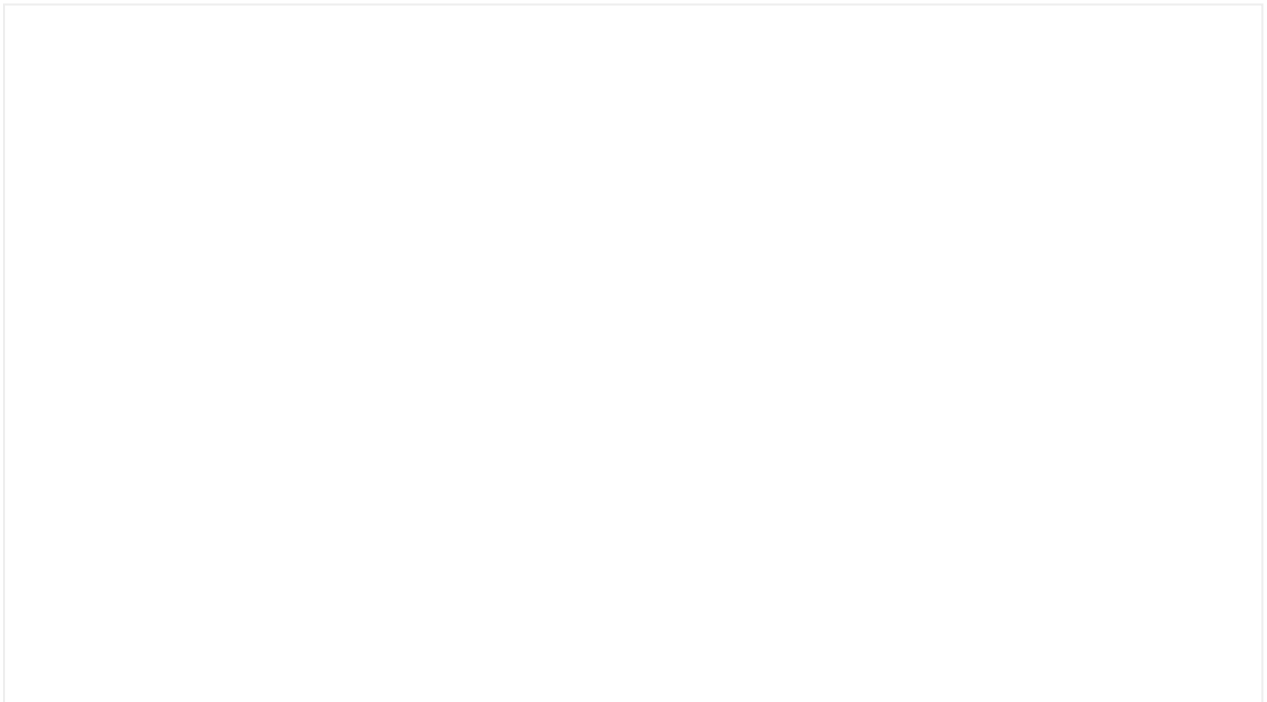
Columns, rows & span

The number of columns and rows of the grid can either be defined, or be automatically generated depending of the size of the gridItems.



Gutter

The space between columns or rows can be defined independently:



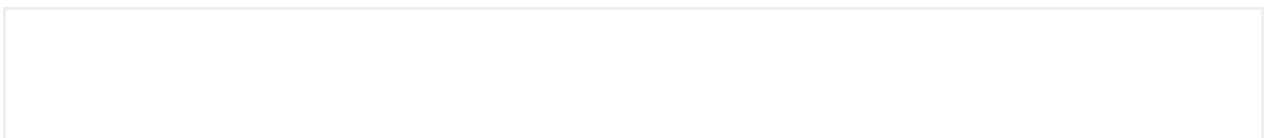
Aligment

The grid items can be aligned both in the vertical and horizontal axes



Horizontal scroll

The Horizontal scroll component allows content overflow outside the parent container limits.



Fixed Footer Layout

The fixed footer layout allow to place content in an overlay fixed at the bottom of the screen in mobile devices. In desktop this overlay will disappear, the layer will not be fixed to the bottom and will behave like other content.

Any content can be placed inside the fixed footer layout since is slot.

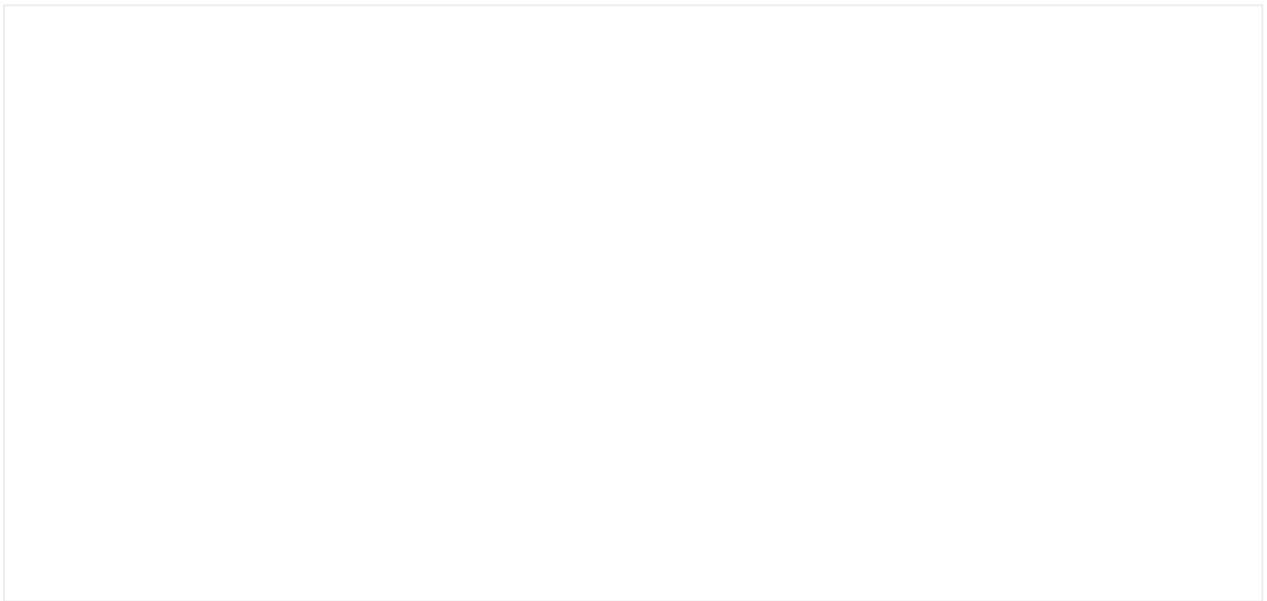


Spacing

Spacing Types

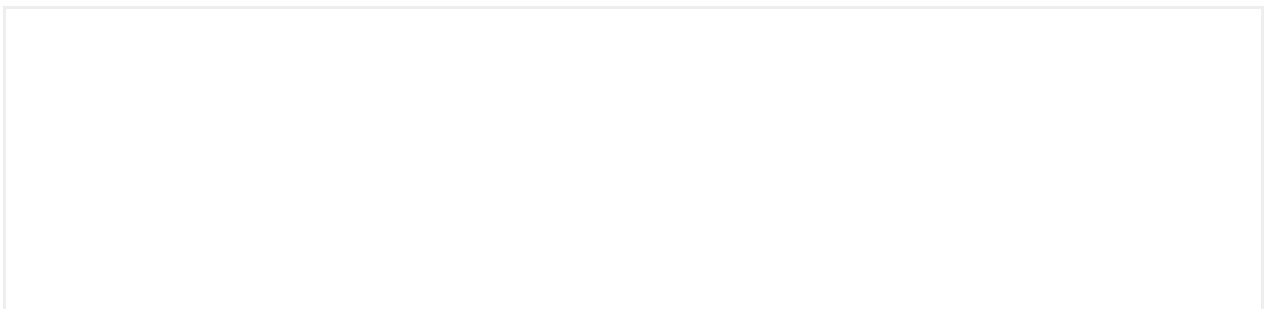
Box

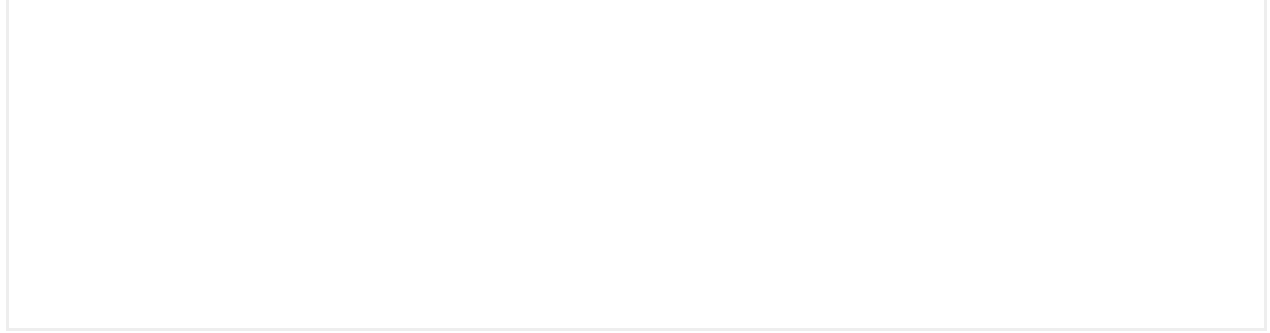
It works as a "wrapping" of the component if applied. We can also add inner paddings.



Stack

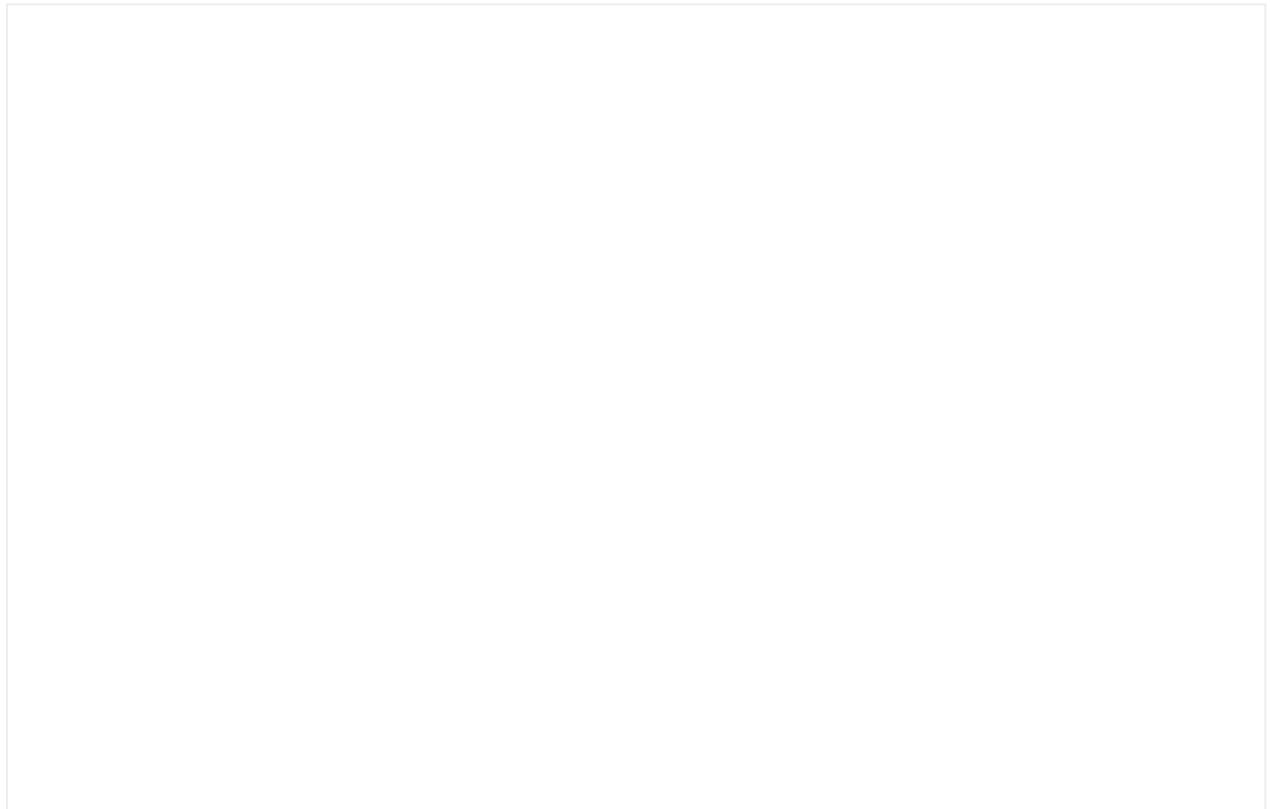
The stack separates the spaces between components **vertically**.



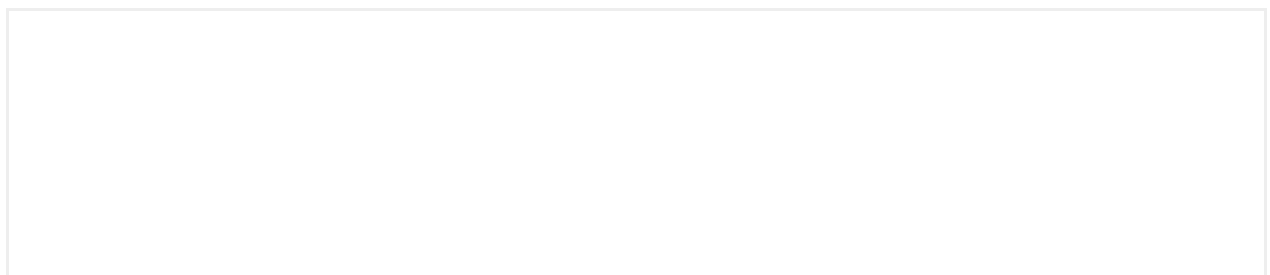


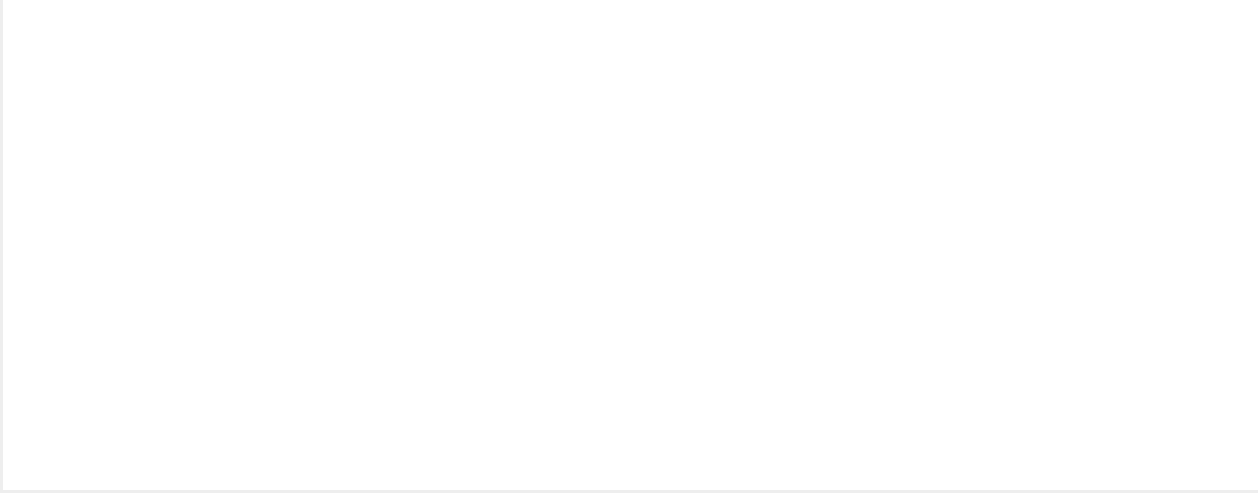
Inline

The inline separates the spaces between components **horizontally**.



Interactive demo



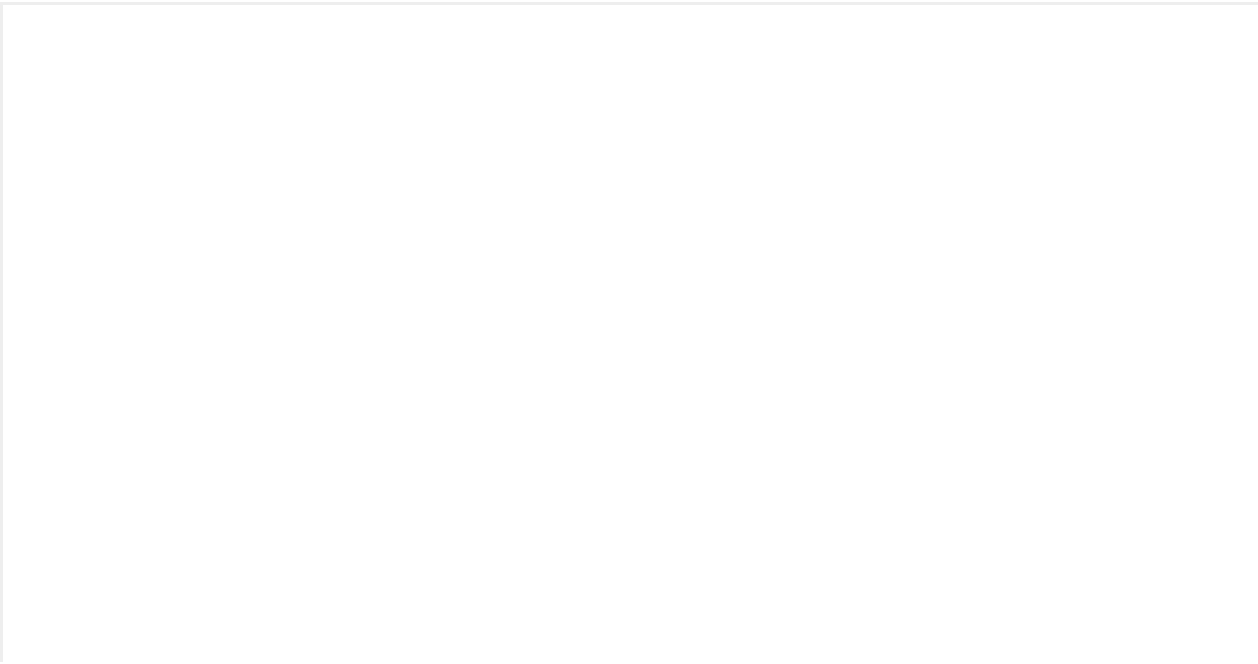


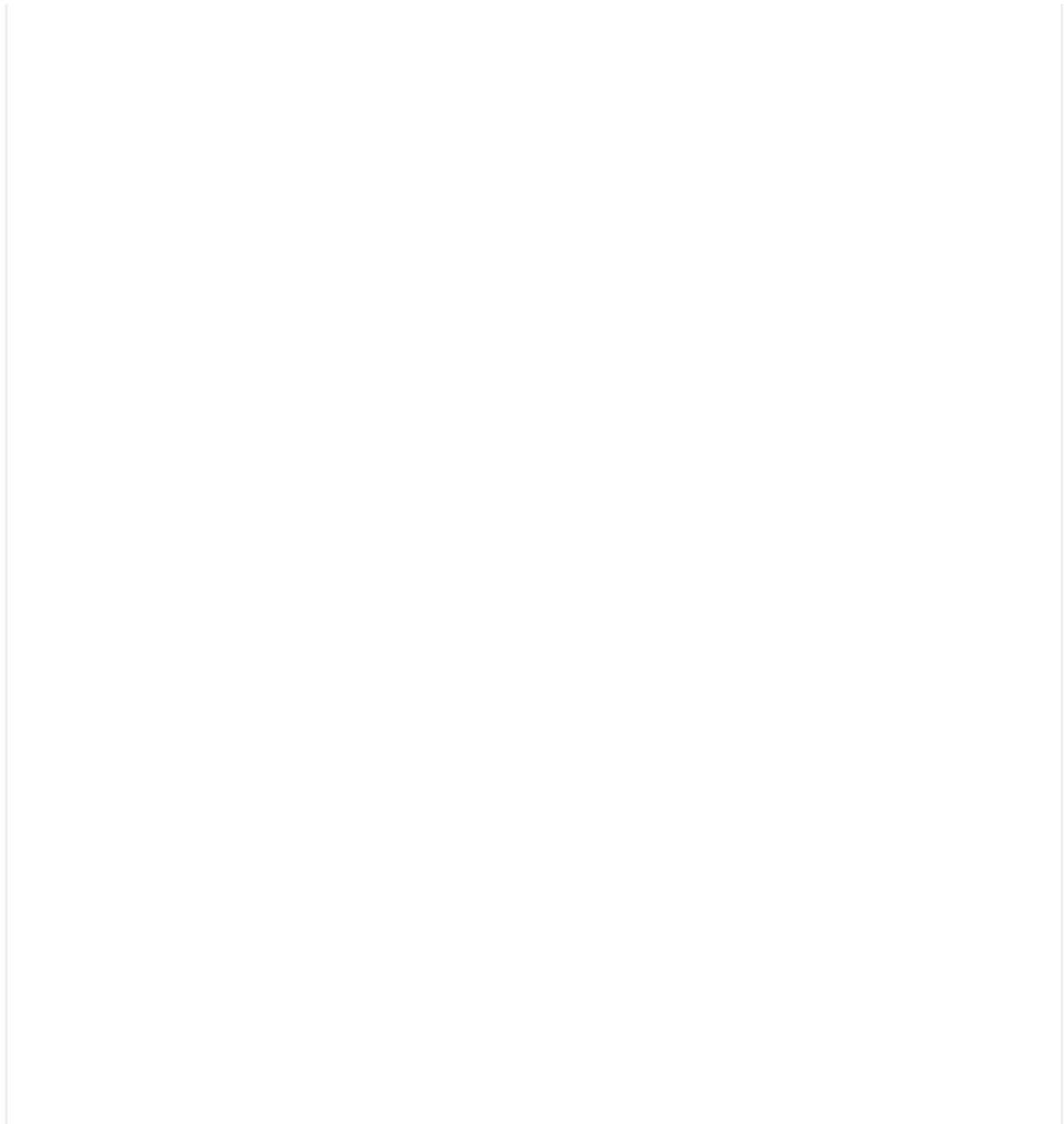
Vertical Rhythm

The vertical rhythm is the way in which we help to organize the different information blocks on the screen.

Both for desktop or mobile, we have established 3 levels of deepness where we will apply defined spaces to help organizing the information on the screen and, this way, have coherence with the rest of the screens.

We apply this rhythm in the body of the screen.





Level 1

We will use Box with a Y-padding (top and bottom):

- Mobile: 24px o 0px
- Desktop: top - 40px; bottom - 80px (always)

This will separate the first and last component of the navigationBar or Header and from the tabBar or the bottom of the screen (if we have a scroll). This way, we avoid the last component to almost reach the very bottom of the screen.

Level 2

Level 2 applies on the blocks inside the body. The blocks help to organize the information by information modules and help the user to structure the content better.

It is responsibility of the designer to define this block organization and how the information should be structured on the screen.

The **spacing between blocks** will be as follows:

- Mobile: 40px
- Desktop: 48px

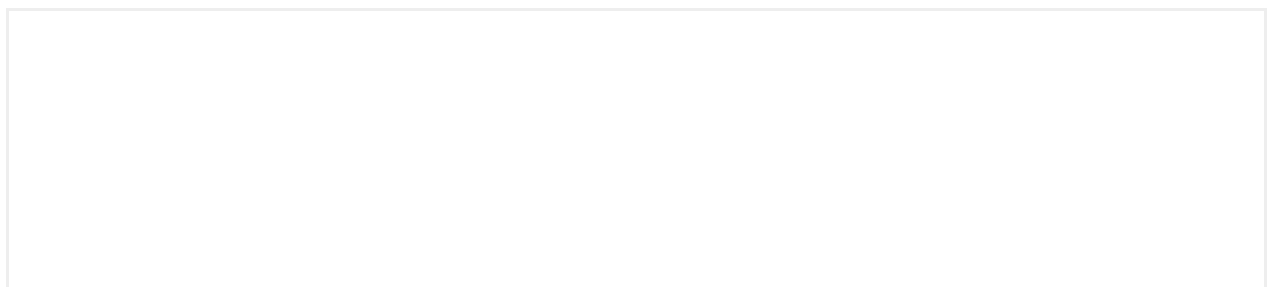


Level 3

The spacing between Titles and components inside a block.

The **spacing between components** will be as follows:

- Mobile: 16px. However, there will be cases in which we might need 8px.
- Desktop: 24px



Level 4

The last level refers to the spacing between components inside a block.

The **spacing between components** will be as follows:

Mobile: 16px. However, there will be cases in which we might need 8px.

Desktop: 24px



Icons

Multibrand sets

From Mística, we support the icons created by the global branding team for all Telefónica's **digital products**. For this purpose, we have created a library for Design and another one for Development ([mística-icons](#)).

This library contains icons in three styles: **Light**, **Regular** and **Filled**. A total of +2100 available icons to use in all Telefónica digital products. To better know the usage of different weights, you can have a look on the guidelines in [Brand Factory](#).

General rule for icon sizes

- On mobile and desktop resolutions, we will use light style for sizes of 48px or bigger.
- On mobile and desktop resolutions, we will use regular style for sizes smaller than 48px.
- The filled style is used on devices in which the rendering quality is lower, for example, on TV screens.

Multibrand icon system

Mística icons is ready to be used with per brand icon sets. That means icons can change style automatically when the brand is changed, so from the product and design side you won't have to worry about using the right icon.

Digital system icons repository

Icons catalog



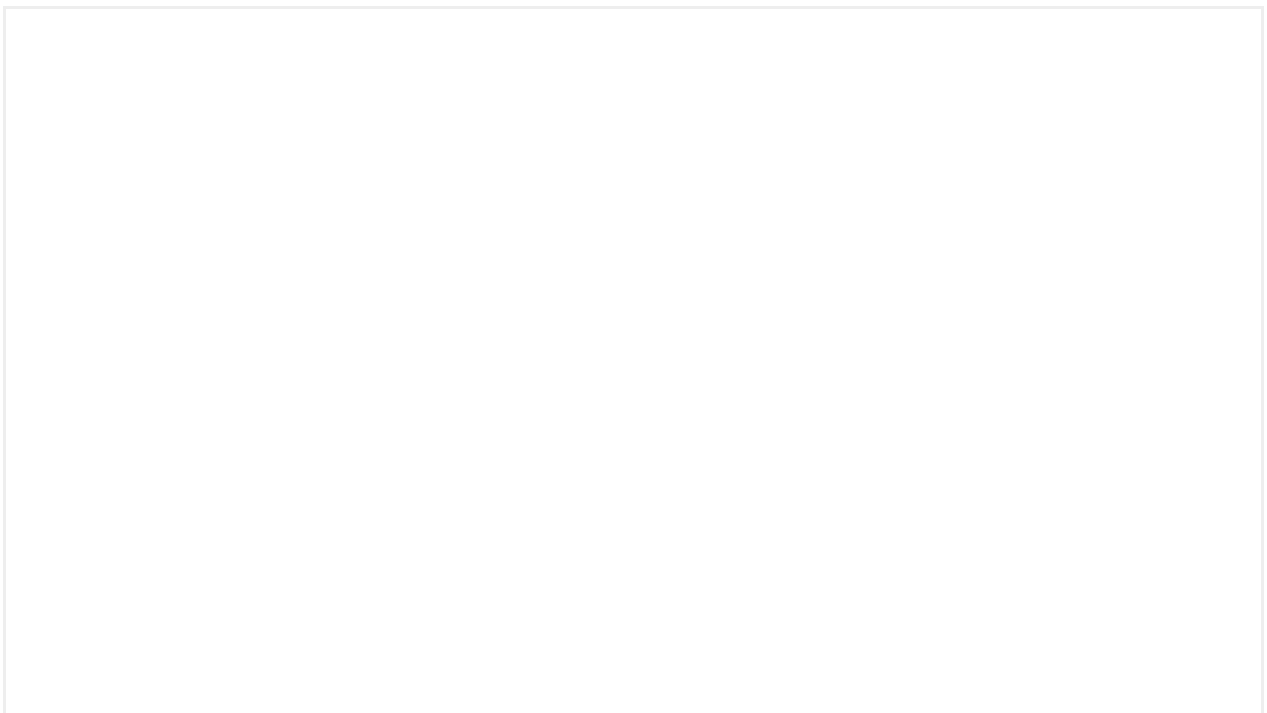
Motion

Mística provides some animations out of the box:

FadeIn

Used to make items appears. Commonly is used when new components or content added in the screen.

Emotional branded animations





Theming

A theme is a collection of design tokens that define how Mística components are displayed.

Each brand supported by Mística shares the same theme base definition* and only the design token values change. The values applied by each brand are what we call a **skin**.

(*) For some brands, there are extended definitions in the theme. These definitions are tied to extended components that are only available in that brand.

Skins

Not all visual properties in our components are available for brand customization in the theme definition. At the moment, each skin can only define the following values:

- Color
- Text-presets `font-weight` from 5 to 10
- Border radius

Color

Within a theme, the color type design tokens also vary depending on the following:

- **Color scheme:** Adjustments on the whole interface based on configuration or user preferences.

- **Theme variant/context:** Adjustments at the component level based on configuration.

In order to understand the complexity that represents color inside Mística, we will cover the topic in-depth over the following sections.

Color scheme

Mística components have built-in support for light and dark modes.

On all Mística implementations, the color scheme can be forced as `light` or `dark`, or work as `auto` (OS dependent)

Variants

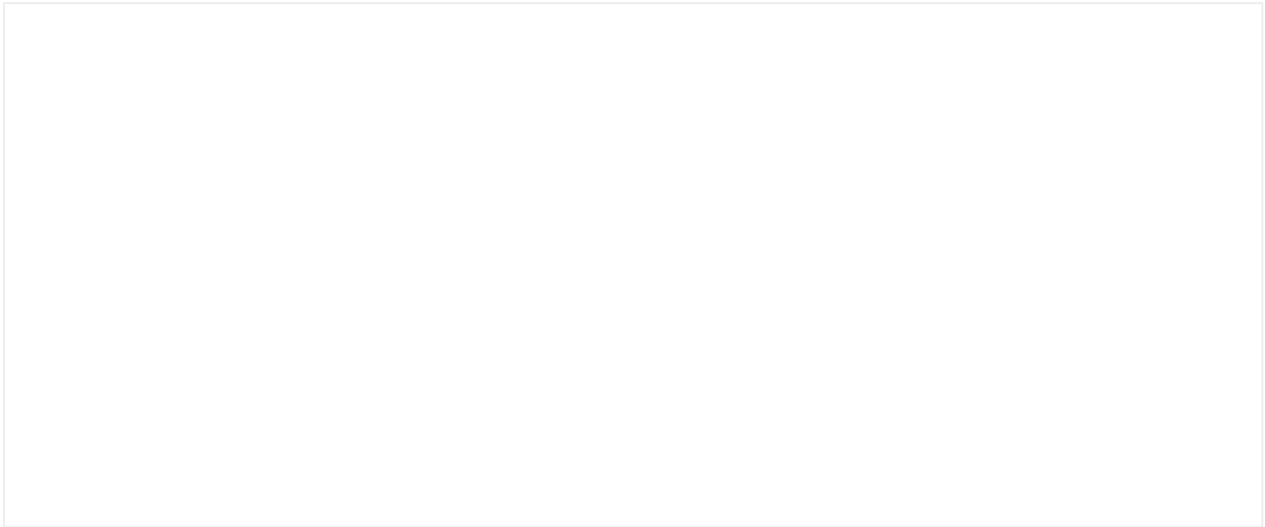
In addition to light and dark modes, components are displayed differently depending of the variant and the variant context they're placed

Variant context

A container variant defined as `default` / `inverse` / `alternative` creates a context. All components placed in this context change to improve contrast.

- **Over inverse:** Components that are placed inside a container defined as `inverse`.
- **Over alternative:** Components that are placed inside a container defined as `alternative`.

- **Over media:** Components that are placed inside a container with an image in the background and it's defined as `media`



In the example above, the buttons are placed in an inverse context (they are "over inverse").

- Use inverse contexts to create contrast between content sections or group featured content.
- In dark mode, the inverse context works as default to reduce cognitive load.
- Inverse containers always use the `backgroundBrand` color token as background color.

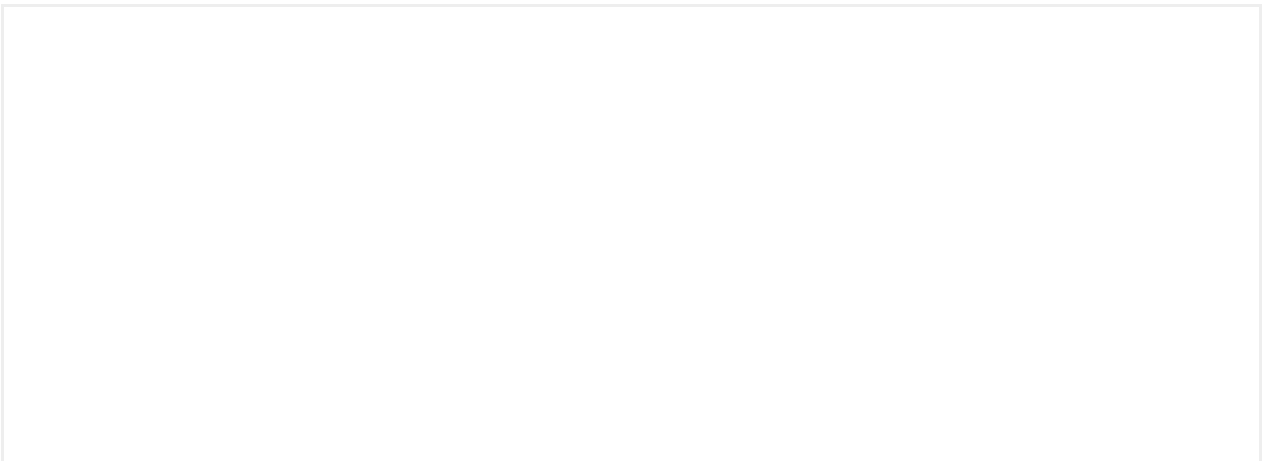
Variant

Specific components can be rendered in different variants (`default` / `inverse` / `alternative`) without being in an context.

Inverse support

The components that can be rendered as `inverse` are:

- Highlighted card
- Snap card
- Header
- Navigation bars
- Boxed row





- Use inverse components sparingly, high usage can increase cognitive load.
- Use inverse components to attract user attention.

Customization

You may encounter times when you are required to build an experience that isn't covered by core Mística components.

The happy path

The ideal option is to identify all the existing Mística components that can be used to create your desired experience. We're invested in helping you through this process and we have multiple channels where you can contact us directly:

- [Github discussions and Q&A](#)
- [Mística Help Teams channel](#)

Before using custom solutions it's always better to reach the Mística team in case there's something planned in the roadmap or can be included if it's identified as a global need. In addition, some of our component APIs have a lot of flexibility so it may be possible to cover your need.

Extended needs

The main goal of Mística is to provide the basic elements necessary to cover a standard digital product.

We are aware at Mística that it's impossible to include the more specific needs of products in the core system. That's why we believe it's common for a product to be built with a mix of Core + Extended. A core base of Mística and a customized part to cover the specific needs of that product (we believe in this balance of 80% Core / 20% Extended).

We call this extension of the system in the product the **Extended Library**.

- **Core Library:** Main library of Mística
- **Core Component:** Component that comes from the main library of Mística (e.g., Button)
- **Extended Library:** Library that stores components with specific needs of a product

- **Extended Component:** Atomic construction designed to cover product, design, or business needs of a product. (e.g., Component of a rate table)
- **Extended token:** A design token created to cover a specific component or UI need, that either by logic or by definition a design system token is not covering.

Extended components

Mística provides wide coverage of UI components, but there may be times when you need to build a custom component. This could be because:

- It's not on the Mística roadmap yet or will never be. In these cases, creating a parallel workstream may be necessary to meet your product deadlines.
- It's a unique product experience that doesn't fit into a core component. Mística will not aim to cover every brand and product experience to avoid bloating the Design System.

These custom components can still benefit from Mística in different ways:

- Mística design tokens
- Primitives

Design tokens

When building custom components you still can use our design tokens to avoid desynchronization between visual styles.

Primitives

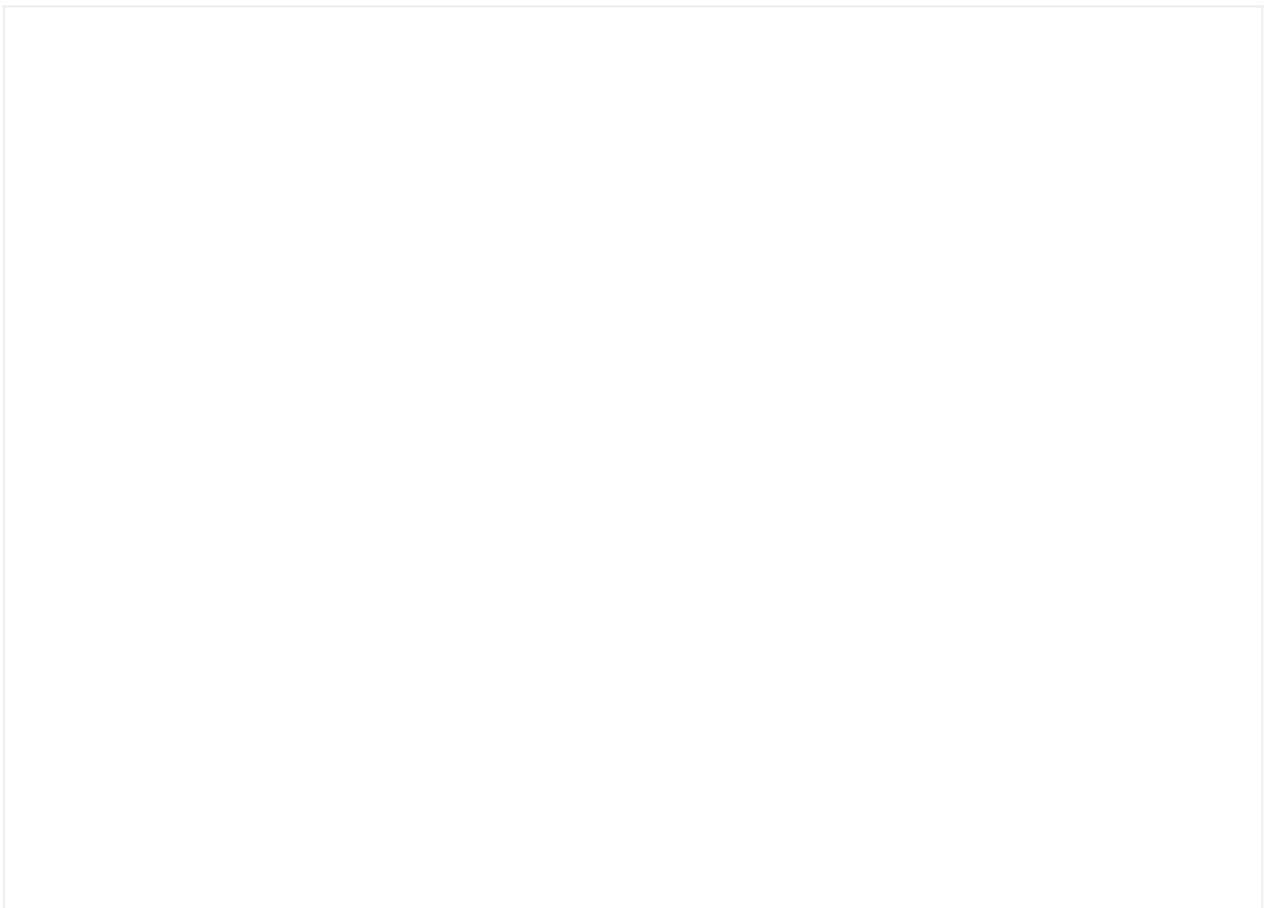
Primitives are low-level UI components that we reuse internally to create core components and are an exceptional tool to create atomic components.

Text

Our text presets have a predefined `font-size`, `line-height` and `font-weight`, when needed it's possible to use the text primitive, where those properties can be defined for desktop and mobile devices.

Boxed

Meant for creating containers it works out of the box in inverse contexts or as an inverse container.



Circle

Meant for creating circular containers, its background can be customized to include a `background-color`, a `background-image`, an icon, or any other content such as text.



Image and video

Any media content can be included using these primitives. They allow a high level of configuration of the media content size, `aspect-ratio`, `border-radius`, and so on.



Images

Formats supported are PNG and JPG, both web optimized and with a recommended size of 2.5x - 3x to avoid the image could reduce its quality on a Retina display.

Videos

The component also supports MP4 videos without sound. It is recommended that they do not exceed 10 seconds in length and can work in a loop.

It is recommended to use a standard aspect ratio for images and video (16:9, 4:3, 1:1) to keep consistency across the products. But custom ratios are allowed.

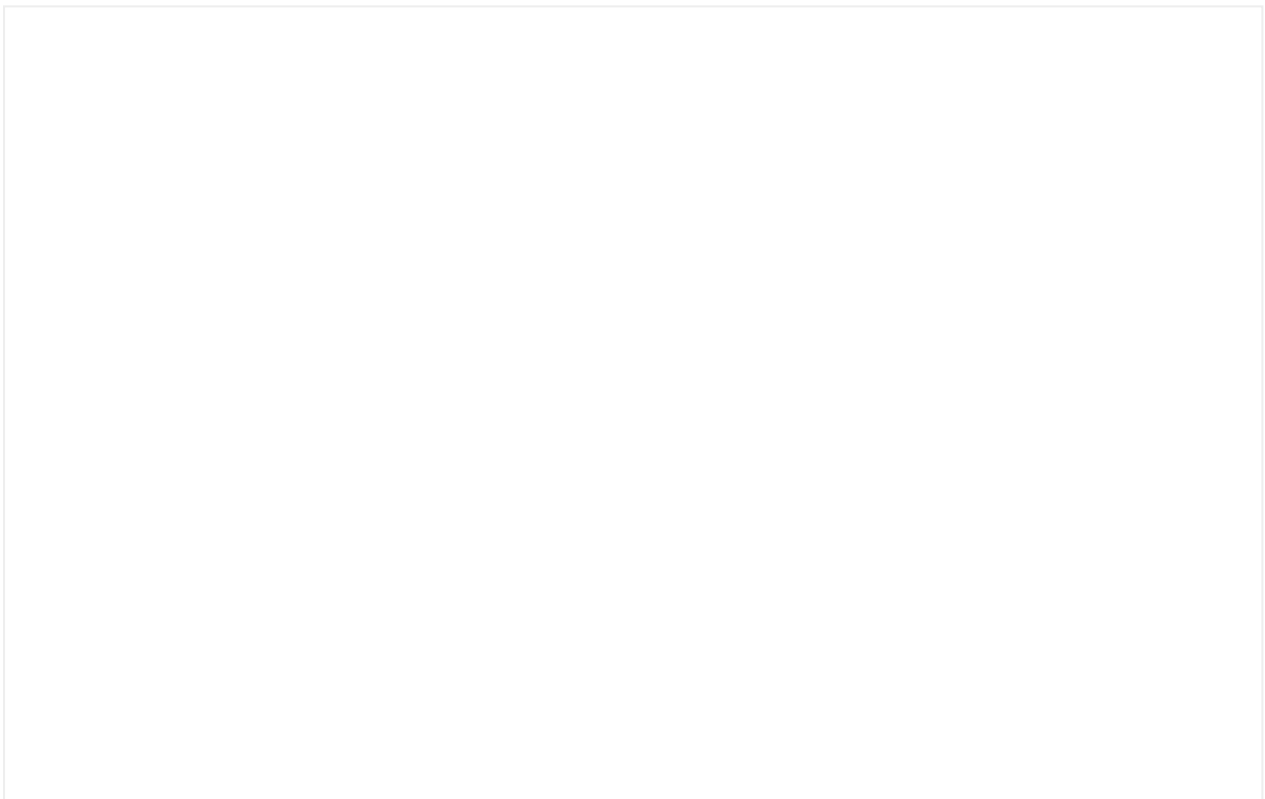


Atomic builds

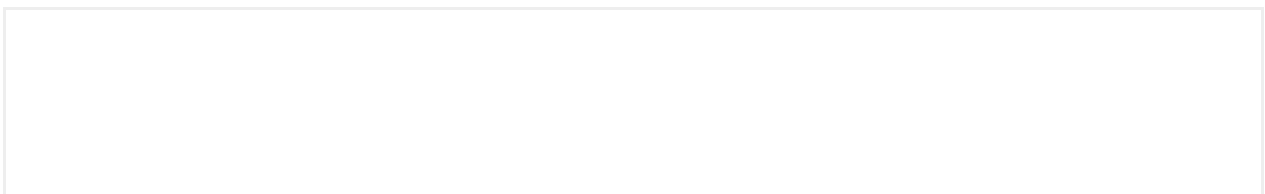
Each core component or primitive can be used and mixed with custom elements to create more complex solutions that are not provided out-of-the-box in Mística. We use the term "Atomic builds" when referring to this.

- **background** tokens: are meant for application or section backgrounds
- **backgroundContainer** tokens: are meant for group content

An example of their usage, change between values and to dark mode to understand their behaviour:



The available tokens are:





Accessibility

Localization

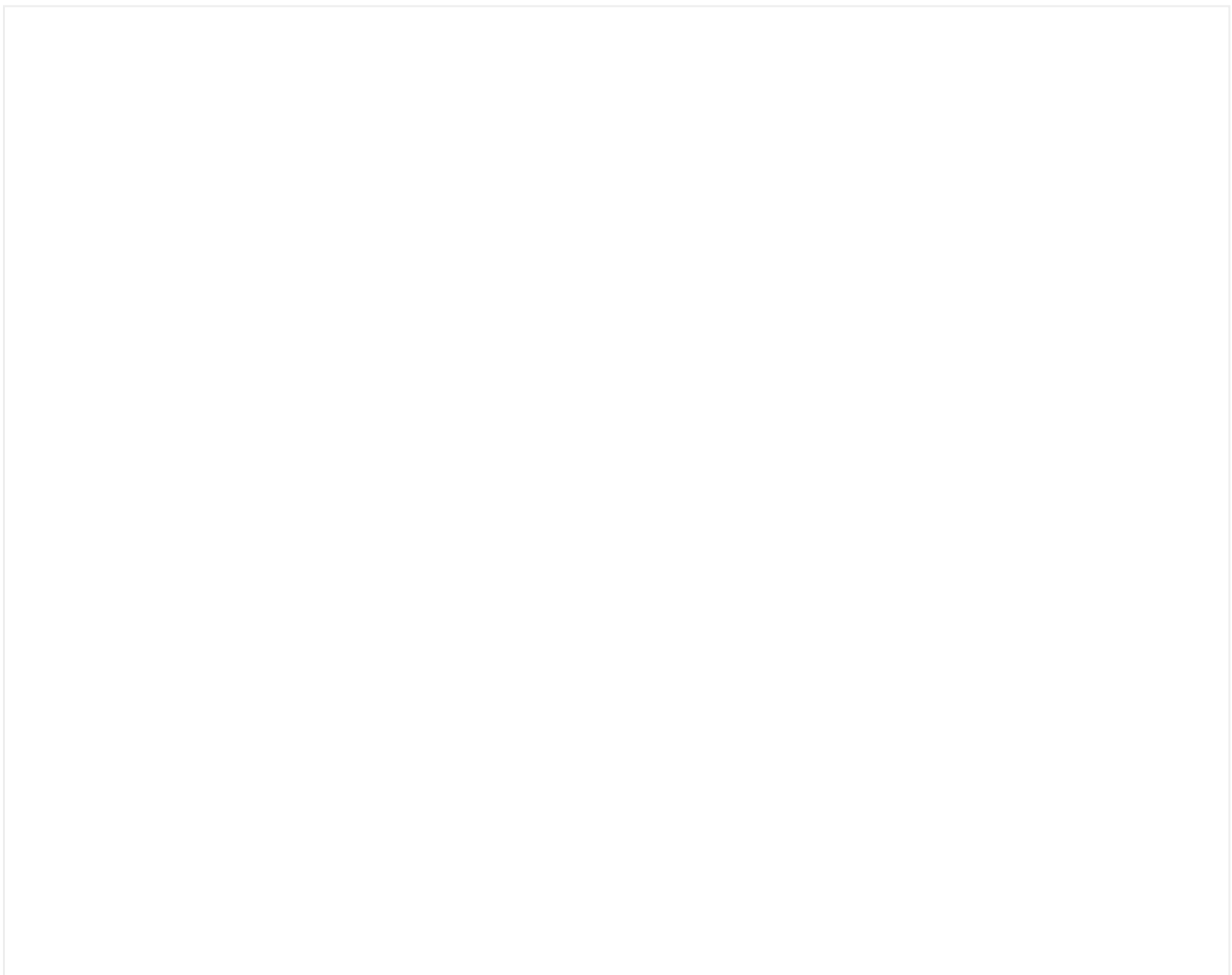
In Mística, there are certain elements that adapt based on the brand's location and the region code.

The following list comprises predefined texts within Mística's components, but each product has the flexibility to customize these texts to align with their specific requirements.



Phone numbers

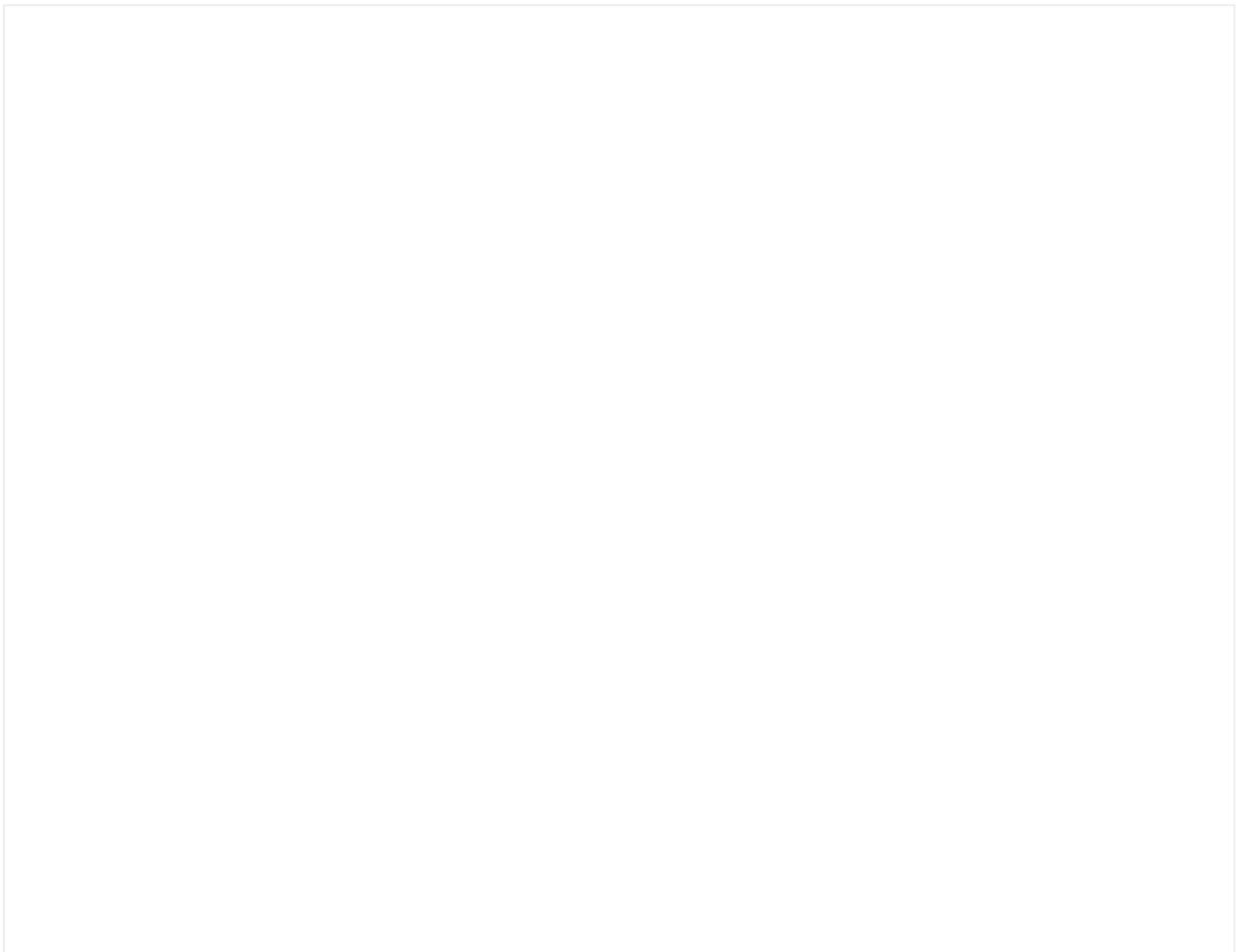
Phone numbers automatically change in Mística according to the configured region code of the skin. Here is an example of how the phone number format is set in the different region codes supported by Mística.



Text wrapping

In Mística, we have defined a rule for hyphenating words that applies across all brands and languages.

If the word does not fit in its container, a hyphen will appear. If the word fits in the container but does not fit on the line, it will move to the next line.



Accessibility

At Telefónica, our mission is to connect people. Therefore, it is essential for us to break down the barriers between people and technology by providing the best possible experience for everyone.

Accessibility in Mística for everyone means caring deeply about making quality products. A quality product should be usable and useful to everyone. Mística provides accessible components that offer a solid foundation for creating inclusive products at Telefónica. However, **complete product accessibility is not guaranteed by using these components alone.** It's essential to integrate them properly and consider other design and user experience (UX) aspects to avoid unforeseen barriers and ensure an accessible experience throughout the product.

Inclusive design requires that accessibility be considered at every stage, from early designs to the final product. This means that all team members must collaborate to ensure that accessibility is implemented correctly.

International Standards

Global Framework

Annotation Kit

Essentials guide





Overview

This is the state of each component in Mística

Accordion

Accordions allows users to expand and collapse content, helping to lighten the interface.



Anatomy

An Accordion is an individual component, but it is very common to display multiple Accordions stacked vertically to visually present them as a group.

The Accordion is made up of a header and a content panel and it has two different views: collapsed and expanded. When is expanded, the panel displays text as a paragraph or if you need it, you can include custom content.

Header

1. Asset (optional)
2. Title (required): should be as brief as possible while still being clear and descriptive.
3. Subtitle (optional): should be short and concise.
4. Detail: Secondary text to provide brief information.
5. Action chevron (required): indicates if the panel is expanded or collapsed.

Panel

An accordion must always show content associated with the header.

5. Body content (required): content inside of a panel may be paragraphs and include custom content if needed.
6. Divider (optional)

Grouped accordions

Although **it is not a type of component accordion** as such, given that it would be built at the atomic level, it serves as a solution in cases where you need to group accordions together to better organize the information.

To do this, **you have to build it through atoms** using the **boxed** and **accordion** components.

The following are examples of different compositions built using this method.



Slot

The body content of an Accordion can be quite versatile and adapt to various information presentation needs. Use the Slot if you need to include links, text in different formats, or other types of elements in addition to plain text.





Accessibility

Additional info

Avatar

Avatar is a visual representation of a user or entity profile in the interface. It supports images, text as initials, and icons.



Typology

The avatar definition is split into 4 types:

- Image
- Initials
- Icon
- Inverse

Furthermore, each type supports a badge on the top-right corner.

Image

Whenever it's possible, **try to use a thumbnail image of a user**. This gives an additional humanizing touch to the interface and helps to visually organize the information.

If there is an error loading with the Avatar Image, the component uses **2 fallbacks alternatives** in the following order:

- If there's a name prop, we use it to generate the initials.
- If there's no name prop, we use a default icon avatar.

Initials

If there is no image source provided to the Avatar, the two character of the name provided will be used as a placeholder. The initials displayed in the avatar try to be smart based on the user name initials. The avatar will still always **max out at 2 characters**. The initials will display capitalized.

Icon

Icons can also be displayed instead of images or initials.

Inverse

Use this Avatar type on coloured background.

Sizes

By default, the **Avatar component is available in Large size** but you can resize it if you need.

Mística recommends use these standard sizes:

24px 32px 40px 56px 64px

Avatar with Badge

Each **Avatar type support a badge** (non-numeric or numeric) on the top-right corner.

Usually used when new information is available, for reminders and notifications.

Read **Badge** section for more information.

Custom colors

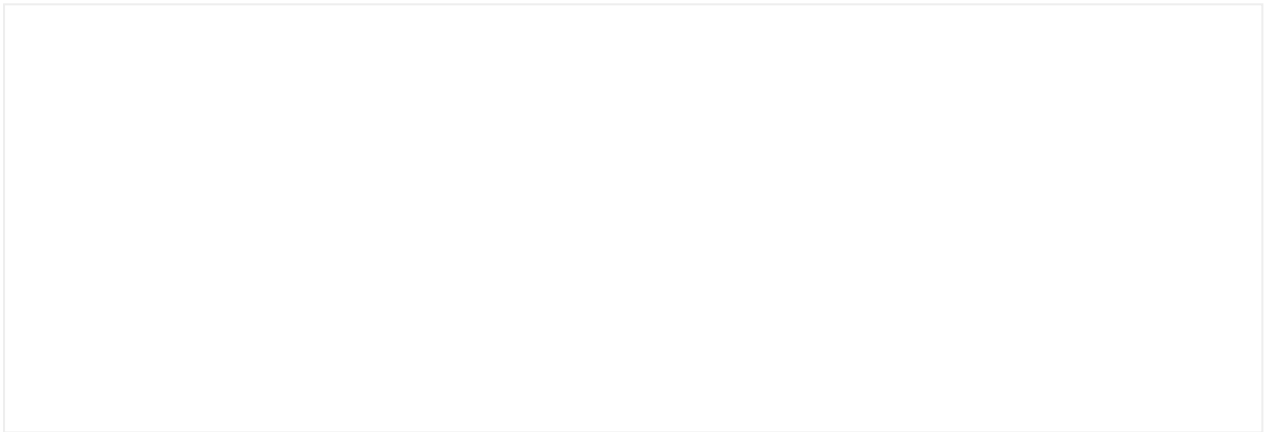
Implementation

Additional info

Breadcrumbs

Breadcrumbs are a navigational element to help users to understand where they are in a website as well as content structure and hierarchy.

Interactive Demo



Usage

When to use

- **Use the Breadcrumbs component when you need to help users understand and navigate between multiple pages in a website.**
- If a breadcrumb trail is used in the application, it must appear on all pages other than the front page. All elements in the breadcrumb path should be clickable links and the current page should appear as plain text.
- Use only like a secondary means of navigation.

When not to use

- Don't use Breadcrumbs to show steps in a process.

Layout

- The most common place for a breadcrumbs component to appear is before the main content of a page preferably just below the header.
- It is usual to start from the highest level (homepage) and end with the current screen as the final item in the list to make it clear where the user is located.

Anatomy

1. Root item: Usually the homepage.
2. Separator: Items are separated using a forward slash (/).
3. Levels: Show the navigation hierarchical order in the path.
4. Location: Current page the user is on. Last item is not a link.

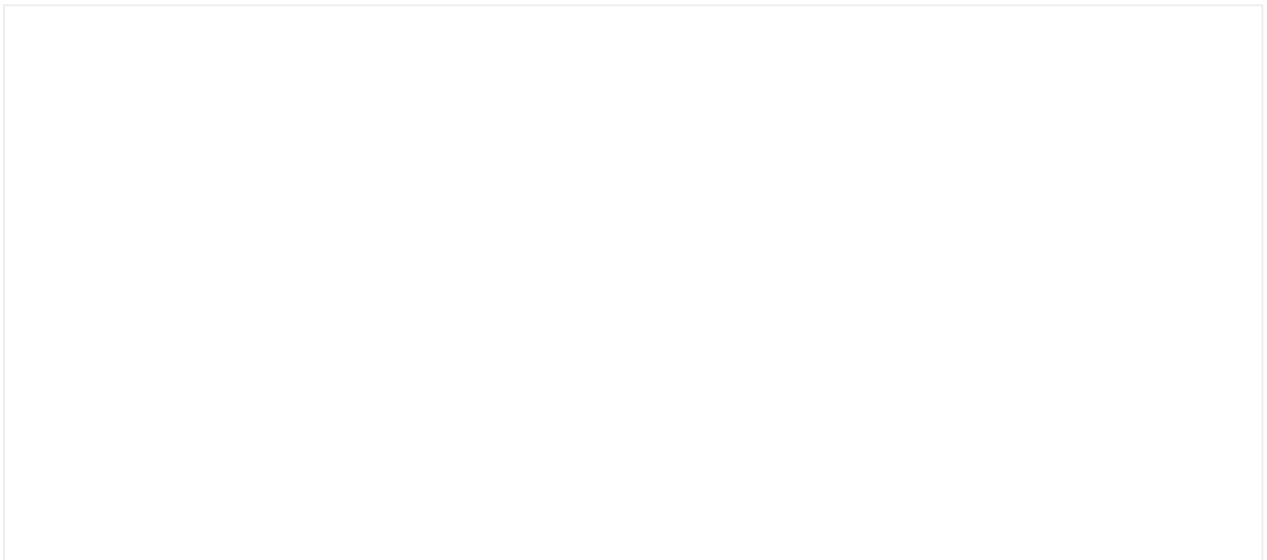
Accessibility

Implementation

Additional info

Badge

Interactive demo



What is a badge?

A badge is an element used to indicate the user that there is new information associated with any component in the app.

A badge is not an element used to indicate the user the existence of new features in the app. Another element should be used for that objective.

What is a new feature and new information in the app?

Let's explain it with examples:

- New feature: a new area in the app that displays new available plans.
- New information: a new plan is included in that area.
- New feature: a new area in the app that shows the latest mobile phone launches.
- New information: a new mobile launch is included in that area.

The badge definition is split into 2 types of badge:

- Non-numeric badge
- Numeric badge

Non-numeric Badge

Definition

The non-numeric badge is an element displayed on any component of the app to indicate the user that there is new non-urgent and impersonal information associated with it.

The non-numeric badge is used for:

- **Non-urgent and impersonal use cases**

To indicate the user that is new information available. The new available information is not personal. It is available for a group of users, not for each single user.

- **The non-numeric badge**

Should not send push notifications or other communications to the users.

Should not count in the app launcher badge.

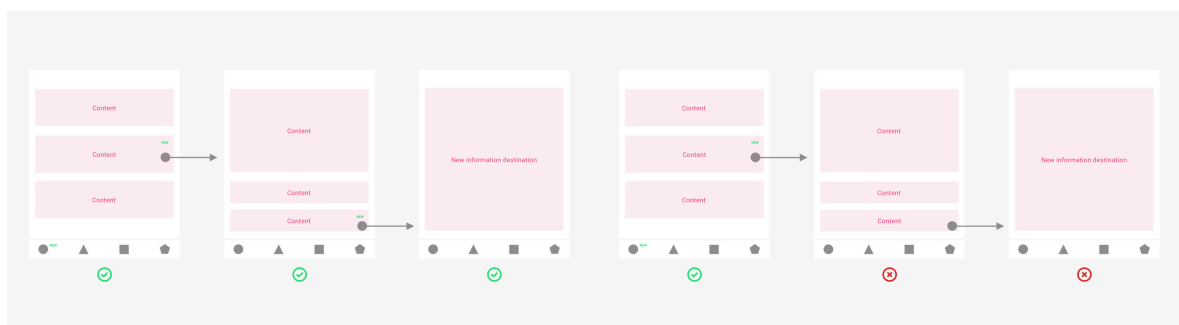
Behavior

New information is available

The non-numeric badge is displayed on the element that contains new information. If an element of the screen contains new information, the tab bar should also indicate the new information existence.

Connection between the badge and the new information

The non-numeric badge should guide the user to the new information. If the new information is placed at inner navigation levels, the badge should guide the user until the final new information destination (like a breadcrumb).



Easy to clear

The non-numeric badge totally disappears when the new information is displayed on the screen or viewed by the user.

Anatomy

The non-numeric badge has no numbers. It is recommended to place the non-numeric badge on the right or upper right side of the element that is indicating the existence of new information.

Numeric Badge

Definition

The numeric badge is an element displayed on any component to indicate the user that there is new personal and urgent information - only conversations and personal communications associated with it. **If the value of the numeric badge is 0, it shouldn't be displayed.**

The numeric badge is used for:

- More urgent and personal use cases.
- To indicate the user that is new information available. It is only available for each single user, for example, to inform of new personal communication or conversation.

The numeric badge:

- Should send push notifications or other communications to the users.
- Should count in the app launcher badge.

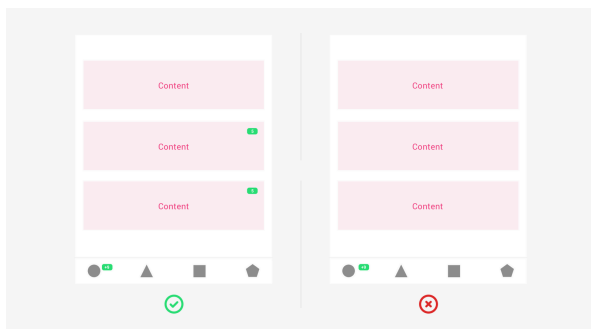
Behavior

New information is available

The numeric badge is displayed on the element that contains new information, conversations and communications.

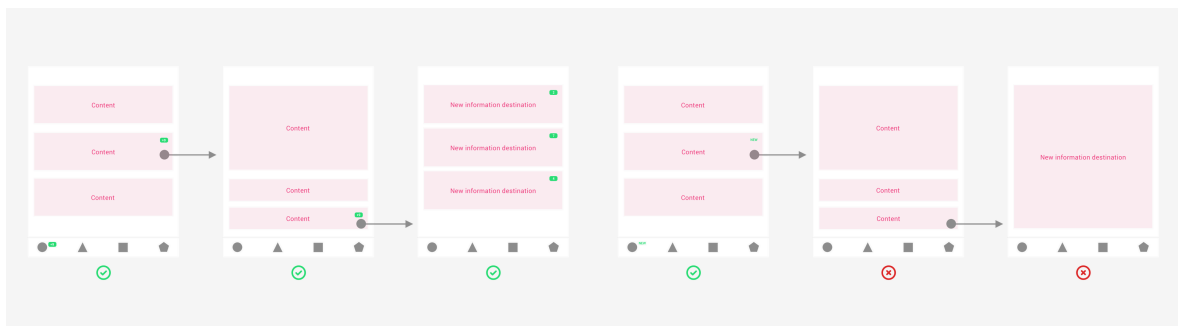
If an element of the screen contains new information, conversations and communications, the tab bar should also indicate the new information existence.

The number displayed on each tab should indicate the sum of the total number of new information, conversations and communications distributed on that screen.



Connection between the badge and the new information

The numeric badge should guide the user to the new information. If the new information is placed at inner navigation levels, the badge should guide the user until the final new information destination (like a breadcrumb).



Easy to clear

The numeric badge totally disappears when the new information is displayed on the screen or viewed by the user.

If the user has a numeric badge with number 9 and displays 5 new pieces of information, the badge should be discounted by 5 showing number 4. The same logic is replied in the app launcher.

Anatomy

The numeric badge has a maximum of 2 digits, from 1 to 9. If the number of new information is more than 9, the numeric number should display a +9.

It is recommended to place the numeric badge on the right or upper right side of the element that is indicating the existence of new information.

Implementation

Additional info

Buttons

Buttons let users do actions and make choices immediately with just a simple interaction.

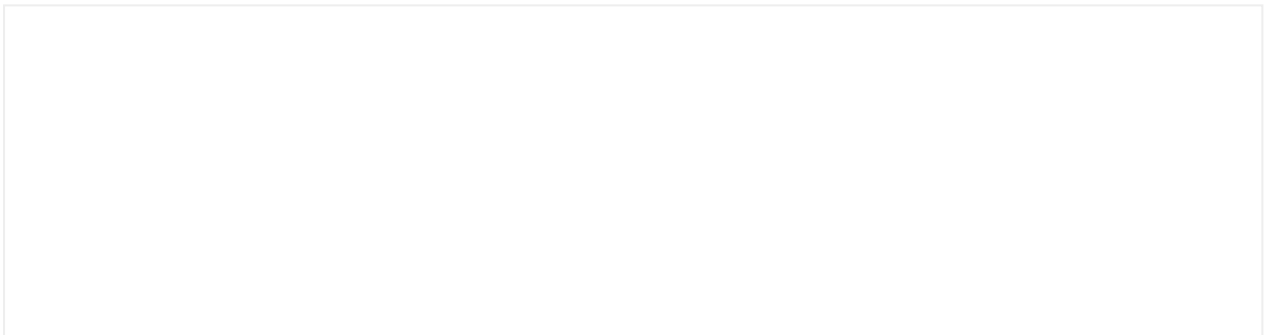


Anatomy

All button types have the following elements:

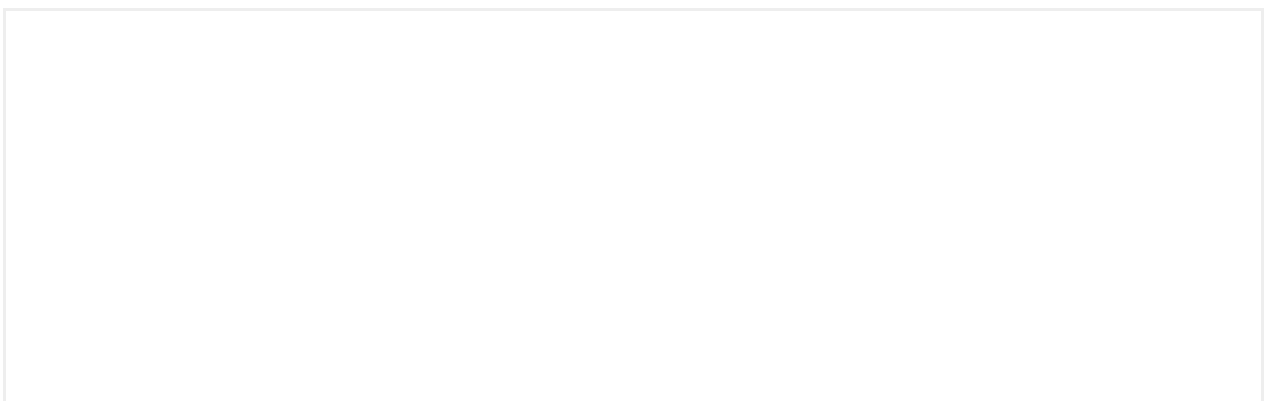
1. **Label**
2. **Container**
3. **Icon**
4. **Loading spinner**
5. **Loading label**

Leading and trailing icons



Button icons can appear before or after the label.

Small variant



Small buttons can be used depending on:

- **Spatial needs:** Usage of small buttons in compact components.
- **On-screen prominence:** Large buttons should be used sparingly, the small variant can be used to create page hierarchy and shift the focus to main actions.

Behaviour

States

A button can present the following different states:

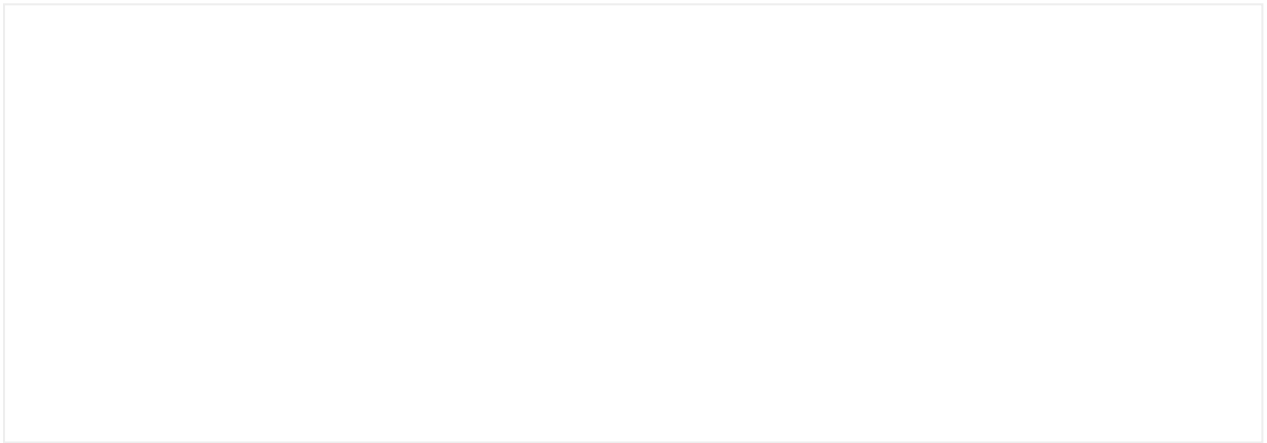
- **Normal**
- **Pressed** (Android), **Selected** (iOS) or **Active** (Desktop)
- **Hover** (Desktop)
- **Loading**
- **Focus** (this accessibility state is determined by the system)
- **Disabled**

Loading state

It's very important for users to get feedback when they interact with the buttons.

- Each button can present a loading state when sending or receiving information from the server.
- Although the loading label is not required, it can be also provided to offer more context to the user.
- The maximum time (TBD) the button will stay in a loading state should be defined.

Content



Text label

Describes the action that will happen when the user interacts with the button

- Should be always in sentence case.
- Keep the copy always short and concise

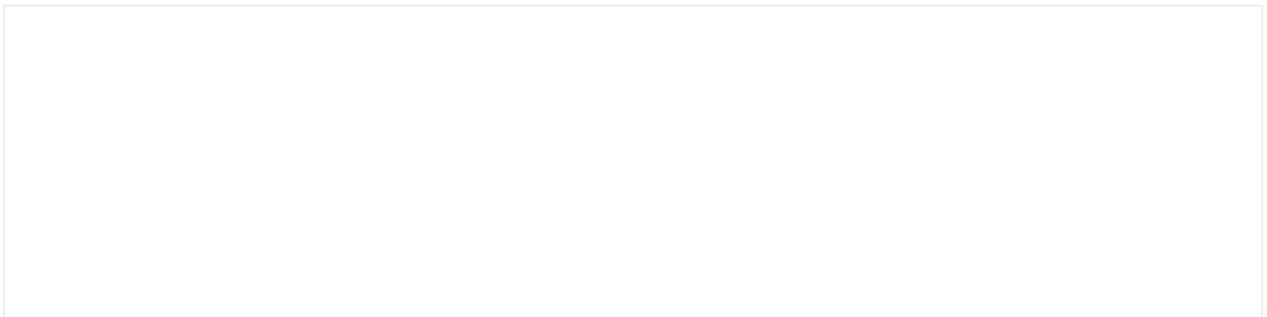
Icon

This element can accompany a text label to communicate a message more clearly. When working independently, it should have a strong visual and representational impact.

- Use established conventions, avoid icon usage if it doesn't represent the concept clearly
- The label is the best way to avoid ambiguity, think twice before adding an unnecessary icon

Button

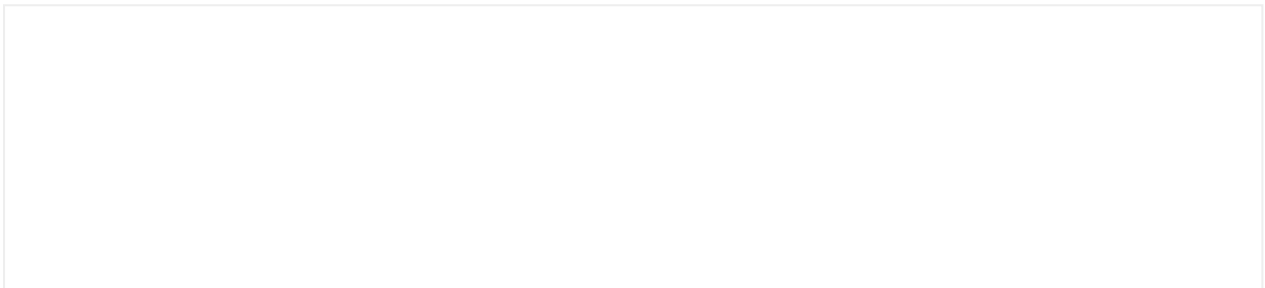
Button primary



The button primary is used to represent the most important action on the screen.

- There should only be **one primary button on a screen**.
- It should be the most used button in flows and processes.

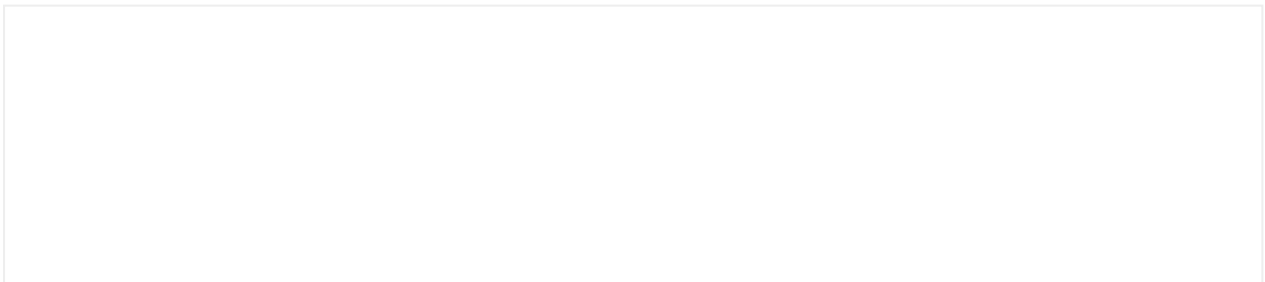
Button secondary



The button secondary has three possible uses:

- It represents a **complementary** or **alternative action**.
- Secondary actions **can appear isolated or in conjunction** with a button primary.
- They are also used to represent **non-continuing actions** that cause a return to a "0 state" of the process.

Button link

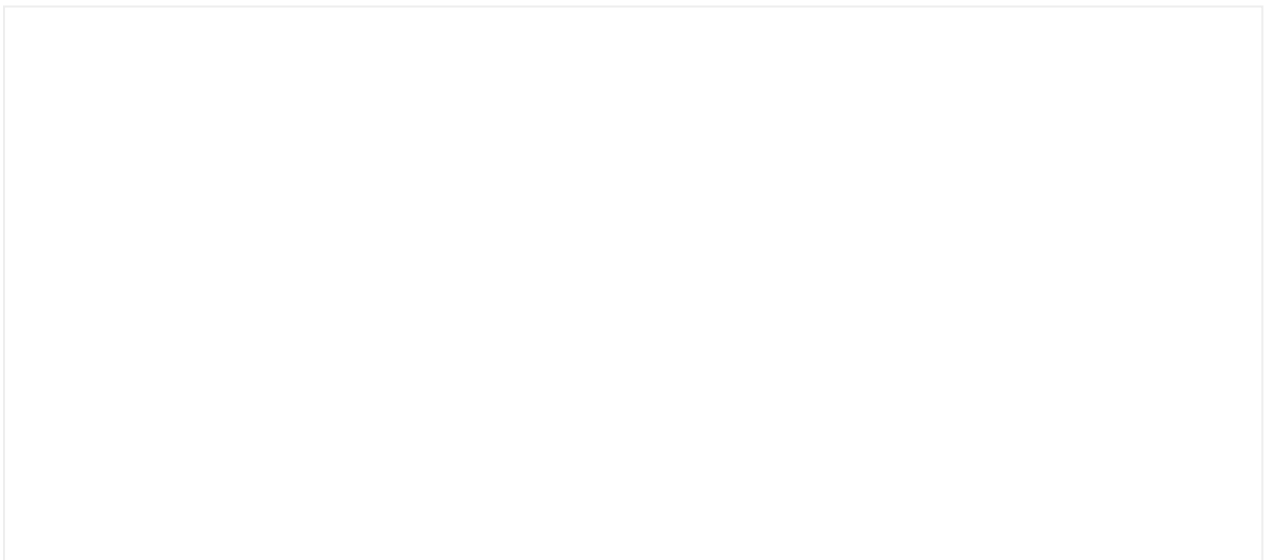


Button links are used:

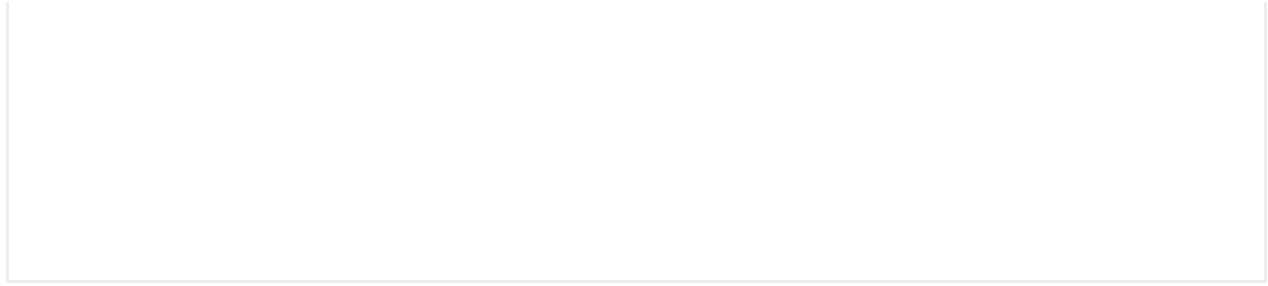
- To take users out of the current flow and **return them to the same screen**.
- To **expand information** about the current content.

Bleed

The buttonLink has different bleed properties to allow the content to be aligned with other elements in the layout.



Button danger

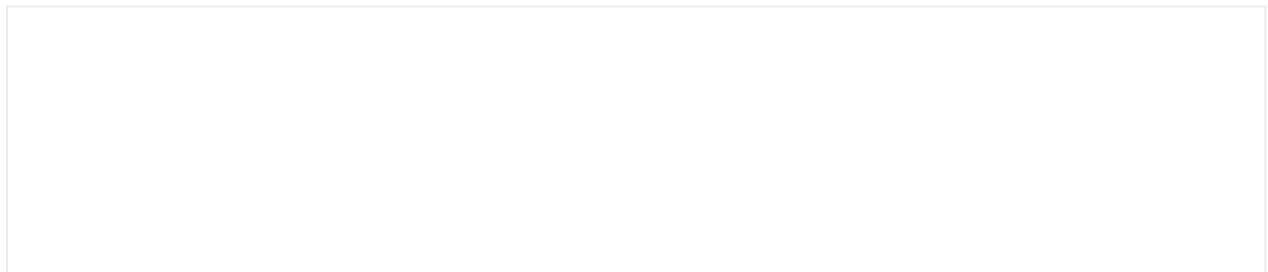


They are used exclusively for destructive actions (delete, remove, deactivate).

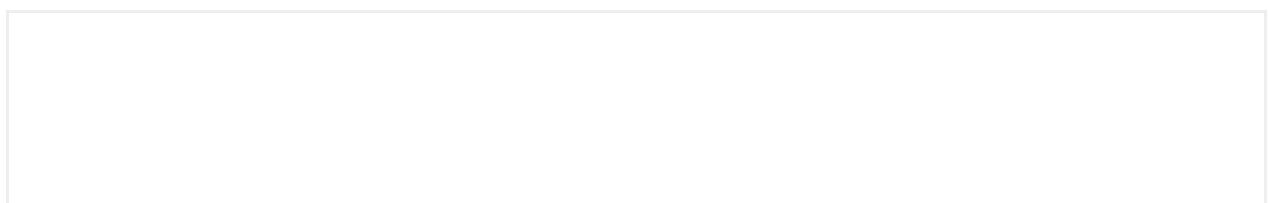
Its usage should be different depending on whether the user is on mobile or desktop:

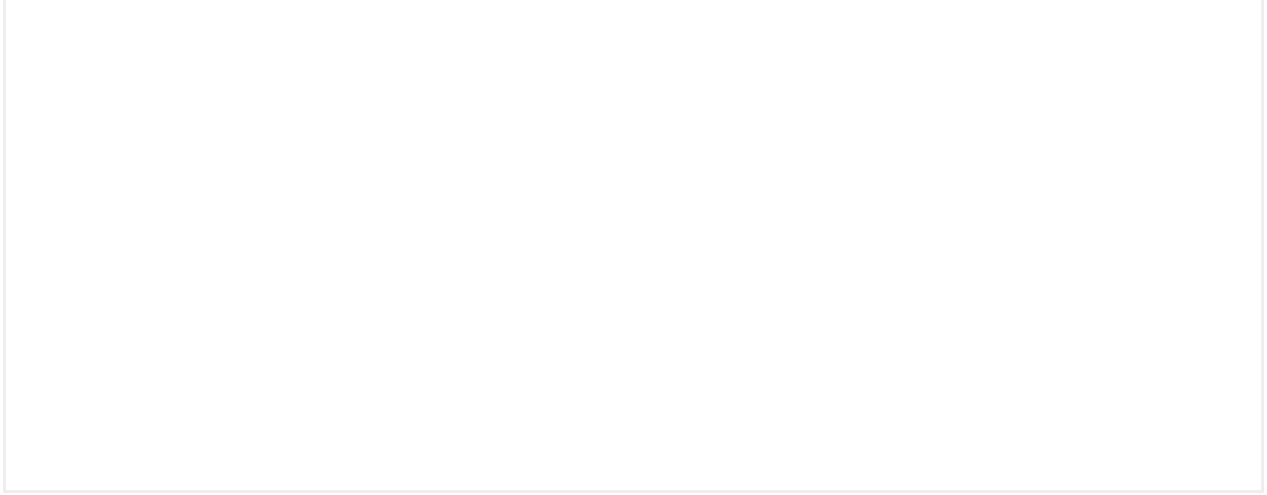
- **Mobile:** Danger buttons open the platform's native dialogue to confirm the action.
- **Desktop:** Danger buttons lead to a confirmation interface where there is another danger button with greater visual force for executing the action.

Button link danger



Is used to precede a destructive action. For example, if you don't want to draw too much attention to a destructive action you could use a Button Link Danger that opens a confirmation modal, and within the modal use a Button Danger to end that destructive action. **The last element to confirm a destructive action always has to be Button Danger.**





Icon button

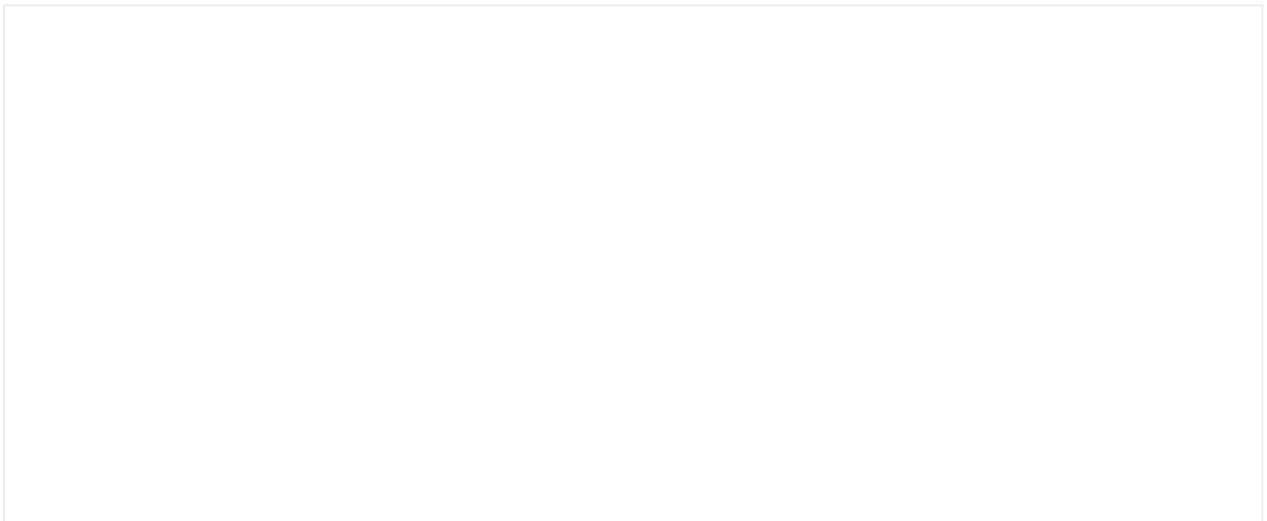


Use the icon button when there are restrictions in the available space or the action can be represented without a label.

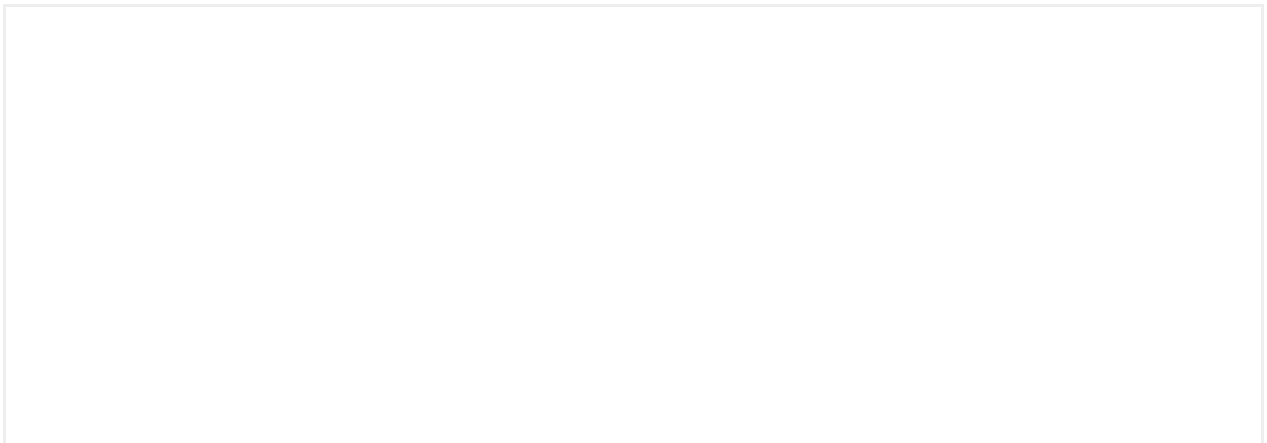
- Always provide always a descriptive `aria-label`.
- A Mística icon or an image can be used as button asset.

Bleed

The Icon button has also different bleed properties to allow the content to be aligned with other elements in the layout.



Toggle icon button





The toggle icon button allow users to change between two states (checked, unchecked) and shares the styles of the Icon button.

Usage

- `type` and `backgroundType` properties can be defined either for check or unchecked states separately.
- A Mística icon or an image can be used as button asset for each of the states.
- Always provide always descriptive aria-label for checked and unchecked states separately.
- The toggle icon button can also use `bleedRight`, `bleedLeft` and `bleedY` properties.
- For on/off states consider using a **switch**.

Customization

The icon button component allows:

- Custom assets (svg or image)



Hierarchy

The hierarchy can be achieved by giving the buttons emphasis and using the **combination** and **coexistence** of buttons on the layout.

Emphasis

When used properly, emphasis can help direct the user's attention more effectively. Emphasis can be represented on buttons like in a conversation:

Emphasis	Button
Yell !!!!	Button danger
Shout !!	Button primary
Cheer!	Button secondary
Murmur	Button link
Whistle	Icon button

Hierarchy in the layout

A hierarchy should also be established on every screen between the elements that make it up and the actions presented. Two fundamental rules are defined for that purpose:

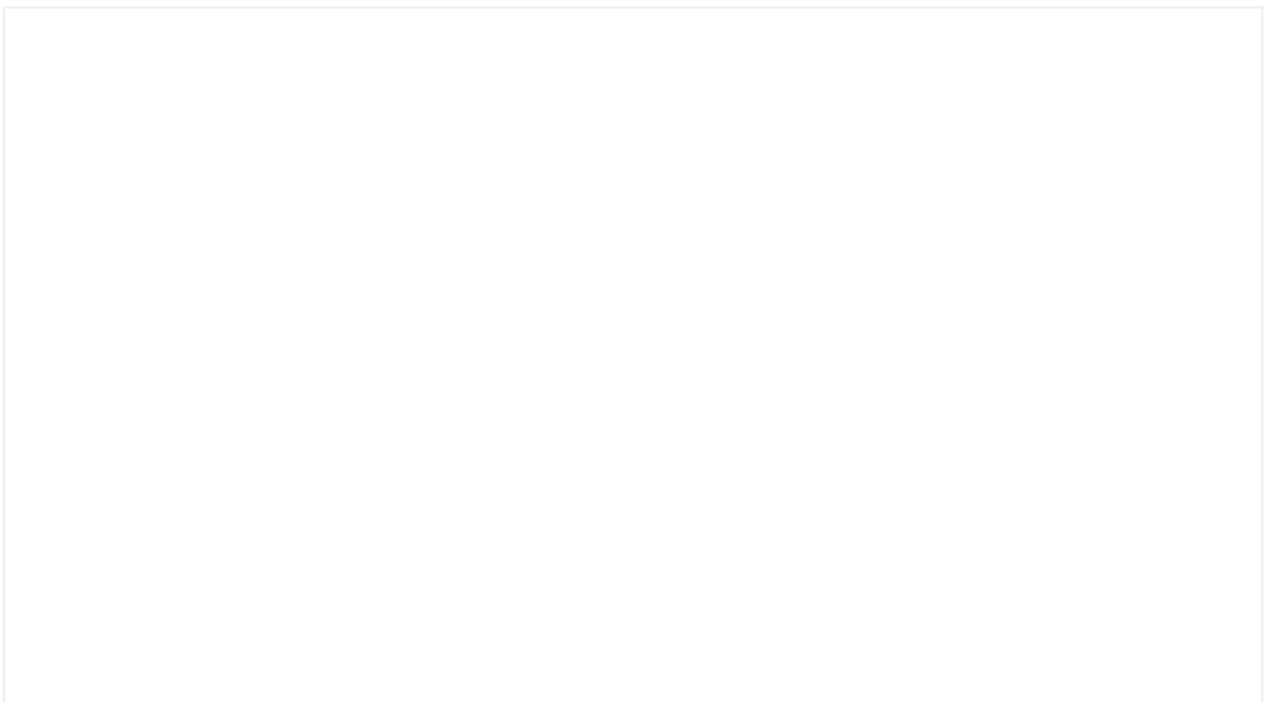
1. Every screen should have a single main button

Every layout should have a single primary button to put the user's focus and attention on the main action.

2. Other complementary buttons

The main layout action can be accompanied by other buttons with less emphasis to establish a hierarchy between actions on the screen.

When several main actions need to coexist in the same layout, for example, when there is a list of services the user can activate, the **small button** variant should be used.



Placement & alignment

Some basic patterns for positioning buttons throughout the app need to be established so an appropriate coherence and hierarchy can be maintained across all screens.

When we talk about positioning, we need to pay attention to 2 important things:

- Positioning the buttons **in relation to each other and with other interface components**.
- Button positioning **in relation to the layout**.

Button layout



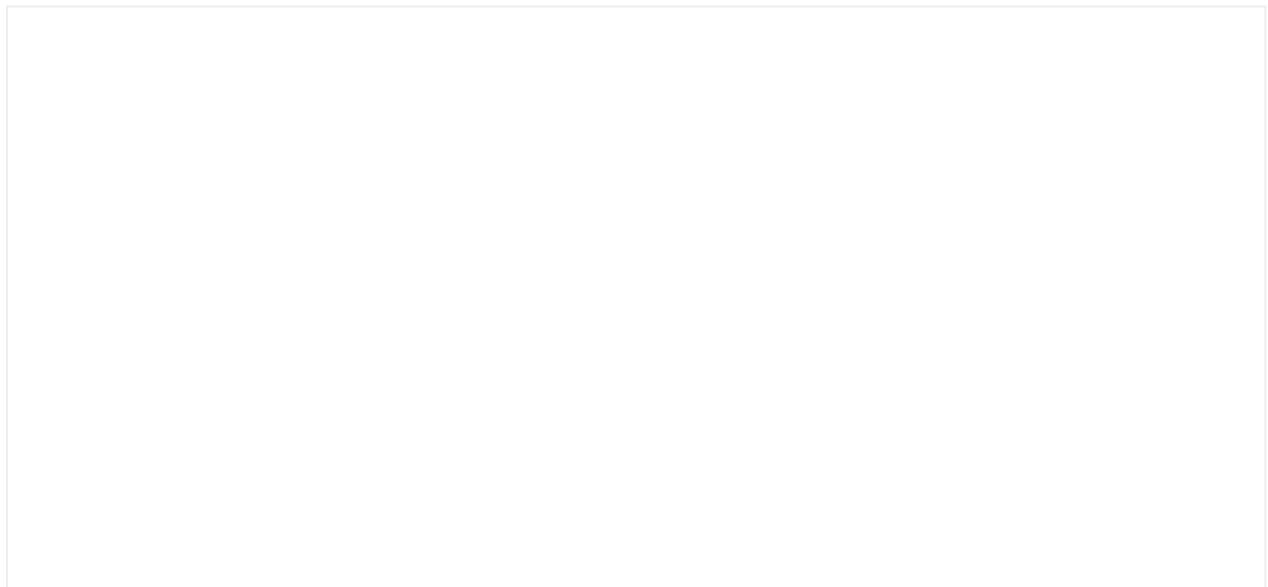
Use the button layout for actions related to a block of content or a related component.

Actions number

- It's not recommended to use more than two actions.
- When needed, the button link should be always placed below the actions.

Alignment

- The buttons should be positioned in relation to the distribution of related content.
- When there are multiple actions, the primary is always placed on the left (desktop) or top (mobile).
- When the width of the text is greater than the minimum button width, the buttons are stacked vertically.



Button Fixed Footer Layout

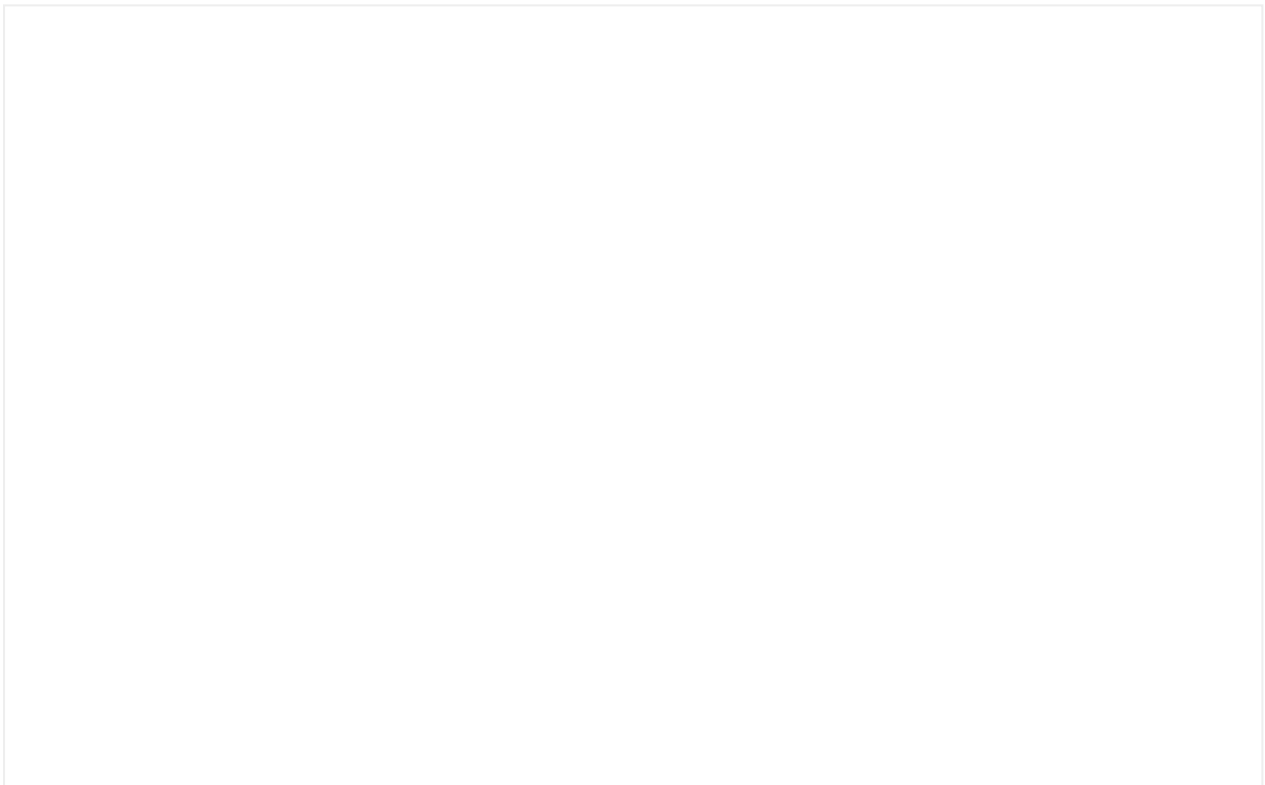




Use the button fixed footer layout for flows and processes where users need to move forward and make decisions that affect the entire screen.

- Action follows the same alignment rules as the button layout.
- Actions are placed above each other taking 100% of the width.
- If the content is greater than the screen height, the actions will be placed in a bottom-fixed container.
- When the keyboard is displayed, the actions container is placed above the keyboard.

Button group



The button group is mainly for building components.

- When there are two actions, they're always placed horizontally.
- When there are three actions, the link is placed below, in a second line.

Accessibility

Implementation

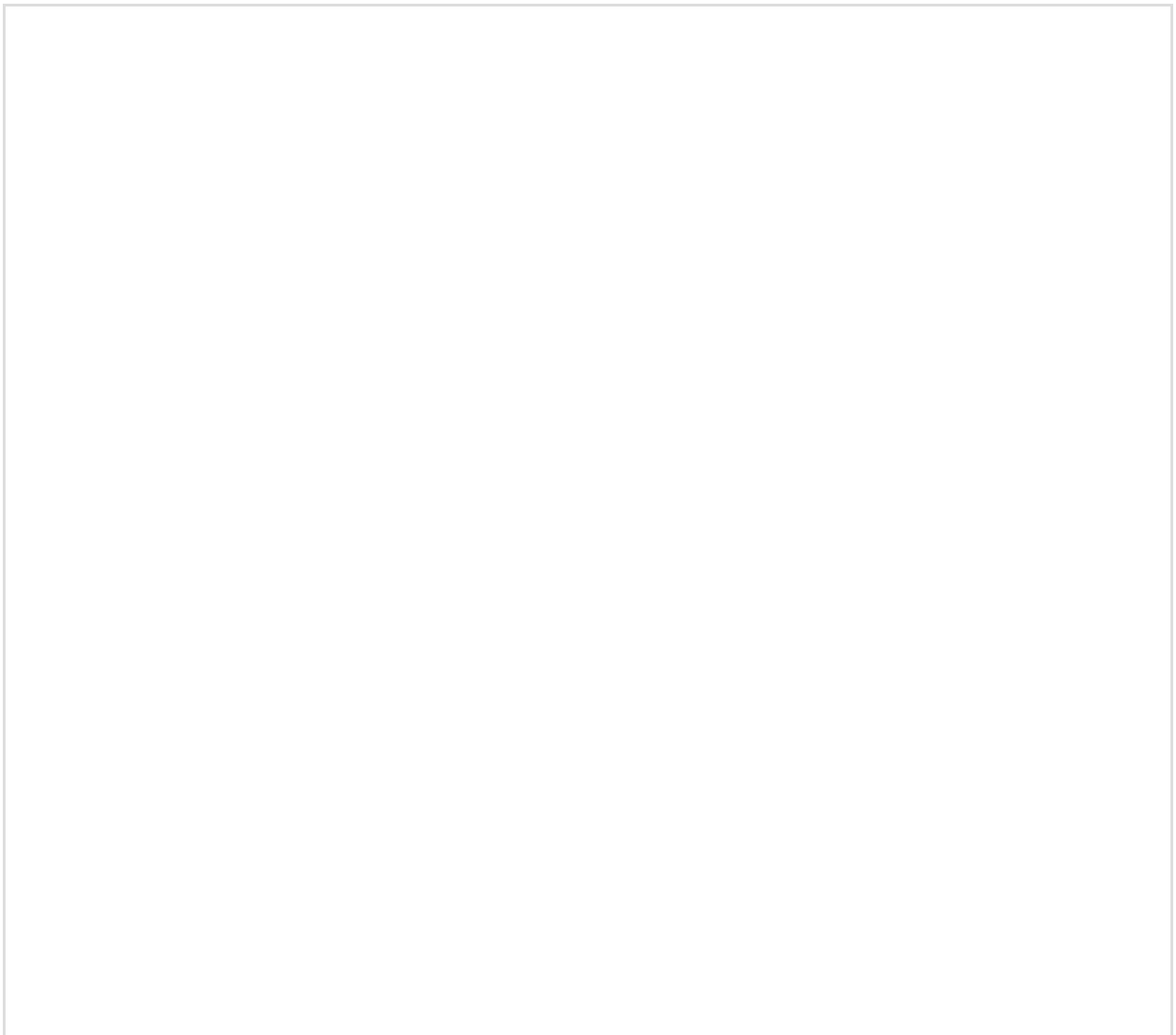
Additional info

Callout

Callouts are a snippet of information that draws attention to important content.

Anatomy

A callout can be made up of numerous elements. All of them are optional except for the description, which must always be present.

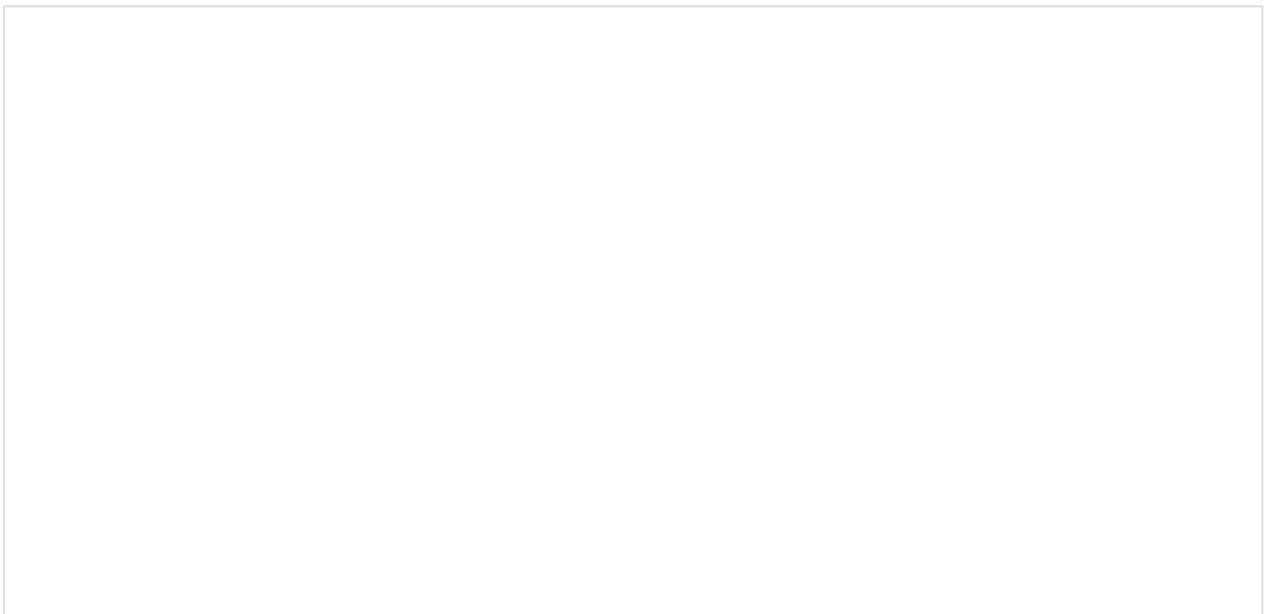


Usage

With the use of different icon and color combinations, you can create different types of Callouts to express different types of messages.

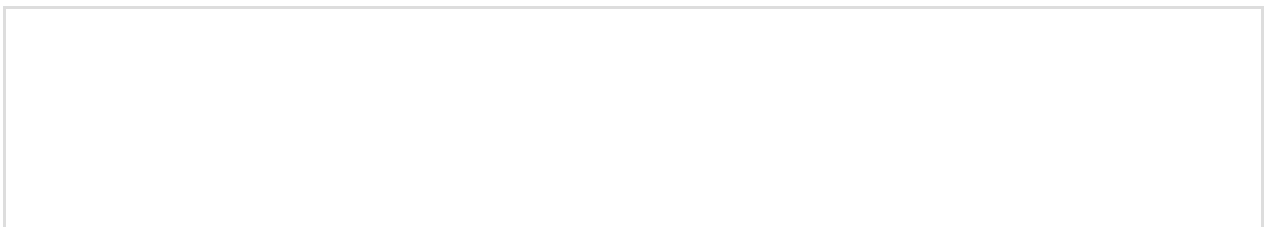
Alerts

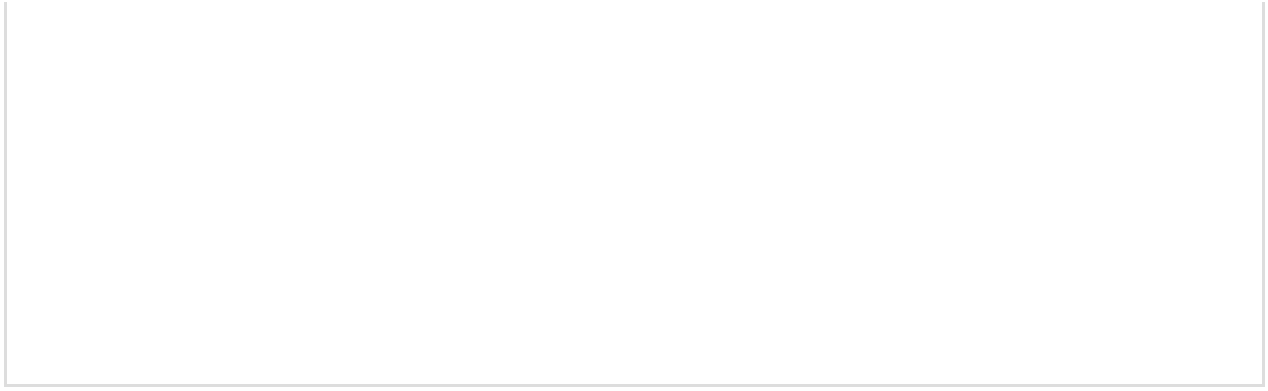
We will never use a callout as a warning feedback element. Rather, it will simply be used to highlight warning information that the user should be aware of.



Critical alert

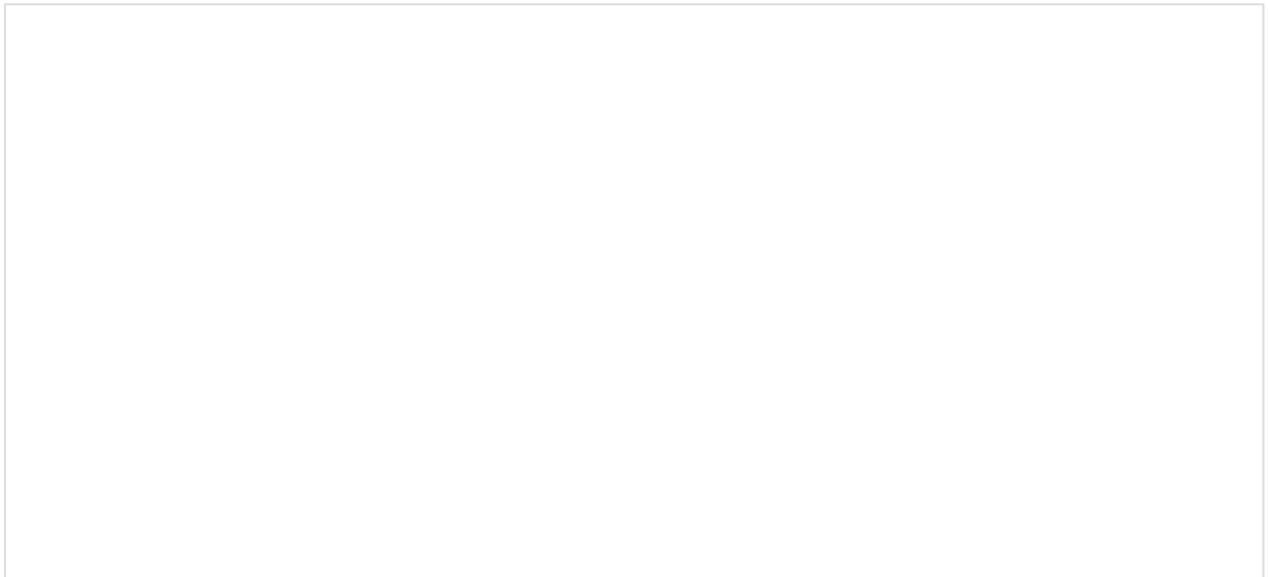
We will never use a callout as an error feedback element. Rather, it will simply be used to highlight critical information that the user should be aware of.





Informative or tip

To highlight information to the user or to give them a tip.



Accessibility

Implementation

Additional info

Cards

A card is a component that displays grouped content on a single topic. It is always displayed inside a box-shaped container.

It is used as the entry point to access more detailed information and it can also start a process for the user to complete.

Types and sizes

Common elements

All card types **share the same set of common elements**, which are optional and can be combined freely depending on the content type or communication goal:

- **Asset:** 40×40 asset such as a vector or bitmap.
- **Dismiss:** Closes or hides the card.
- **Top actions:** Customizable actions.
- **Tag:** Used to display content states. [Visit tag documentation.](#)
- **Pretitle:** Complementary information for the title (e.g., categories).
- **Title:** Include at least a title to ensure hierarchy. Keep it within 2 lines.
- **Description:** Short text giving context or details. Try not to exceed 2 lines.
- **Body slot:** Custom content area. [Check Slot section.](#)
- **Bottom actions:** Can include a link, a small button, or both.
- **Footer:** Used to separate content at the bottom of the card. Optionally visible, with customizable background and theming. Includes a **dedicated slot** for custom content. [Check Slot section.](#)
- **Media** can be an image or a video, and it can be used in two ways depending on the card type:
 - Cover Card: As a **background**, occupying the full card.
 - Media and Naked Cards: As a **positioned element**, placed at the **top**, **left**, or **right** of the card.

Aspect ratio

In **Cover Cards**, the image is used as a background, so the **aspect ratio defines the base height** of the entire card. In **Media** and **Naked Cards**, the **aspect ratio applies only to the image**, not to the total height of the card.

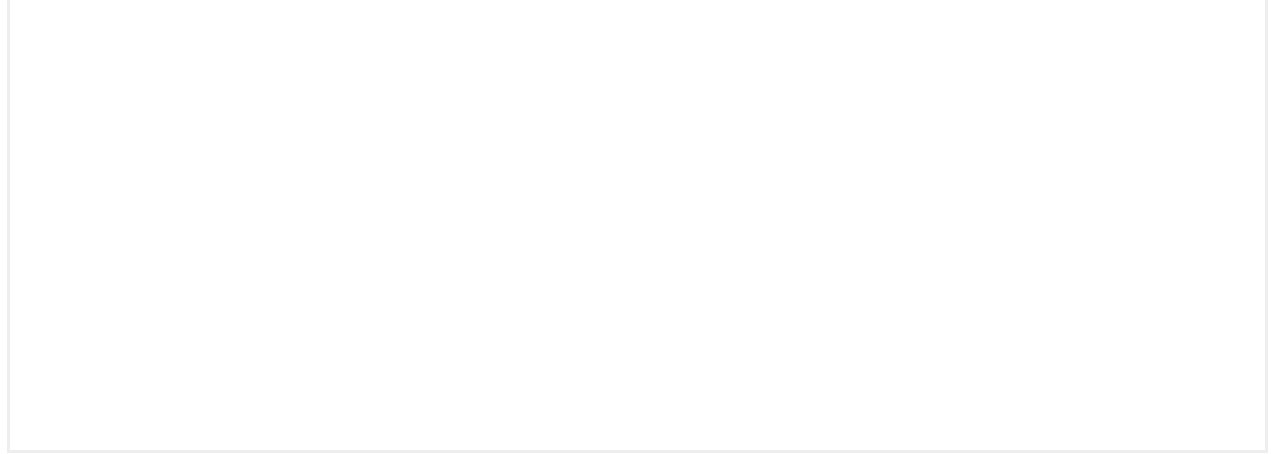
If the content height exceeds the height defined by the aspect ratio, the card will grow independently to accommodate the content.

The supported aspect ratios are: **16:9**, **21:9**, **7:10**, **4:3**, **1:1**, or **free**, which allows the height to adapt to the content with no fixed proportion.

Cover Card

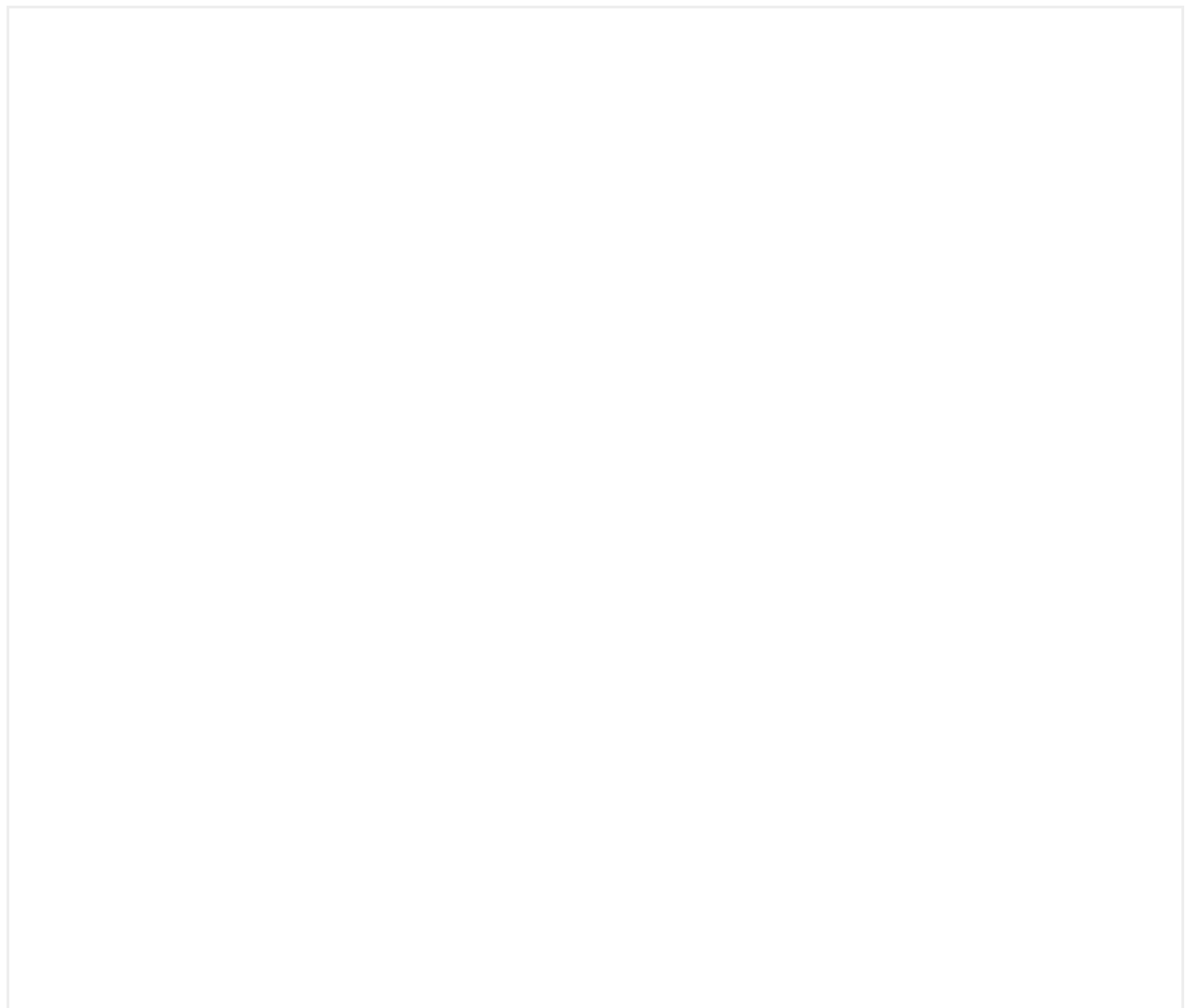
It has been is designed to create strong visual impact and draw the user's attention to imagery or audiovisual content.





Media Card

It is used to display highlighted content and it is always accompanied by an image.





Media position

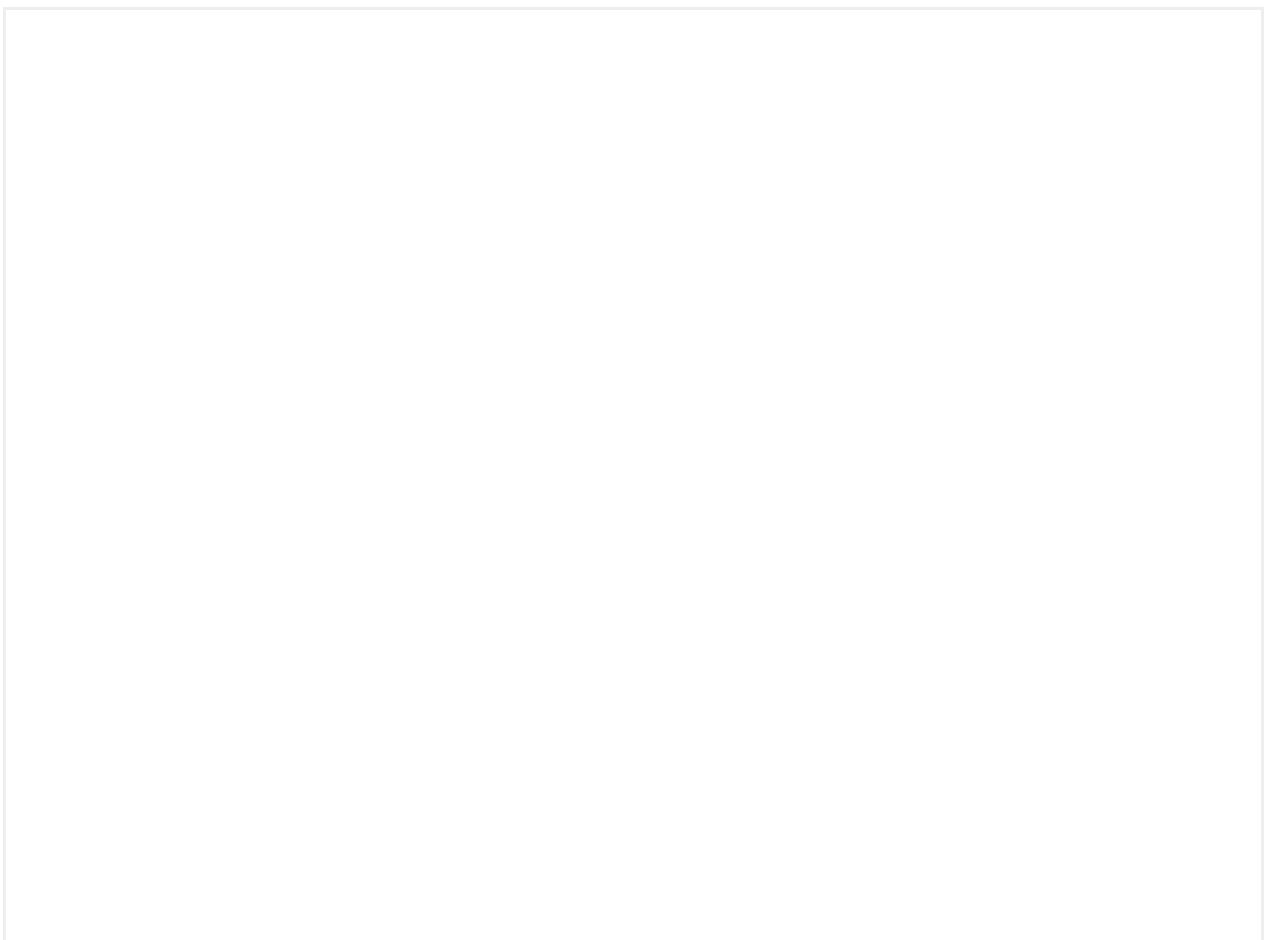
This card type will allow to change the position of the media element with the following options:

- Top positioned (default)
- Left
- Right

Media card examples

Data Card

This card is specifically designed to display relevant information about the product or service for the user.





Naked Card





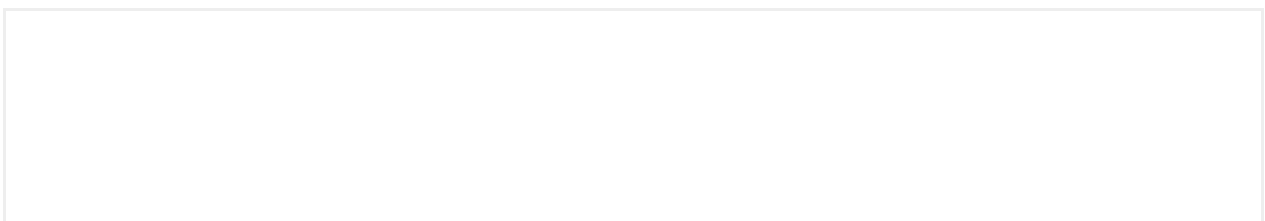
Media

The visual content can be either an image or a video, but unlike other card types, this variant **supports circular images** (not available for videos).

When using circular images, we do not recommend including top actions, as the layout may become visually unbalanced or constrained.

Both, images and videos can be used as positioned elements, with the following options: top, left, or right.

Interaction



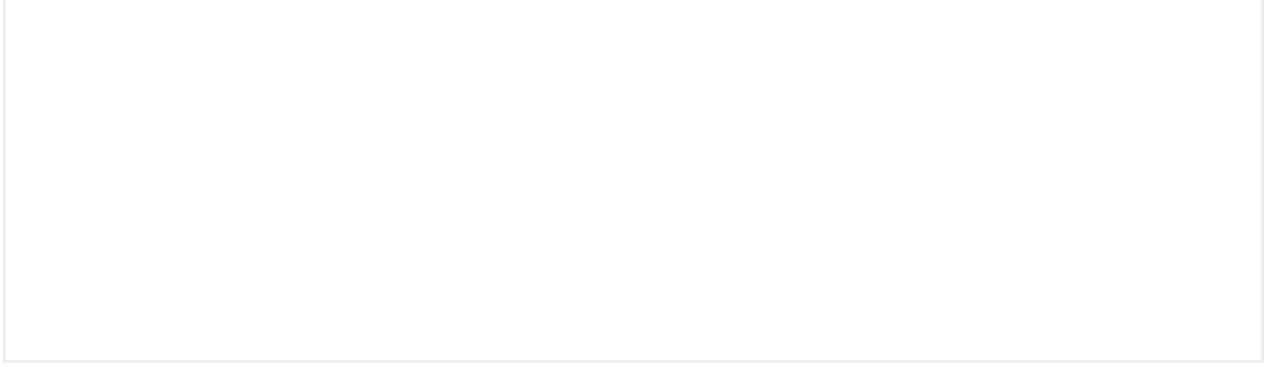
Color customization

Card grouping

When cards are grouped, their height adjusts to match the tallest one in the group. In these cases, content alignment becomes important to maintain visual consistency across variants and sizes.

Depending on the card type and size, the content aligns to the top or bottom of the available space as the height grows:

- **Cover Card** always aligns content to the **bottom**.
- **Media** and **Naked Card** always align content to the **top**.
- **Data Card** aligns **bottom** in Display, **top** in Default and Snap.



When cards are inside carousels they also modify their size and alignment properties, check the [carousel documentation](#) for more information.

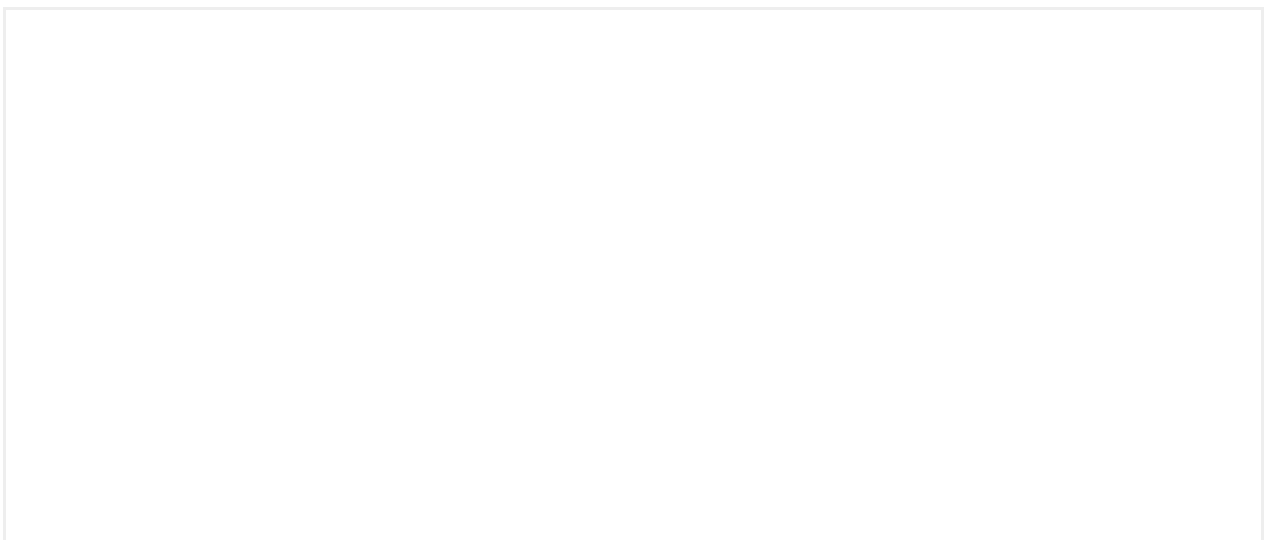
Slot

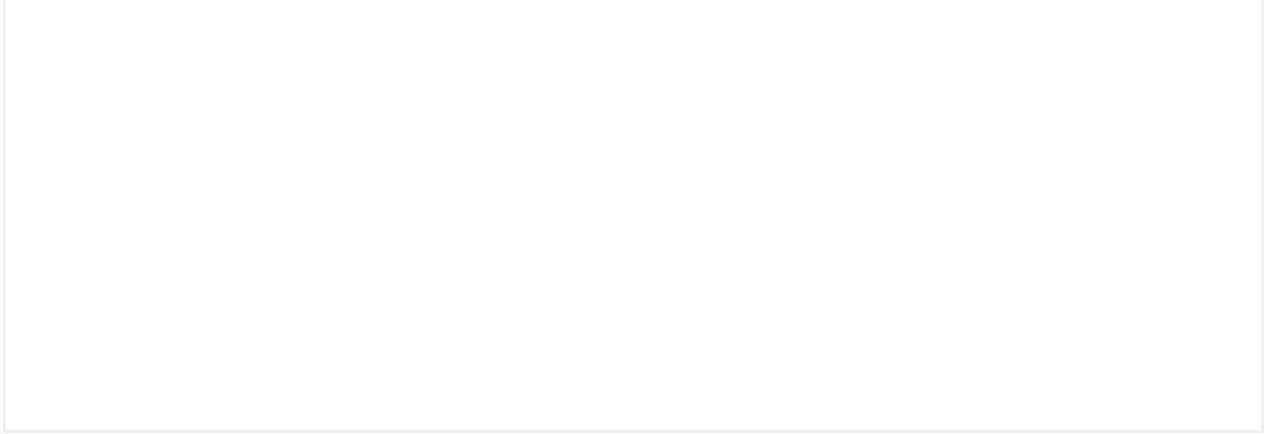
Slots inside the component allow the customization of the component content.

Their main function is to offer flexibility and scalability for a component to be adapted to the specific requirements of each product.

They are inserted in the body of the card, between the content and the actions.

Examples of Slots in cards





Accessibility

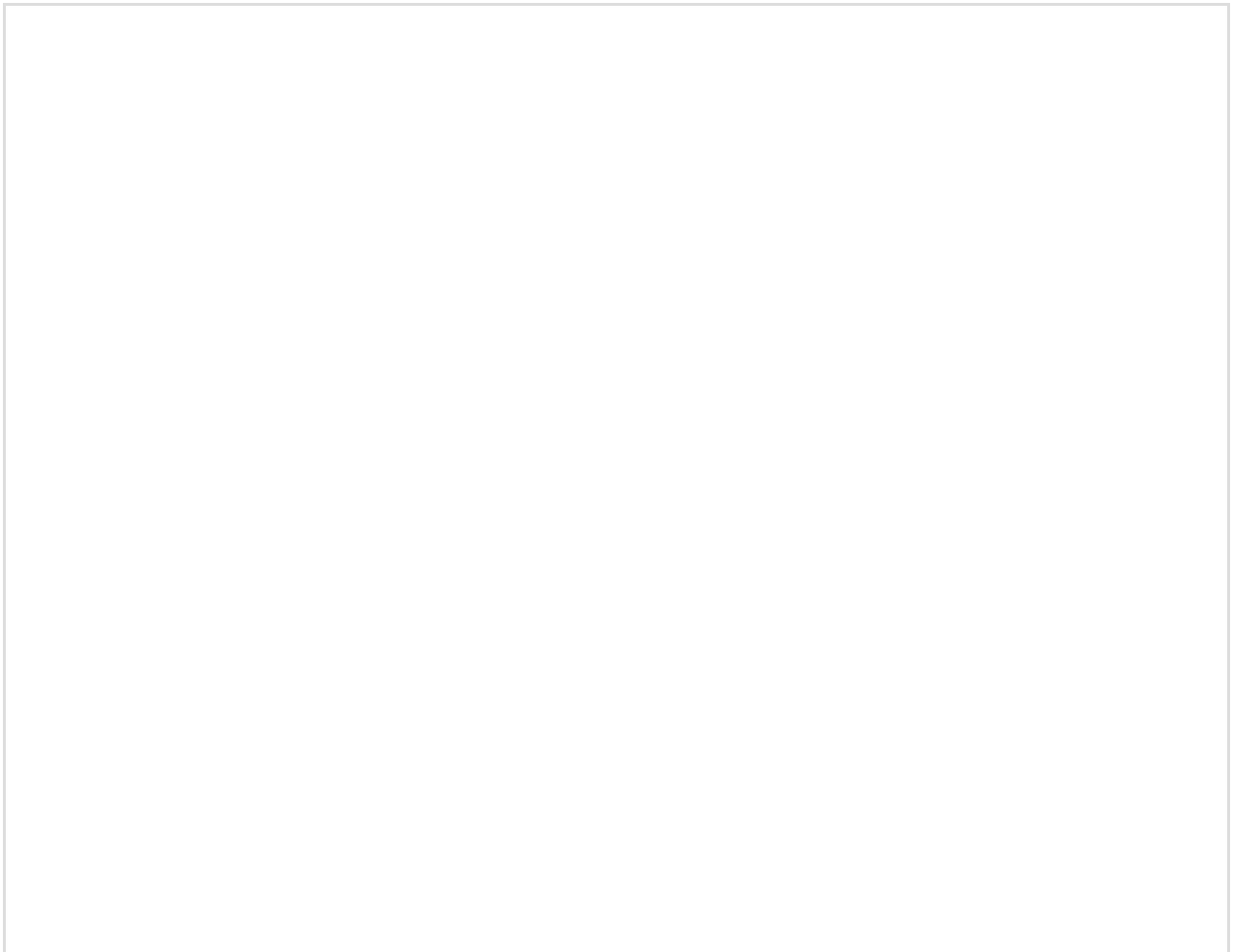
Carousels

Carousels allow several elements to be grouped together in a reduced space with the help of a horizontal scroll.

There are three types of carousels:

- Carousel (the standard one)
- Centered carousel
- Slideshow

Carousel





A carousel has a wide range of configurations that allow you to adapt the component to different needs. **Remember to provide these configurations to your development team!**

Free

The user will be able to scroll freely, i.e. the elements do not conform to an anchor point.

Paged

Makes the scroll of the carousel 'paged'. This causes the items to be anchored to a certain position on the screen and for there to be some friction when moving to the next item.

Autoplay

Causes the carousel to change items automatically (5 seconds by default).

Loop

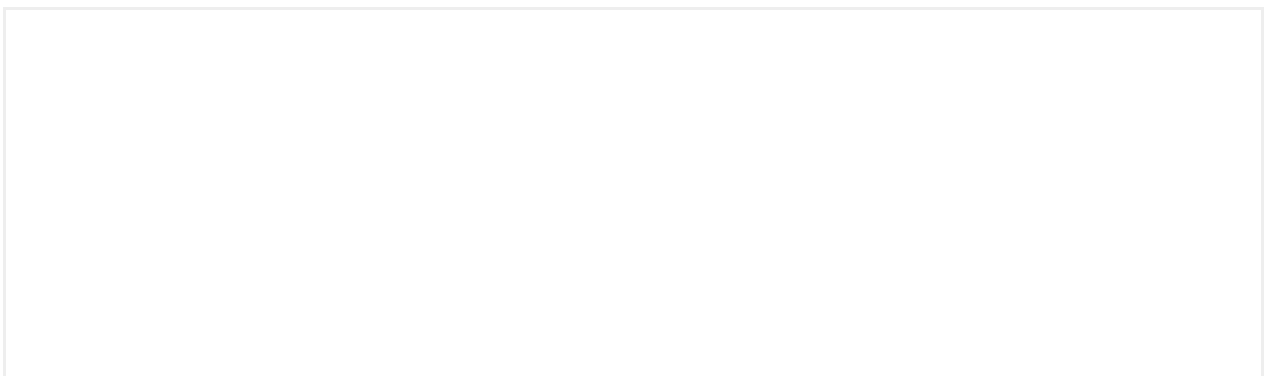
Means that the carousel has no start and no end; when you scroll past the last item you will return to the first one. This option is only available when the carousel is in autoplay mode to prevent it from stopping at the last item.

Page Offset

Manages the quantity displayed by the next item. There are two possibilities Regular and Large.

Bullet points

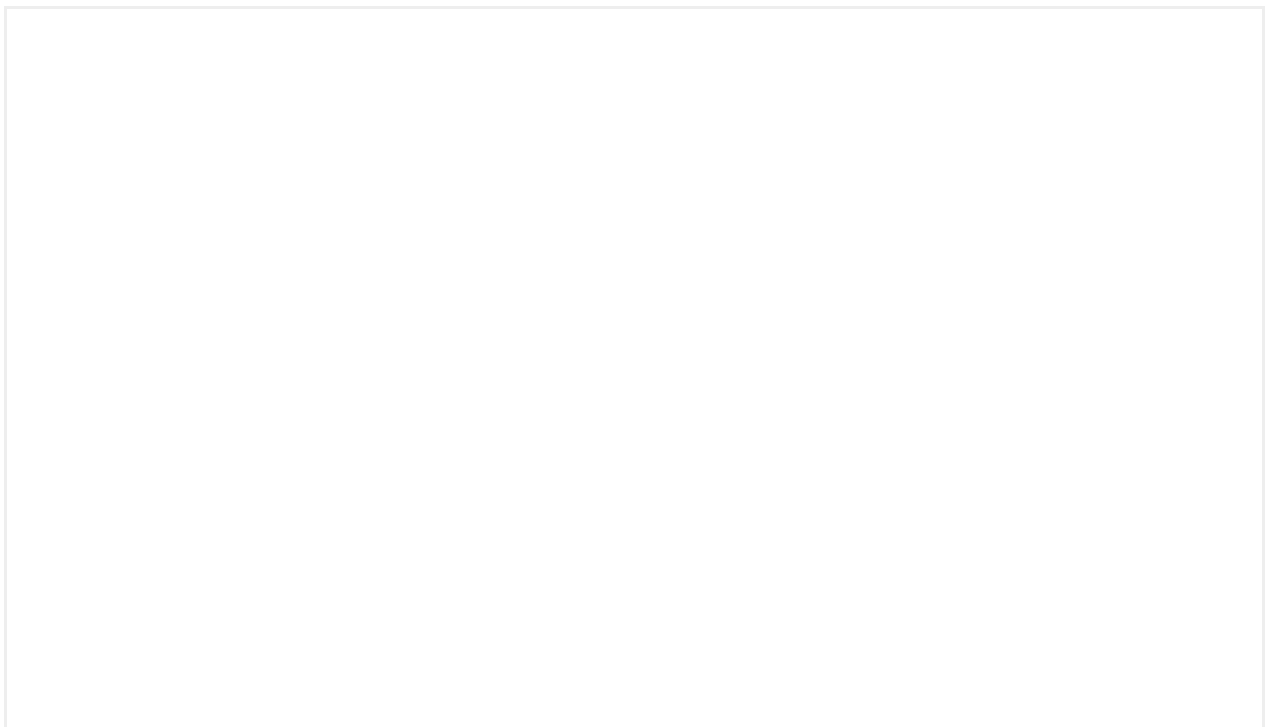
You can show or hide the circles in the carousel navigation.





Slideshow

Full-width image carousel.



A carousel has a wide range of configurations that allow you to adapt the component to different needs. **Remember to provide these configurations to your development team!**

Free

The user will be able to scroll freely, i.e. the elements do not conform to an anchor point.

Paged

Makes the scroll of the carousel 'paged'. This causes the items to be anchored to a certain position on the screen and for there to be some friction when moving to the next item.

Autoplay

Causes the carousel to change items automatically (5 seconds by default).

Loop

Means that the carousel has no start and no end; when you scroll past the last item you will return to the first one. This option is only available when the carousel is in autoplay mode to prevent it from stopping at the last item.

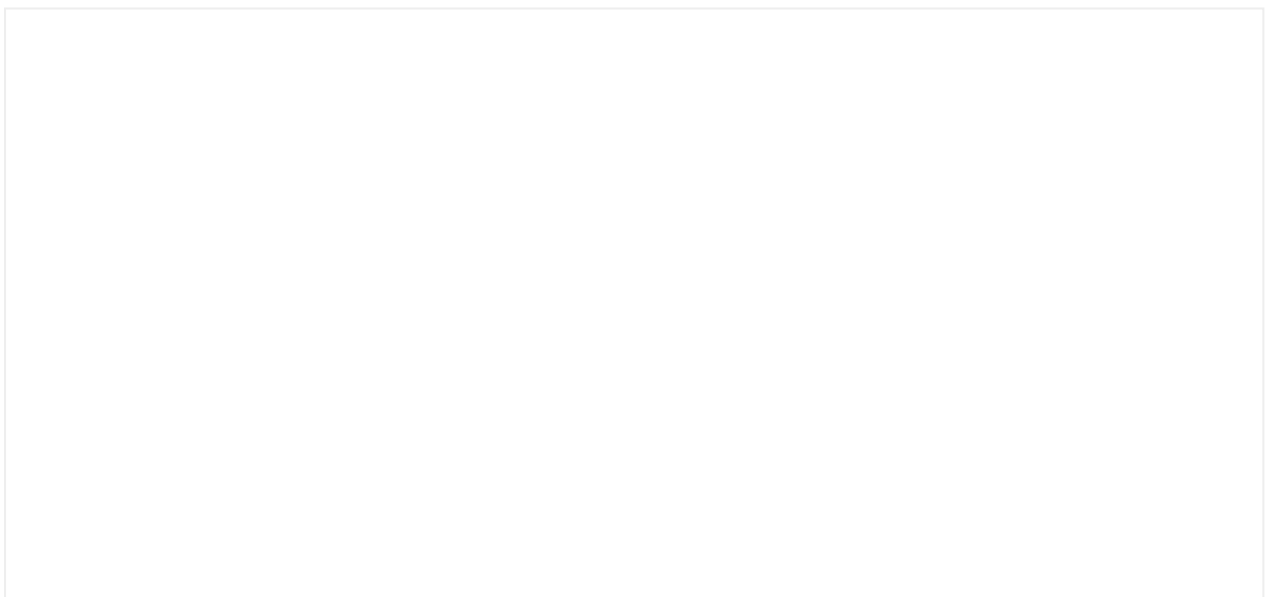
Bullet points

You can show or hide the circles in the carousel navigation.



Centered carousel

Carousel of centered elements.



This carousel forces the elements to appear centered on the screen.

Free

This carousel cannot be free.

Autoplay

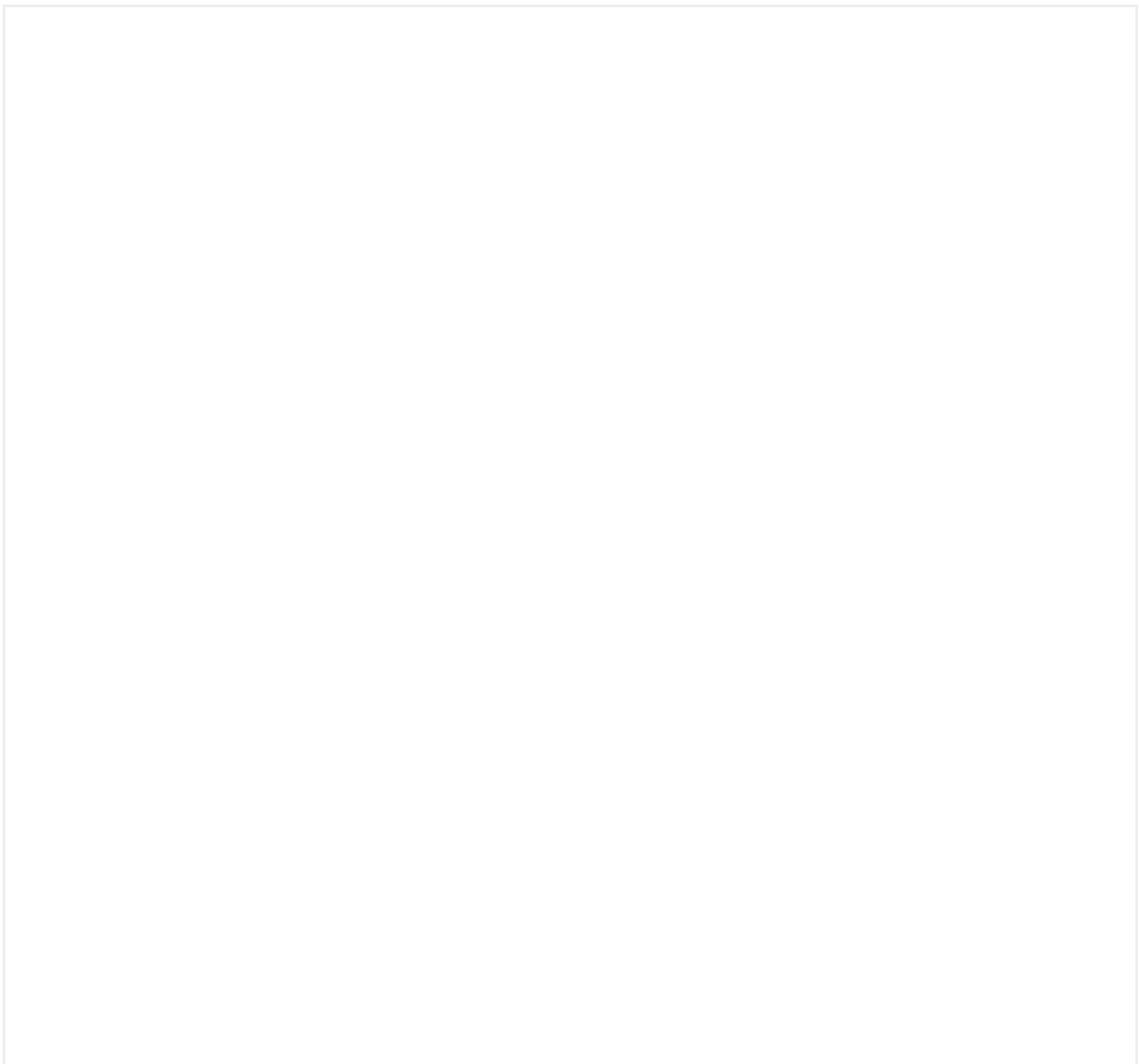
This carousel does not have autoplay.

Loop

This carousel has no loop.

Bullet points

You can show or hide the circles in the carousel navigation.



Custom controls



Implementation

Additional info

Checkbox

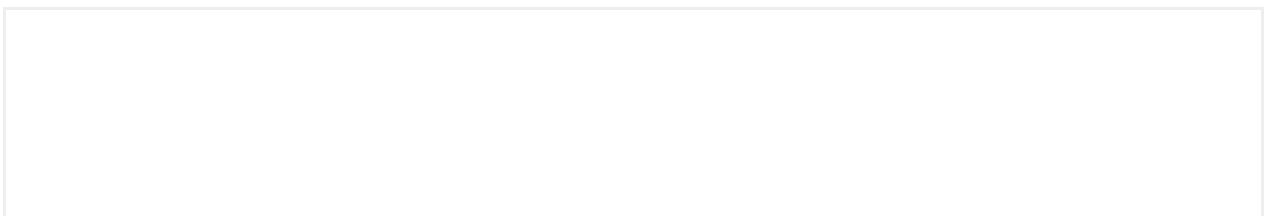


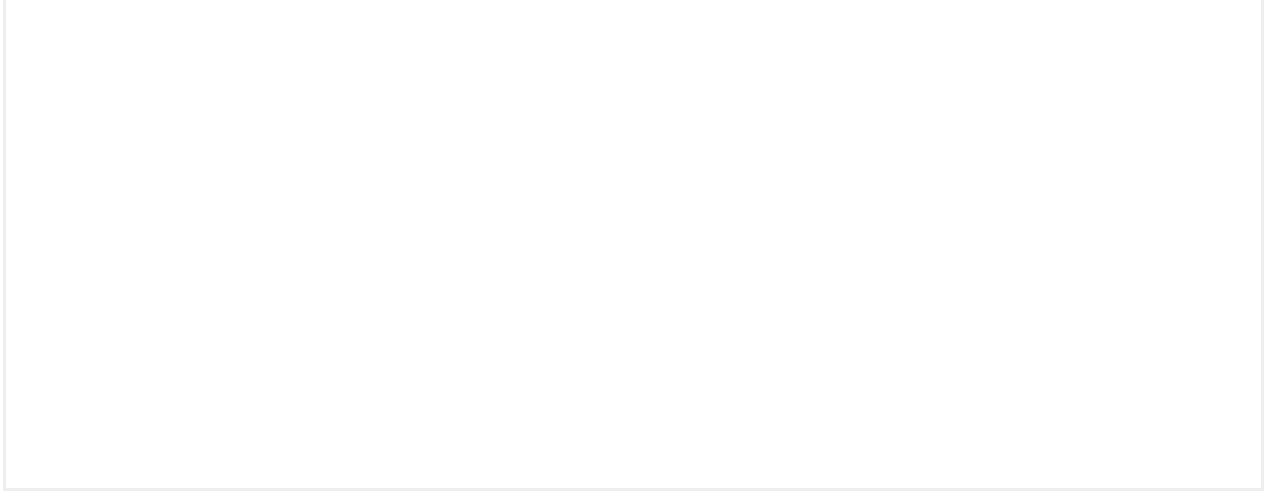
This element will be used if we have a list of options in which the user can select one, several or no options. This means that each option (checkbox) is independent from the others and that, for example, selecting a checkbox does not deselect any of the others.

If we only use a checkbox for accepting or rejecting conditions, this checkbox will work in a binary manner.

Errors in Terms & conditions checkboxes in forms

Error in forms with Terms & Condition checkboxes always shows alert/dialog component when user doesn't mark this checkbox.





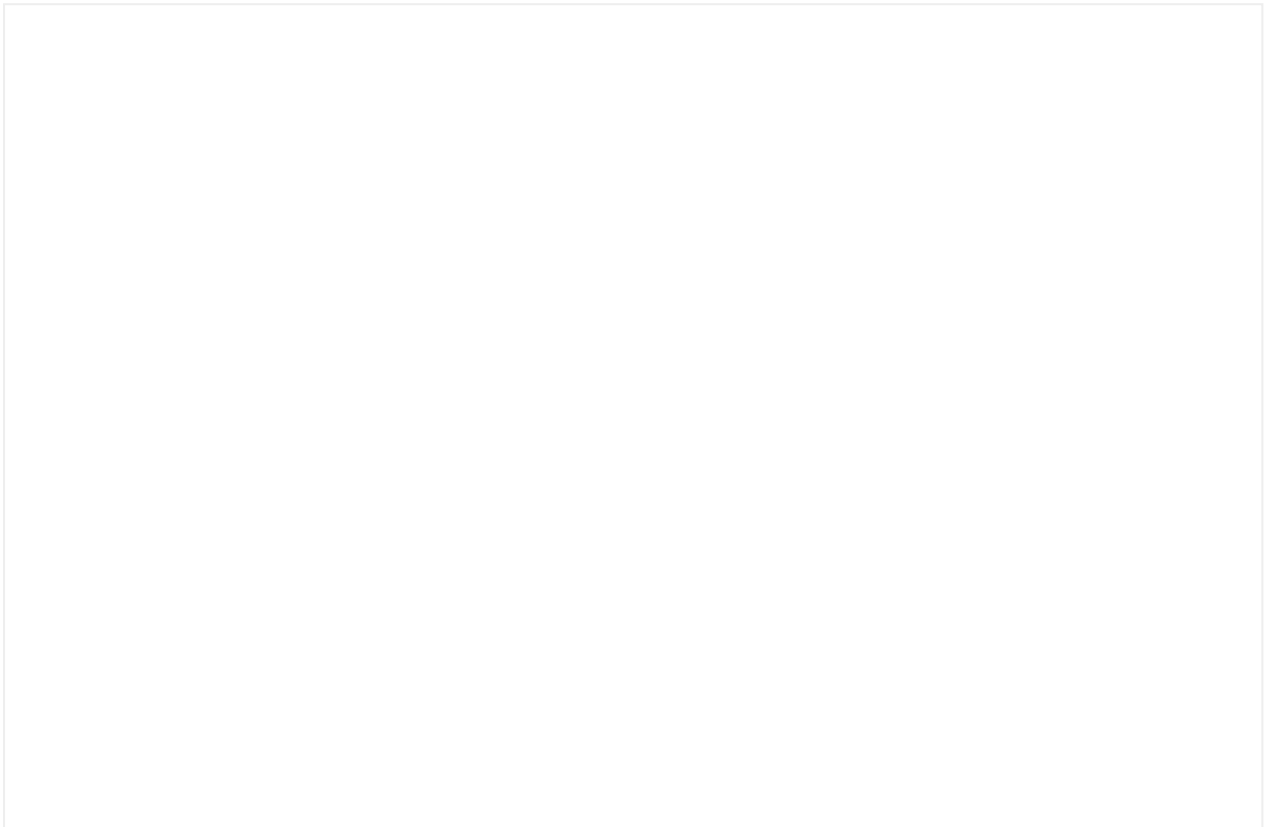
Implementation

Additional info

Chips

Chips are interactive elements that provide users with a set of options. Users can tap a chip to make selections.

Interactive demo



The chips may be composed by the following elements:

Container: The chip container defines the boundary of each chip. All chip elements are wrapped

in a chip container. The width of the container depends on the length of its content.

Text label: Text label describes what each chip stands for. Text labels should be concise.

Icon (optional): It may contain an asset of 16x16, such as vectorial icon or a bitmap.

Closable icon (optional): This area is interactive. A Closable icon to remove the chip.

Badge: A badge component can be added to the chip to describe important or new information available (Mostly oriented to the navigation chips type).

Chips must contain at least a a container with a text label.

Usage

Chips allow users to make selections, filter content, organize information or trigger actions. For example, we can select multiple chips from a set of options (filtering), or only one chip can be selected in a group of chips (selecting).

Chips have different types of uses:

- Filter chips
- Choice chips
- Input chips
- Navigation chips

Filter chips (as checkboxes)

This type of chips function as checkboxes, meaning that, in a group of chips, users

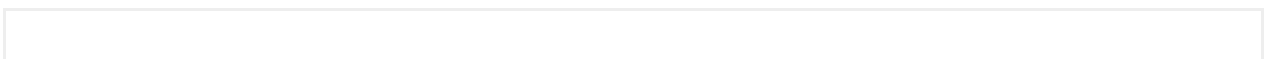
will be able to select as many as they want. This will be helpful for them because they can create search filters to select the elements they want to filter.

Choice chips (as radio buttons)

Chips can function as a group of radio buttons, in other words, the user can select only one option from a group of chips. This is useful in those situations where we need the user to choose only one of the possible options

Input chips

A chip can also be used for cases in which a user enters information and we want this to be reflected. For example, when using keywords on a screen which allows the user to add or delete those keywords.



Navigation chips

A chip can be used as navigation entry-point.

Accessibility

Implementation

Additional info

Counter

A counter is a component used to increase or decrease a numeric value.



Usage

- Is recommended to provide a meaningful default value
- Use the remove button when the element or elements the counter is making reference of can be removed (e.g. a checkout cart)
- Make sure the counter purpose is understandable by context or provide a label
- Ensure the user have information if a minimum or maximum values different from default are defined in order to prevent user error.

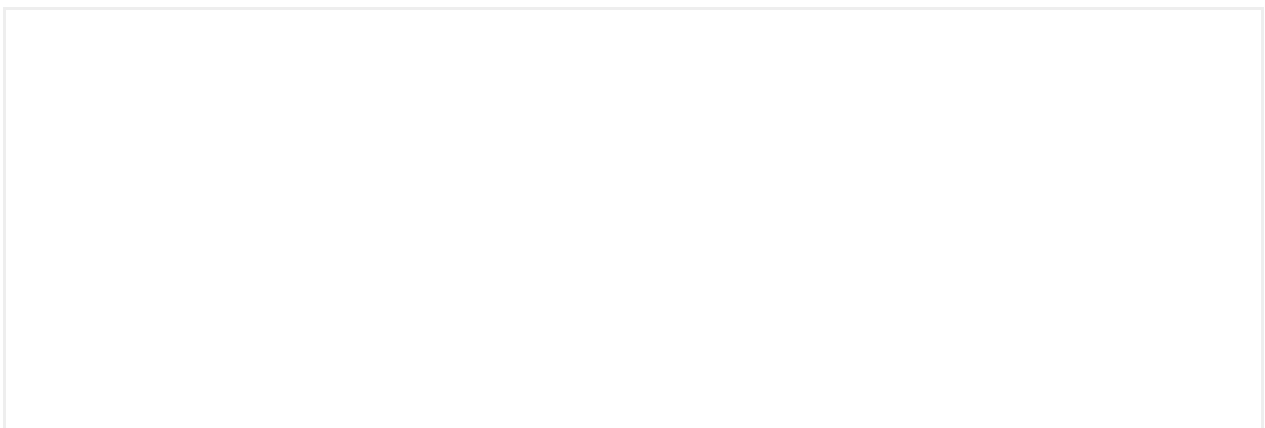
Use a counter when:

- The value range expected is small, otherwise use a integer field or text field
- Users will seldomly deviate from the default option

Anatomy

- **Increase and decrease buttons**
- **Value text**
- **Remove button** (Optional)

Remove behaviour



Accessibility

Data Visualizations

Meter

A meter chart is a representation of quantitative values within a predetermined range. It can be a linear, angular, or circular representation.



Typology

Anatomy

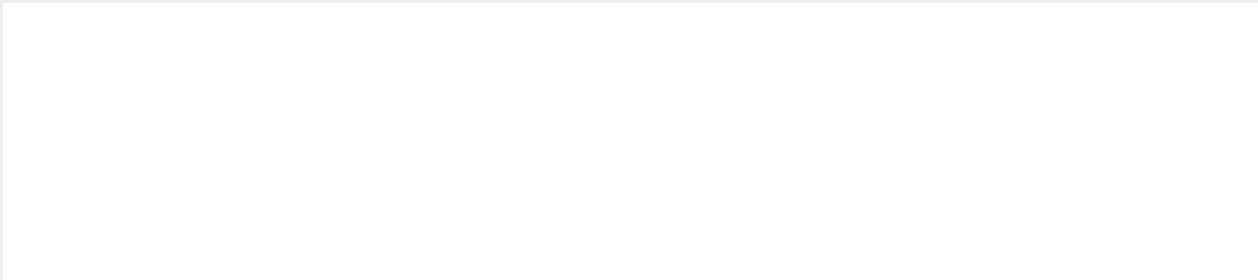


1 Segments

2 Bar track

Usage

Behavior





Slot



Accessibility

Ensure all information provided by the meter is also available in text.

Implementation

Additional info

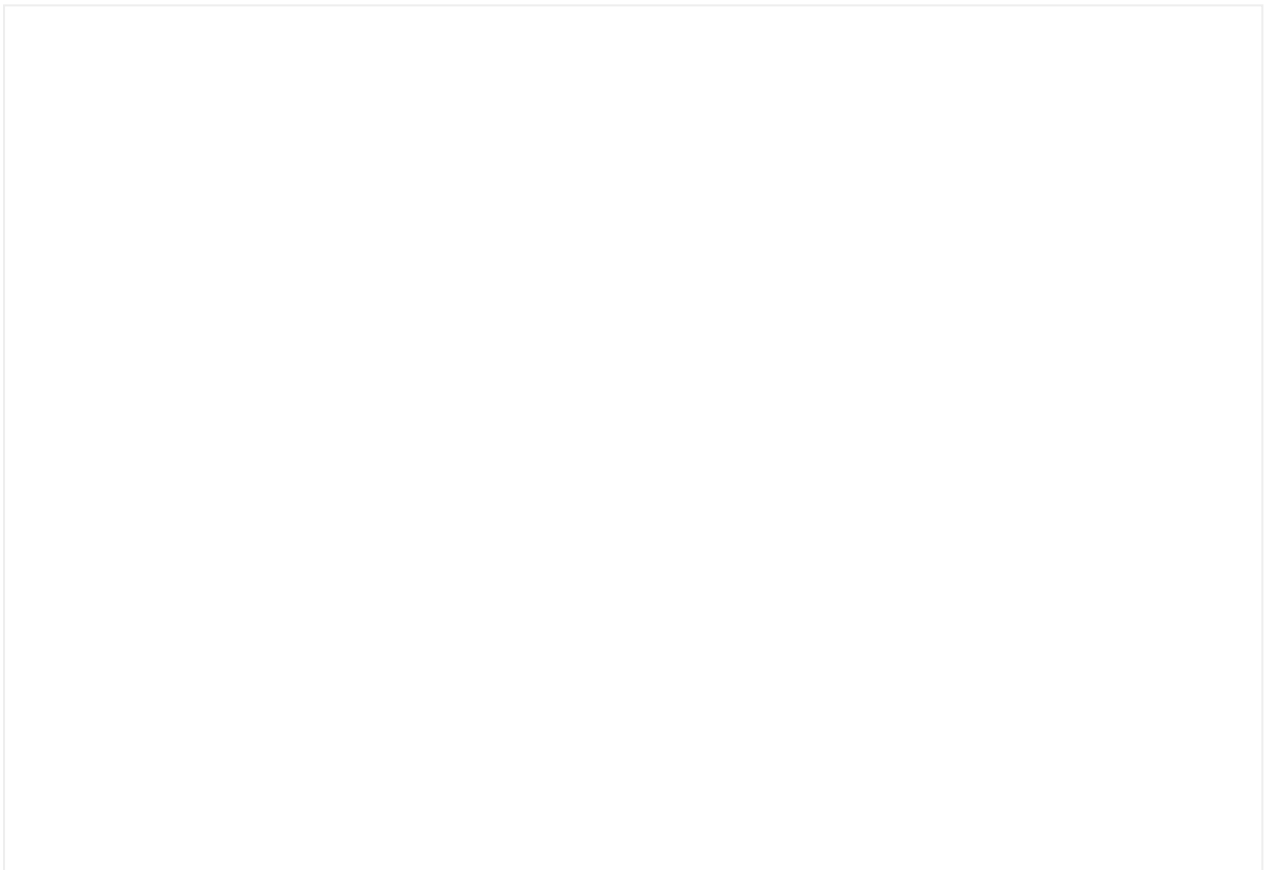
Header

A header serves to **highlight relevant content** or **as a guide** for the user, either giving greater emphasis to content within the context of a page or as additional information to the content that will then be presented to the user.

We have two different types of headers:

- **Header**
- **Main Section Header**

Interactive demo



Usage

- Use a header to highlight relevant content to the user
- Avoid the combination of **large navigations bars** and header in native app environments
- For more conversational content we recommend using the **Main Section Header** component instead

Anatomy

- **Tag** (Optional)
- **Pre-title** (Optional): The pre title provides the necessary context to understand the title.
- **Title** (Optional): The title provides the main message that we want to communicate to users.
- **Description** (Optional)
- **Slot** (Optional)
- **Breadcrumbs** (Optional) [Only desktop]

Content

Language type

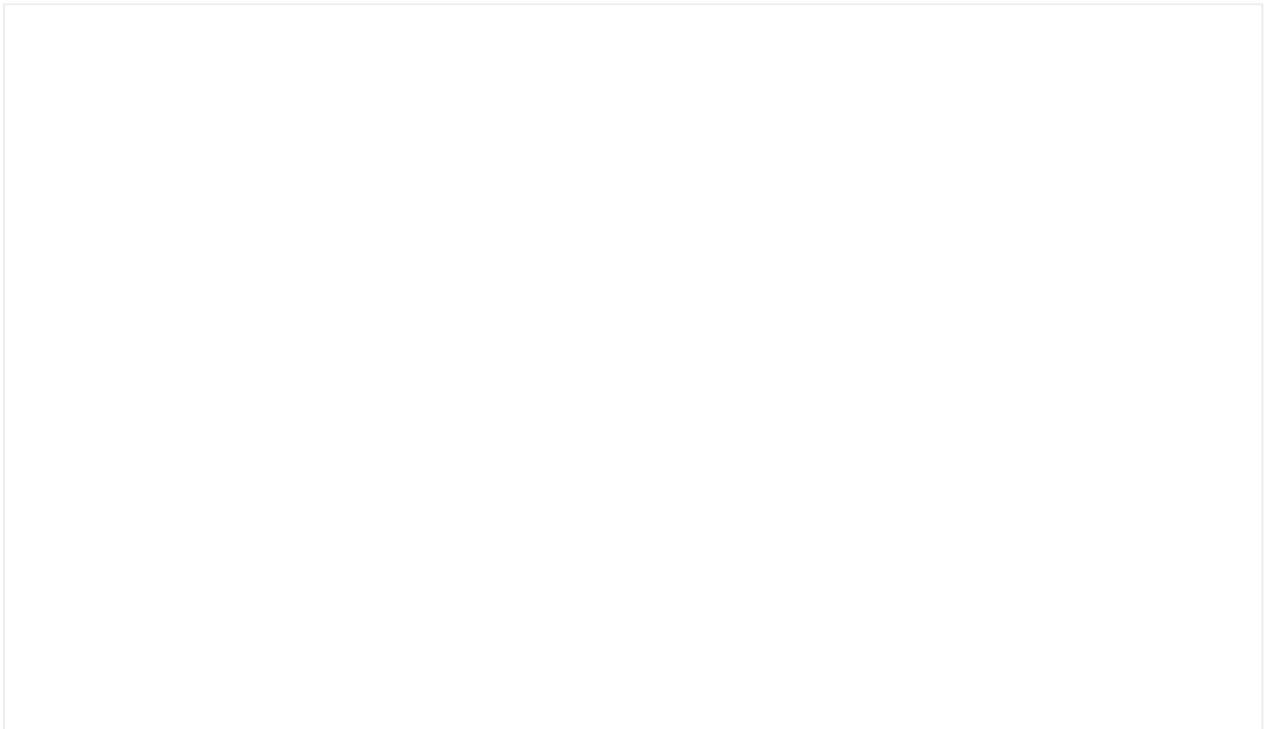
When designing a header, we must ask ourselves two questions: what do we want to communicate and how do we want to do it?

- The modularity of elements allows us to use these headers to help users quickly understand information and display the main actions of a screen.

- The type of language we use will range from a more functional language to a more conversational one, which establishes a dialogue with the user.

Structure

- **Functional:** We can build functional headers in which the elements are arranged in a structured way, allowing the user to identify them quickly and independently.
- **Conversational:** We can design more organic content using conversational language, where the user is provided with all of the information of the message in the same sentence.



Slots

Slots inside the component allow the customization of the component content.

Their main function is to offer flexibility and scalability for a component to be adapted to the specific requirements of each product.

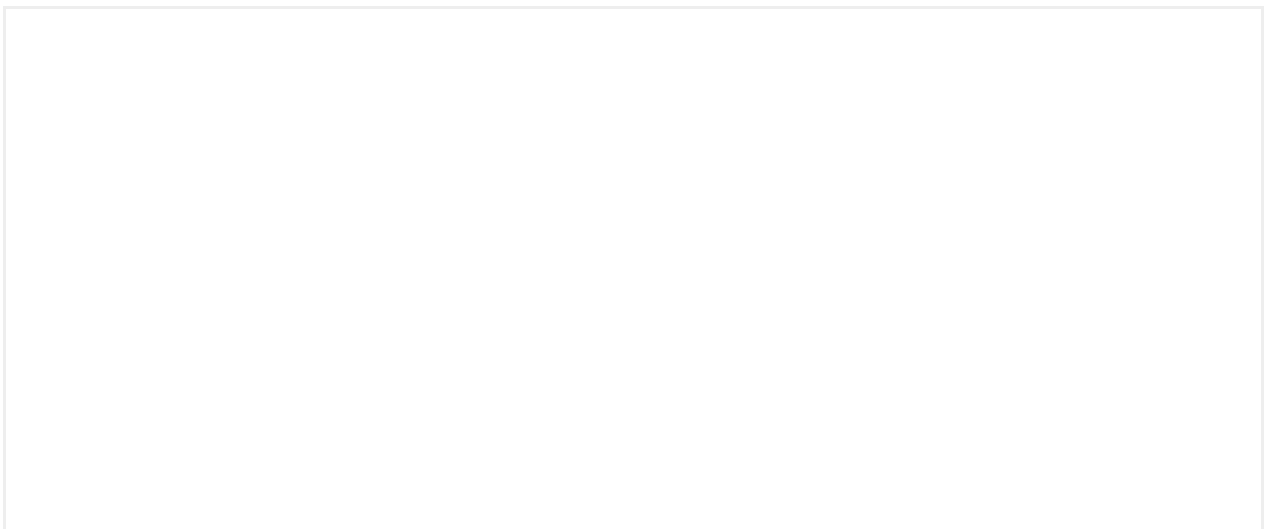
Slot position

The header component supports three different slot positions:

- **Default**
- **Bleed**
- **Side by Side** (Only in desktop)

Main Section Header

Interactive demo



Usage

On desktop we lack the Navigation Bar component, and therefore we use the Large version in the main tabs of our product's app. The main tab header transfers the composition and visual weight of the app's Large Navigation Bars to the main sections of desktop site,

Anatomy

- **Title:** The title provides the main message that we want to communicate to users.
- **Description** (Optional): The description amplifies the information.
- **Action** (Optional): This is an interaction element. It allows users to take immediate action and decisions based on contextualized information.

Accessibility

Implementation

Additional info

Hero

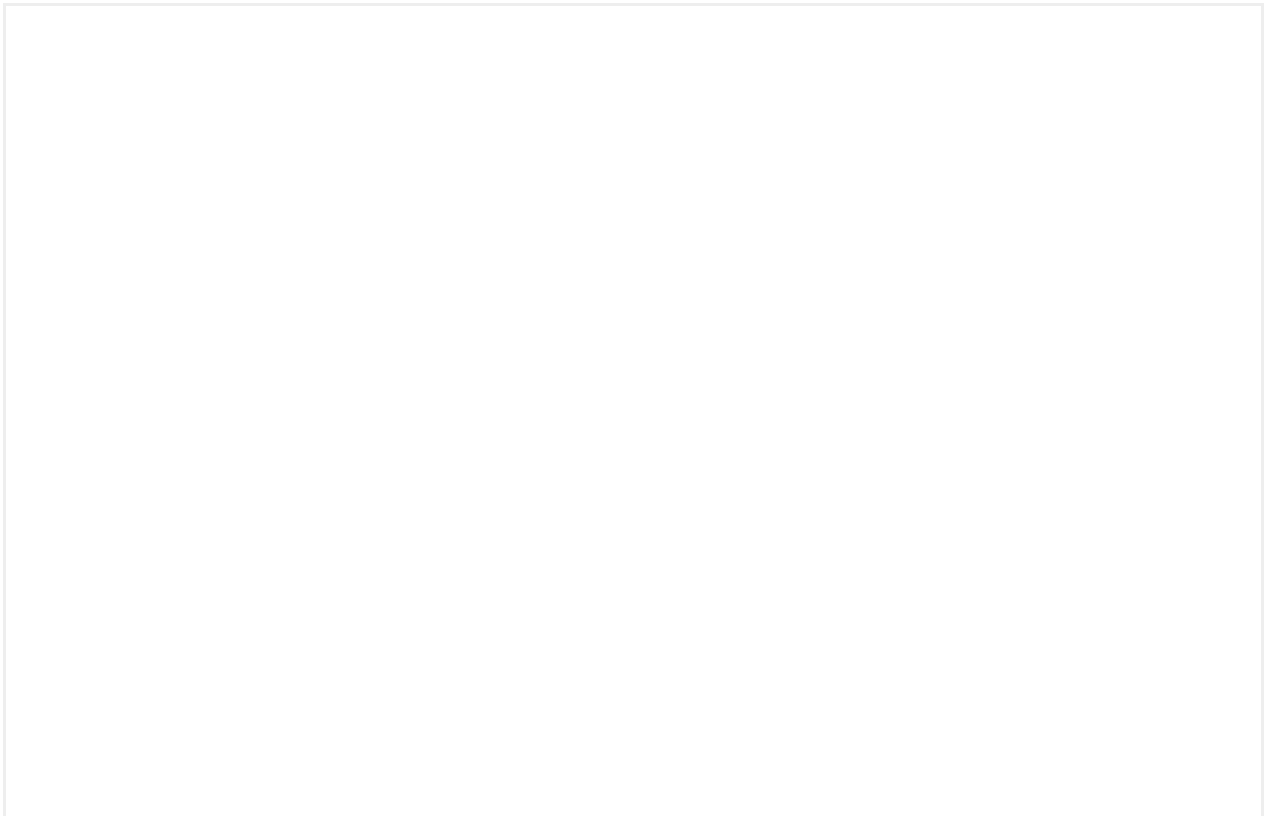
A hero is a promotional section at the top of the page with a clear call to action.

We have two different types of hero:

- Hero
- Cover hero

Hero

Interactive demo





[See desktop version in the full window preview →](#)

Usage

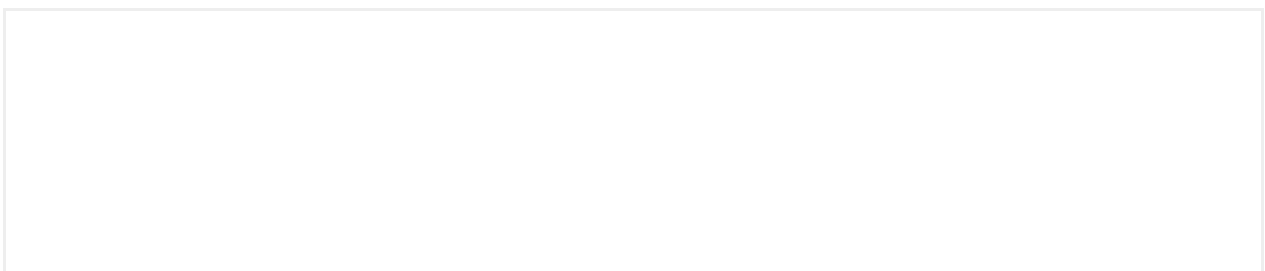
- The hero should be always positioned a the top of the main content.
- Use concise and short titles and descriptions.
- The background can either be `background`, `backgroundBrand`, `backgroundBrandSecondary` or `backgroundAlternative`.

Anatomy

The hero component contains the following elements:

- **Media:** Image | Video
- **Tag** (Optional)
- **Pre-title** (Optional)
- **Title**
- **Description** (Optional)
- **Slot** (Optional)
- **Actions** (Optional)

Sizing





Container

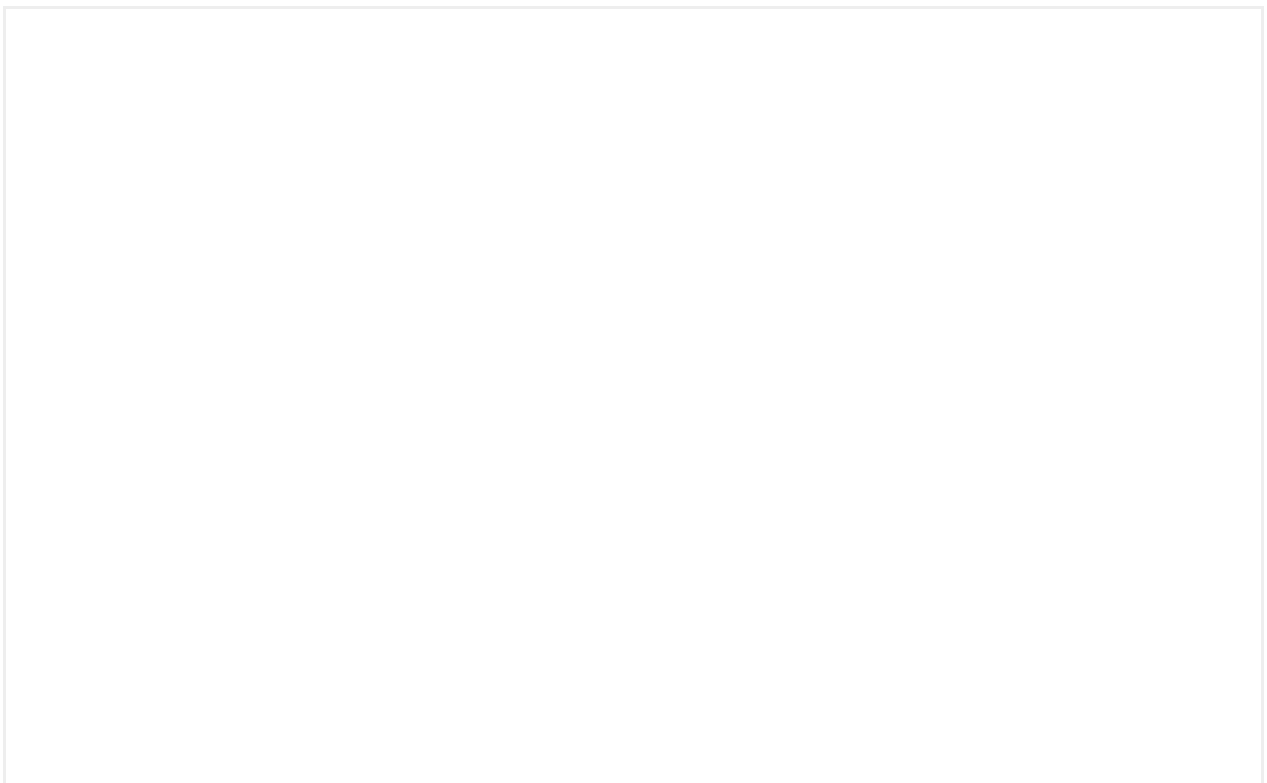
The hero container width is always the 100% of the viewport width. The height supports multiple configurations:

- When **undefined**, the container height will behave as **hug**, it will grow depending of the size of its contents.
- When **defined**, the container height will behave as **fixed**, the content will adapt to the height of the container.

Media

The media height can be defined by the aspect-ratio or a fixed value, in order to understand the available configurations for the image or video components [go to the Image primitive example](#).

The media width is always 100% for mobile devices. In desktop is defined as 6 columns of the [responsive grid layout](#).



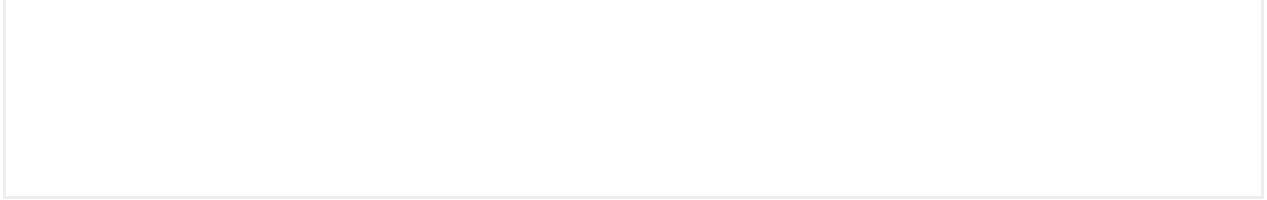


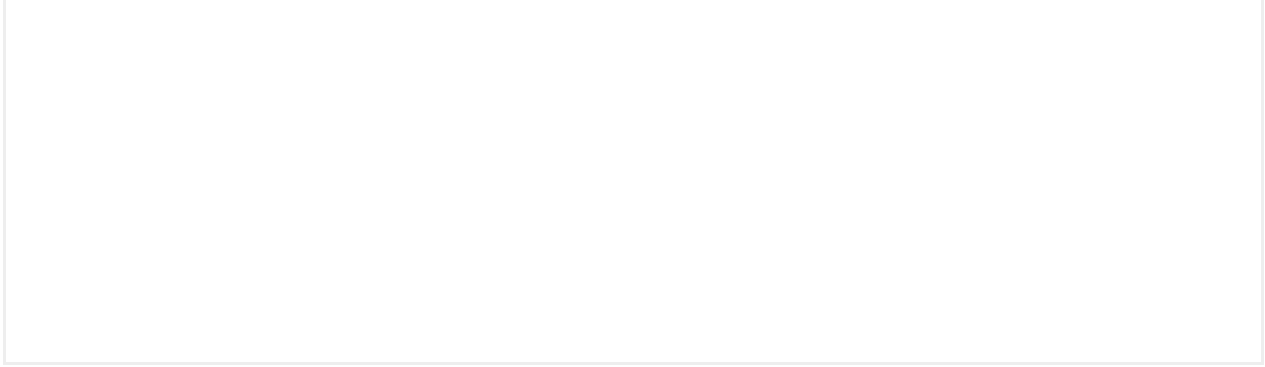
Image positioning (Desktop)



Cover hero

It has been designed to capture the attention of your users when you want them to focus on visual content or audiovisual.





See desktop version in the full window preview →

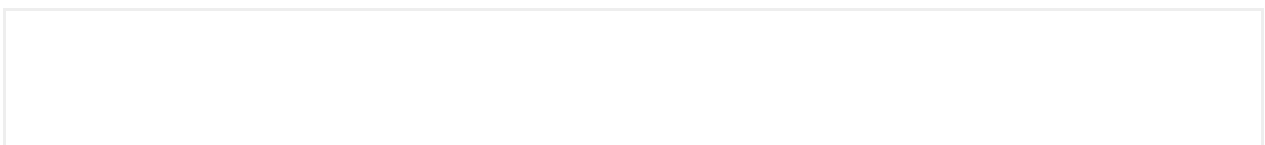
Anatomy

The hero component contains the following elements:

- **Media:** Image | Video
- **Tag** (Optional)
- **Pre-title** (Optional)
- **Title**
- **Description** (Optional)
- **Slot** (Optional)
- **Side slot** (Optional)
- **Actions** (Optional)

Background

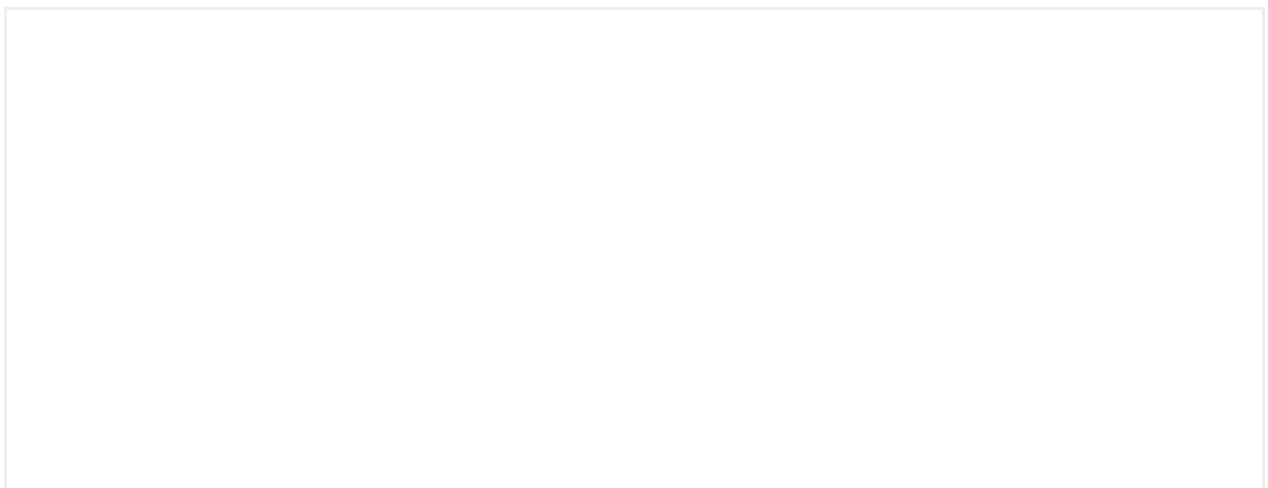
The Cover hero allows using any media as background or use a custom color. If no media or color is provided, the background will behave based on variant definition.





Content alignment

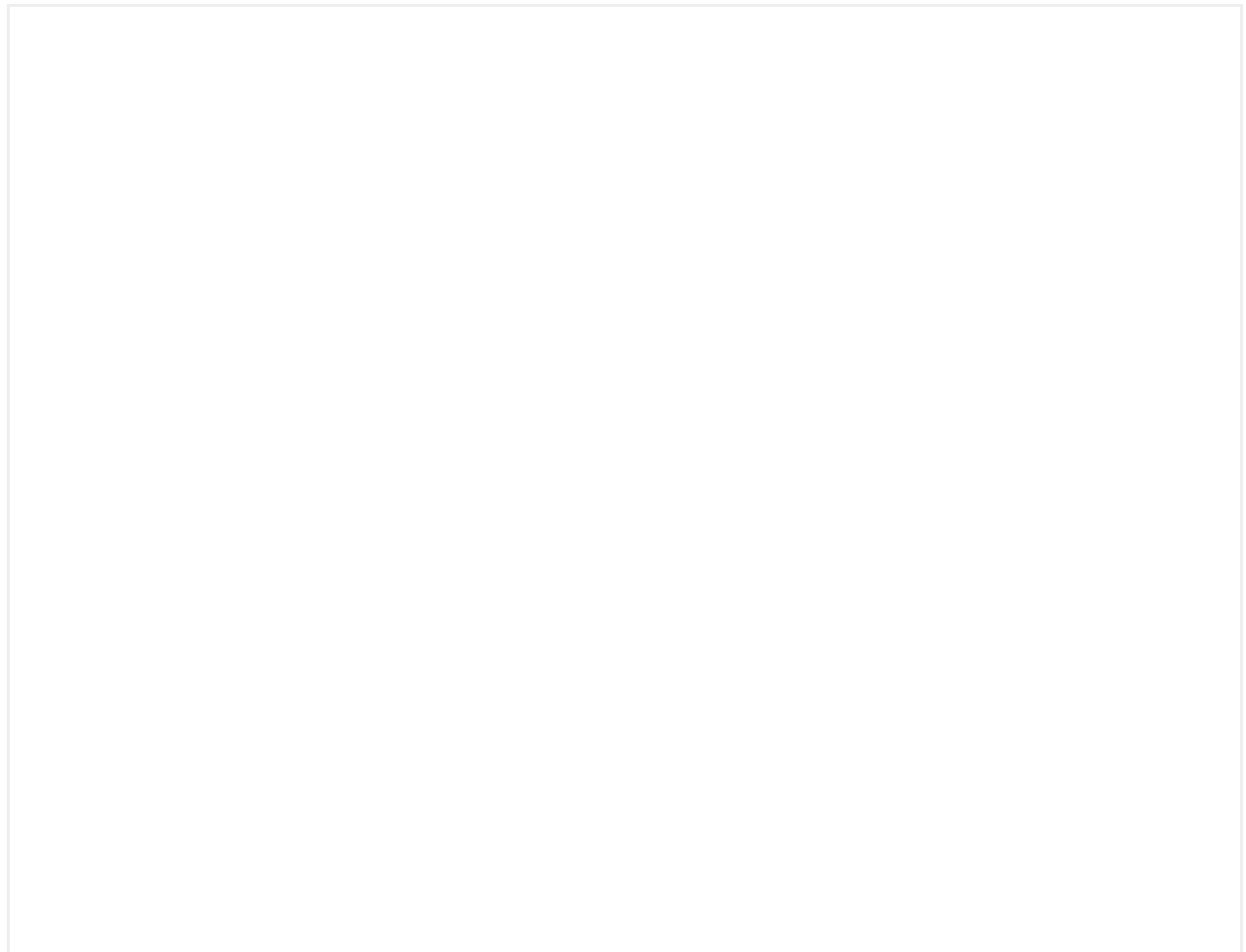
The Cover hero allows to configure the horizontal alignment of its contents between two values: left (default), and center

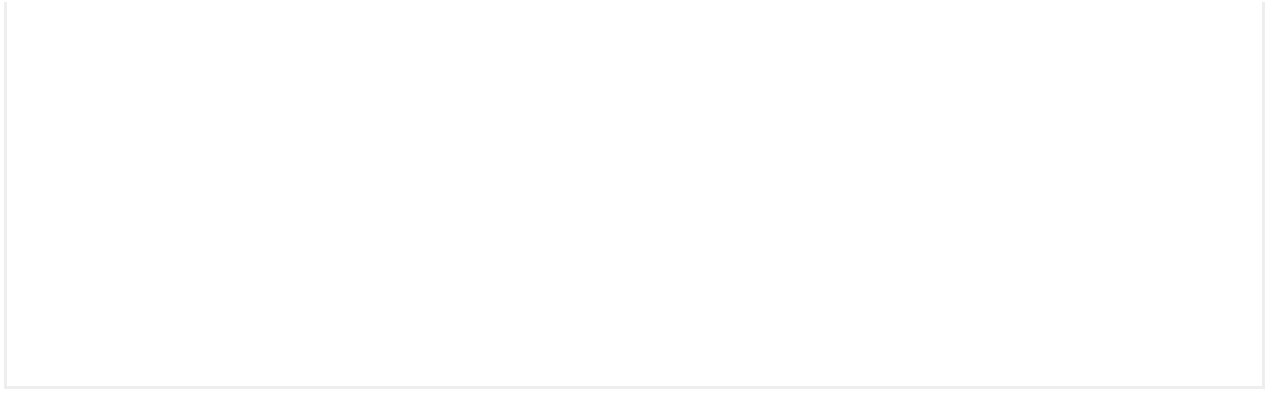




Hero slideshow

A hero can be placed inside a **slideshow** in order to show different elements.





Content behaviour

In mobile devices, the content inside a slideshow will behave as space-between in the vertical axis, placing the actions always at the bottom of the container no matter the height of the latter.

Slots inside the component allow the customization of the hero content.

Their main function is to offer flexibility and scalability for a component to be adapted to the specific requirements of each product.

Accessibility

Implementation

Additional info

In-App messages

What is an In-App?

An in-app message is a message represented by a full-screen modal used to acknowledge users about new or revamped features within the app for whom they are segmented. This means that the message should be **understandable**, **representative** and **achievable** by the segmented user.

An in-app message **is not** an element that indicates anything related about new information associated with an existing component or feature in the app nor new marketing promotions. For that goal, there's another series of components that can be used with that purpose

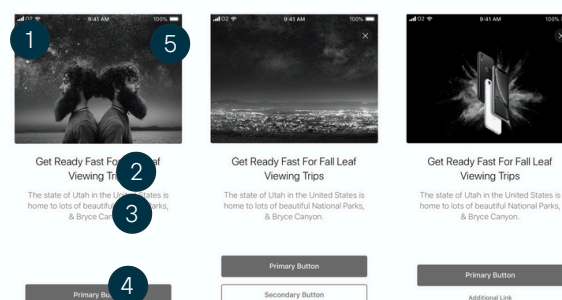
Usage examples

- Replacement of a classic app to a new one, with new features.
- Launch a new functionality and inform the users about this new feature.
- Move a function or section from top bar navigation to the main app bar and inform the users about this change.

Layouts Available

Single Card

The single card message is the default layout used to explain a new feature in a concise and specific way.

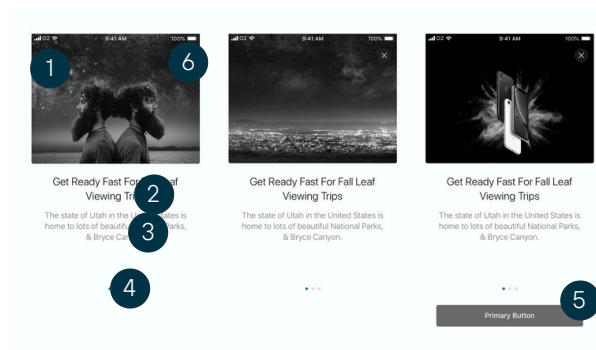


- | | |
|----------------|----------------|
| 1 Header image | 4 Actions |
| 2 Title | 5 Close action |
| 3 Subtitle | |

Slideshow

The slideshow message is the available layout used to explain a series of specific new features that are related to each other or a new feature which is complex and needs more space for more content.

- Header image
- Title (one for each card)
- Subtitle (one for each card)
- Previous card (back button or no button)
- Last card (1 button/2 buttons/1 button + link)

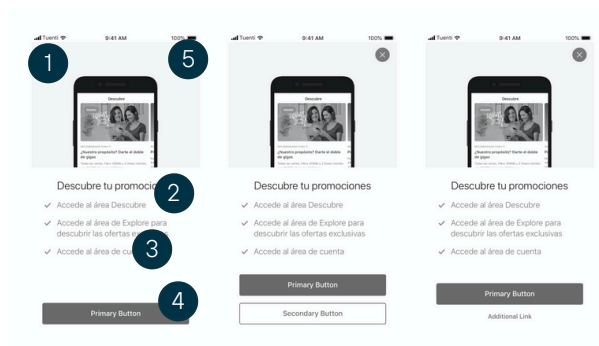


- | | |
|----------------|-------------------------------|
| 1 Header image | 4 Slider controls |
| 2 Title | 5 Actions (only in last step) |
| 3 Subtitle | 6 Close action |

Bullet list

A bullet message is an option used only when the single card is not enough and highlights the key points of the feature in a more scannable way for the user.

- Header image
- Title
- Checks (up to 3)
- Action (1 button /2 buttons/1 button + link)



1 Header image

2 Title

3 Bullets list

4 Actions

5 Close action

Modes Available

First access

Also called "Onboarding". The message is displayed to the user when they access the app for the first time.

Global

The message is displayed to the user when they access the app but they have been logged in at some point in the app.

Navigation

The message is displayed to the user when they tap on a specific tab/section from the app.

Input fields

Allow users to easily enter data in the interface.

Interactive demo



Anatomy

Container

The width of the container varies depending on the type of layout and the type of information that the user needs to enter in the field.

2. Label

Labels inform users about what information needs to be entered in each of the fields. They need to be clear and concise. We must make scanning easier for users. Always keep in mind that labels are not help texts.

3. Input text

- Text: The text that the user has entered in a text field.
- Cursor: Indicates the position where the text is inserted inside a field.

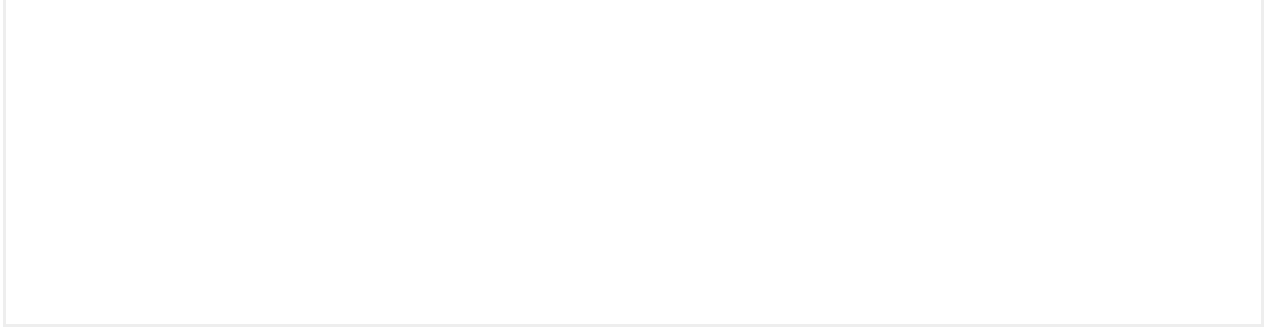
4. Icon (optional)

We can use the icons for two purposes:

- Contextual: To provide the user with greater context about the type of information they have to enter. For example, adding the credit card icon in the field where they need to enter their card number.
- Actionable: This allows users to perform an additional action within the field. For example, tapping on an eye icon to show or hide a password.

5. Help elements

- Help text: Help texts provide additional information on how to complete a field within a form. They can also be used to clarify how said information will be used. Help texts should be concise and easy to read.
- Error message: When the data entered does not conform to the acceptable parameters. This could be because it has not been entered, or because it has been entered incorrectly. An error message gives the user instructions on how to solve the problem.
- Character counter: In text areas where there is a character limit, we will indicate the number of characters used and the total character limit.



Usage

Accessibility

Implementation

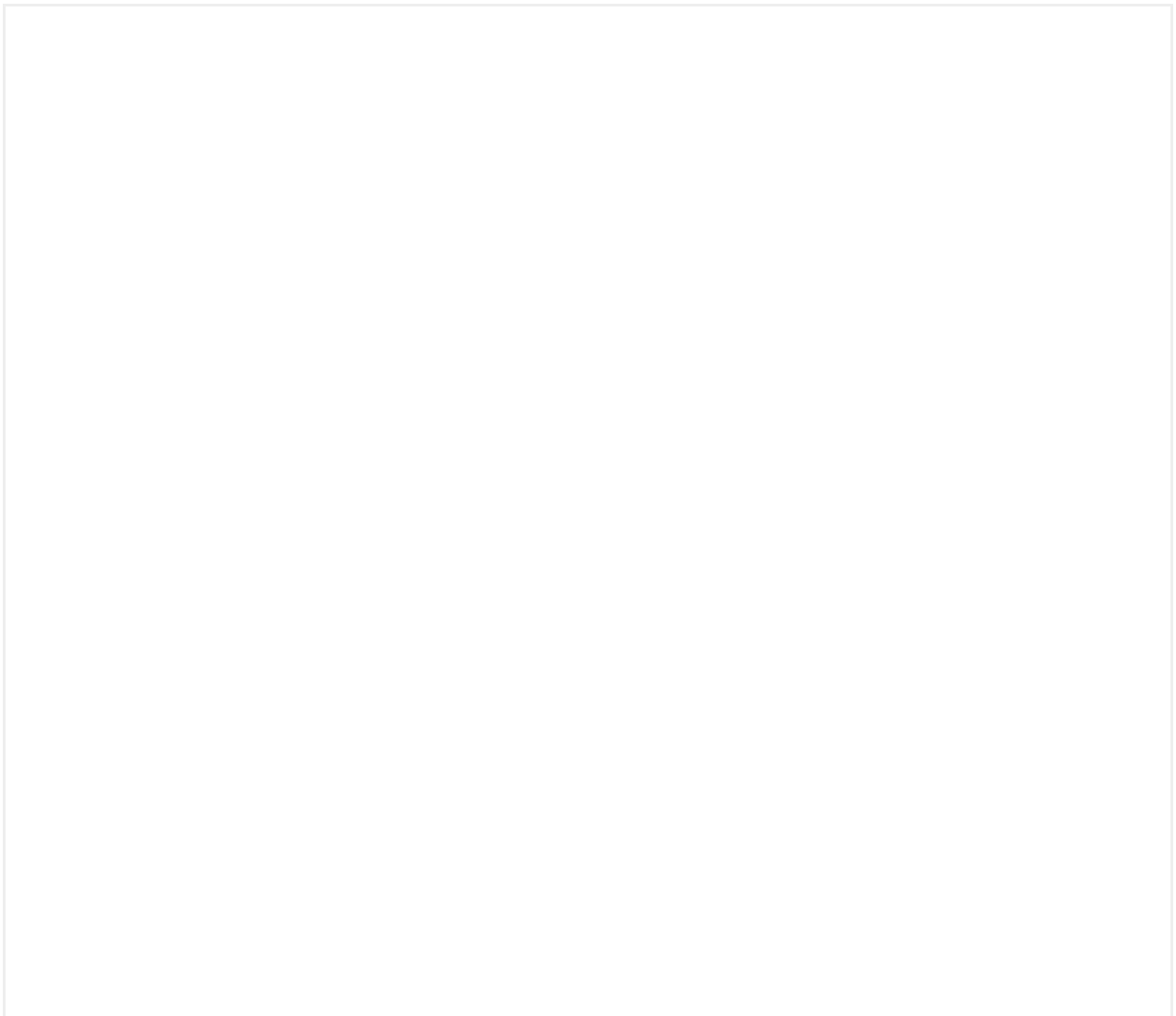
Additional info

Lists

Lists are elements ordered vertically. Lists can contain text, images or components.

Row & Boxed row list

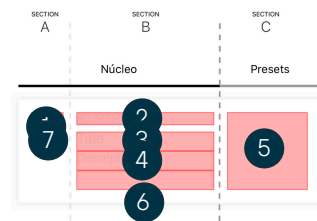
Interactive demo



Anatomy

A list is divided into 3 sections: A, B and C.

Every list will have a nucleus, which is Section A and Section B together. Section C is set aside for the requirements of the different product parts.



- 1 Possibility to adding a 24px or 40px asset (24px + circle of 40) or images. (optional)
- 2 Headline. This may be a tag component or a text. (optional)
- 3 Title (required)
- 4 Subtitle (optional)
- 5 Presets
- 6 Divider (optional)
- 7

Nucleus

The nucleus features on all lists. By adding or removing parts of the nucleus and section C, you can build many different lists that will not lose coherence with the others.

A list will always have a Title. The other blocks are optional, and the list will automatically adjust to the visible elements.

Sections

The sections are groups of elements that may or may not appear.

- **Section A** is optional
- **Section B** will be obligatory, because it will always have at least a Title
- **Section C** is a dynamic section with 3 presets

Presets

Presets are blocks of pre-established configurations. There are four types of presets.

- Empty (no elements)
- Navigation (includes a chevron icon)
- Controls (checkbox, radio, switches)
- Slots (custom view)
- Detail

Typology

Information



Navigation



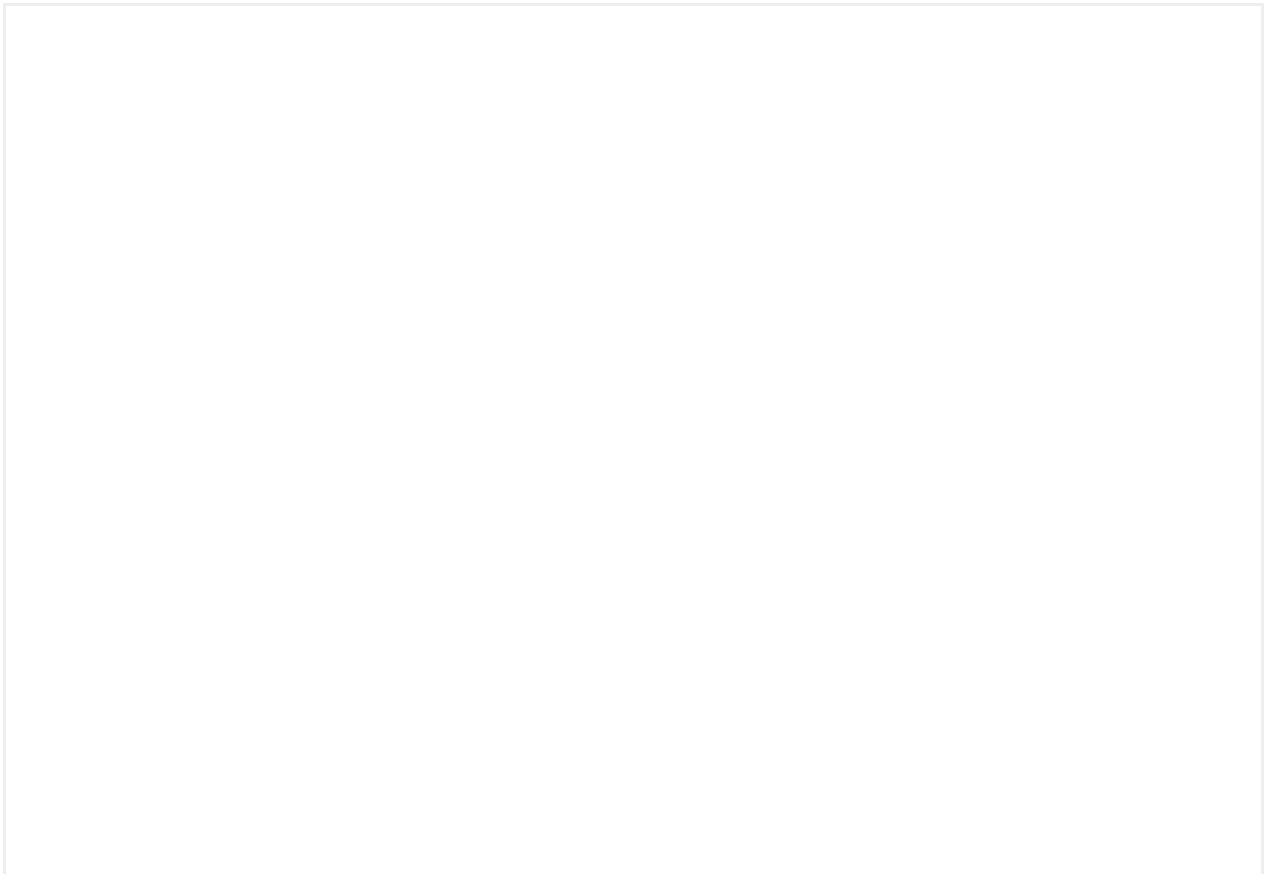
Controls



As the name suggests, the text Detail should be short and concise.
If you need to include more information use Description.



With images & icons



Grouped lists

Although **it is not a type of component list** as such, given that it would be built at the atomic level, it serves as a solution in cases where you need to group lists together to better organize the information.

To do this, **you have to build it through atoms** using the **boxed** and row components.

The following are examples of different compositions built using this method.

Danger action in lists

Danger action lists are a variant of lists that allow destructive actions. A destructive list always anticipates an irreversible action.

Depending on the product need, there may be a danger list item that will have the same structure as the default list changing the color of the icon, title and, depending on the variant, the background will also change. As a general rule, this item will be placed at the end of the list.

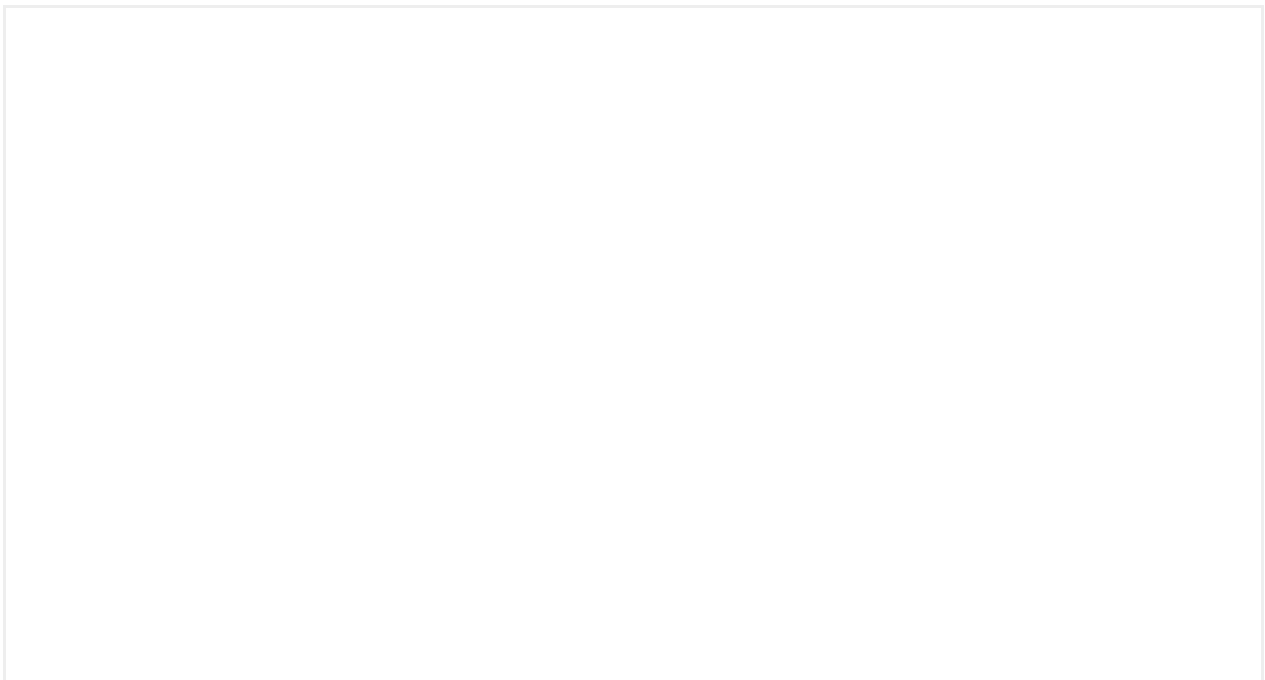
This type of list should be used in case of:

- Destructive actions or navigations that lead to a destructive action.
- Actions of a destructive nature (delete, deactivate) and that have an "irreversible" effect.
- Only for when that type of action makes sense within a list.

We could distinguish two use cases for this typology of lists.

A list that functions as an action that triggers a confirmation modal.

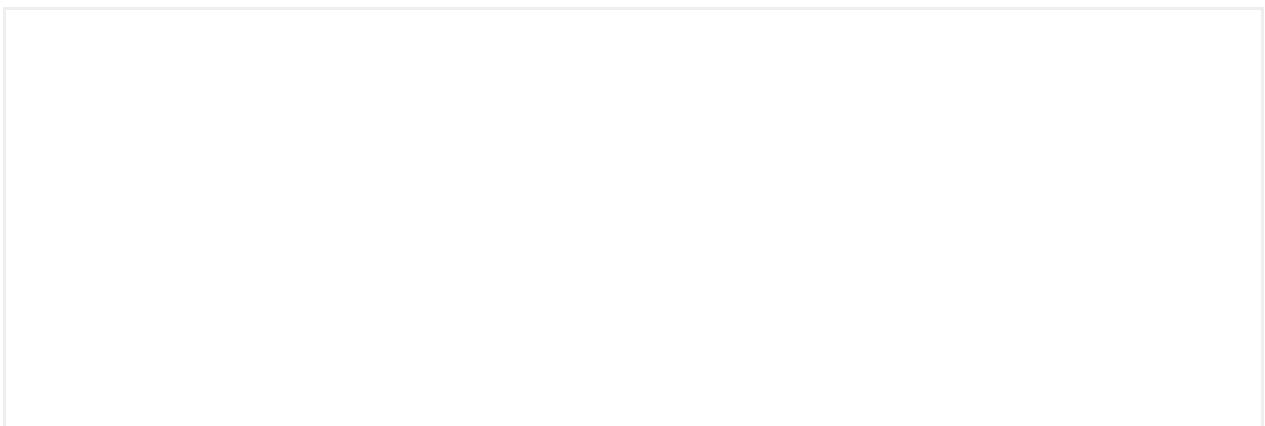
(Always before an irreversible action it is important to show a confirmation)



As a previous step to a screen where we give you extra information before a destructive action.



Ordered & Unordered list

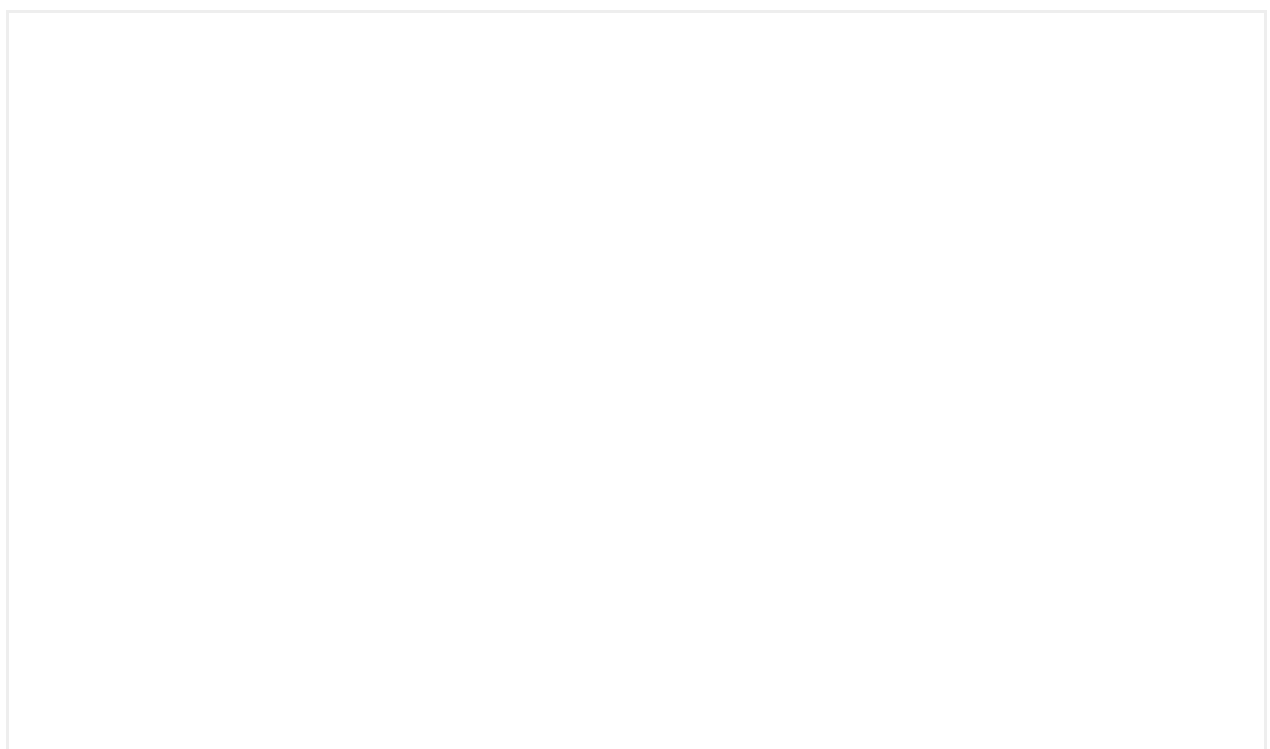


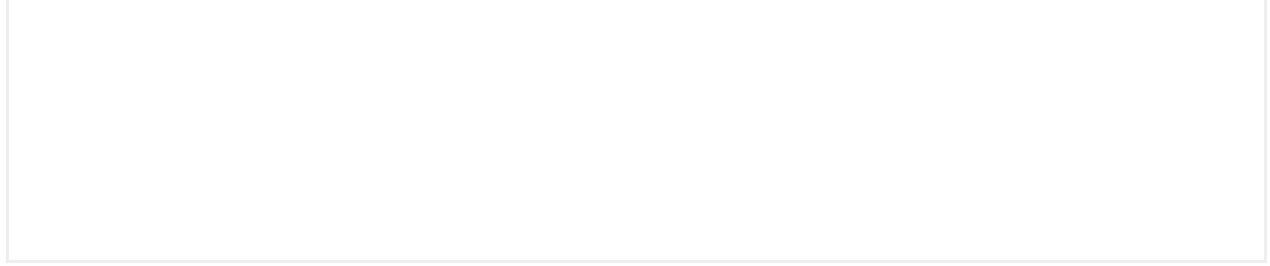


Slot

Mística allows more flexibility with Slots which allows the list component to grow according to the needs of the product, without losing coherence across all lists. Designers will be able to create custom lists by modifying the section C freely.

They are inserted in the body of the card, between the content and the actions.





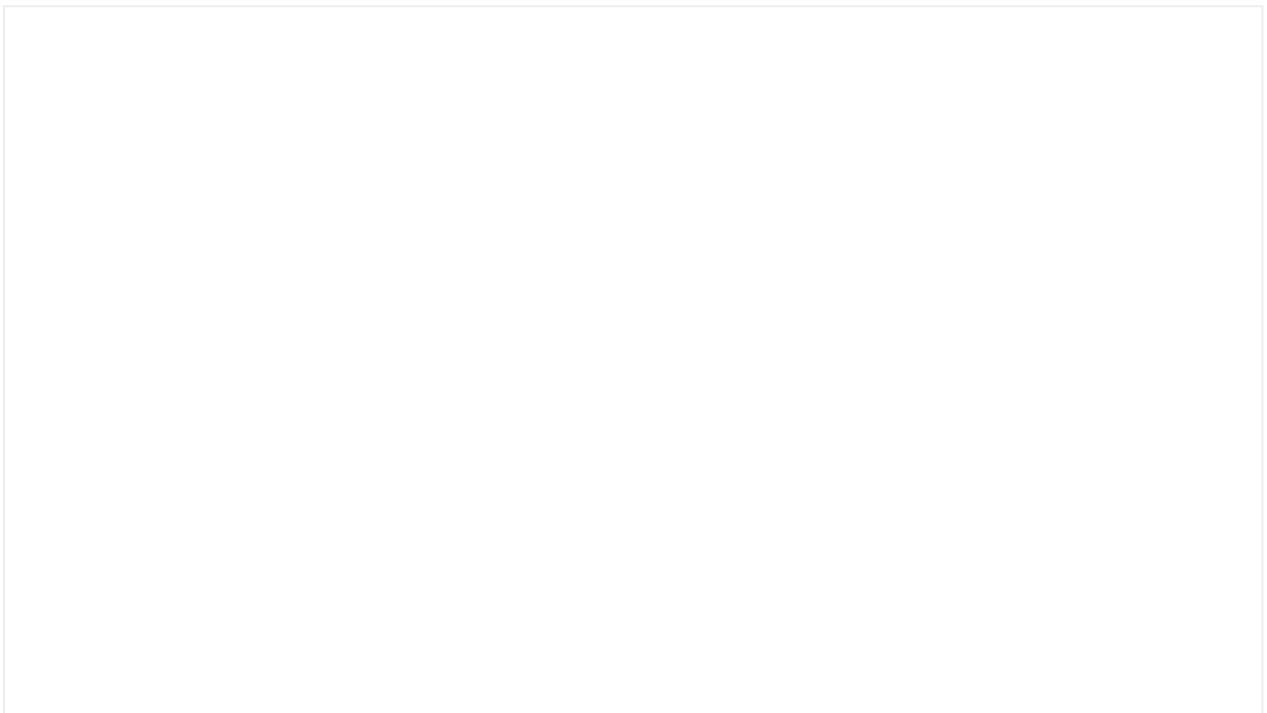
Accessibility

Implementation

Additional info

Load error screen

If the content that couldn't be loaded is just a part of the whole screen and the latter is still comprehensible and has valuable information regardless, use the module instead of showing the full page error, as long as it's also technically possible.



Interactive demo

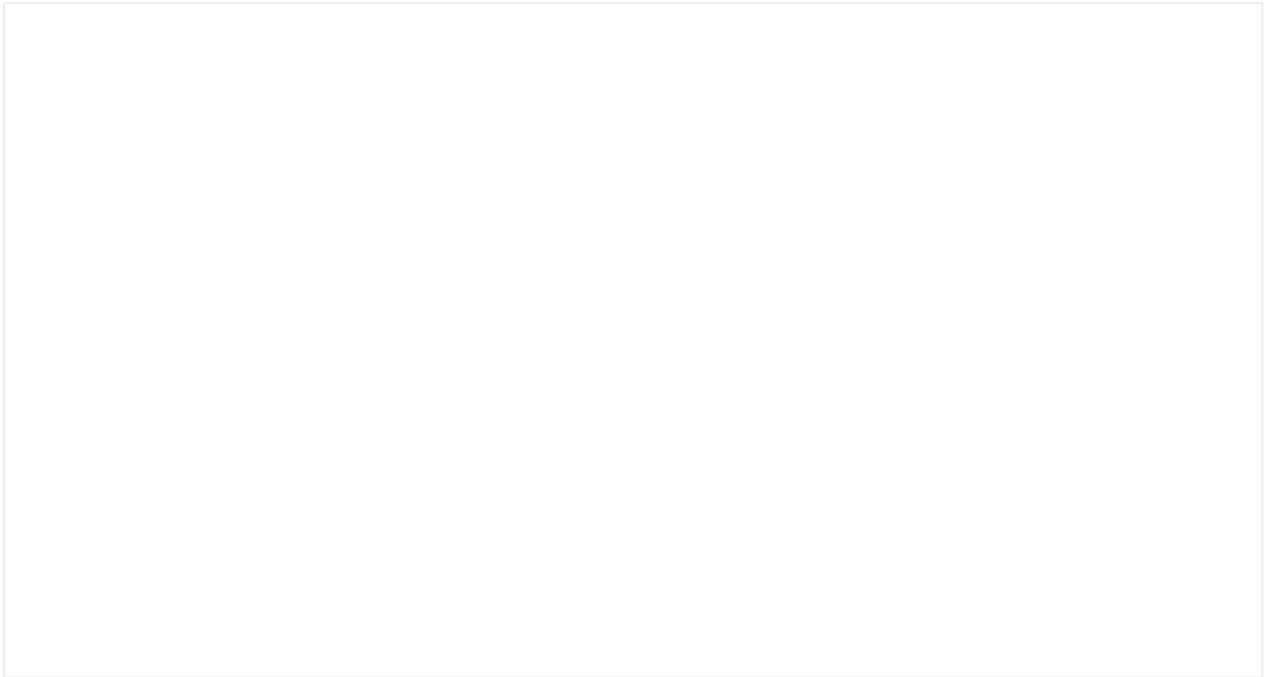
Usage

For loading errors specifically, where the content of a screen or even a specific module couldn't be loaded properly, please use Load Error.

Additional info

Loading bar

Show users when a background loading is running.



Usage

Use a loading bar when:

- You need to load data in the background.
- Some content from the initial load was unable to complete or was unsuccessful.

Don't use a loading bar when:

- Loads triggered by user action. Use the **spinner** component instead.
- Initial load of content. Use the skeleton component instead.

Implementation

Additional info

Logo

Provide different dynamic variants of the logo and changes automatically based on the brand that is using it.



Usage

Imagotype

The imagotype variant of the Logo component displays the brand's icon or symbol along with the brand's name or text.

Isotype

The isotype variant of the Logo component displays only the brand's icon or symbol without any accompanying text. This variant is suitable for situations where the brand's name is already prominently displayed elsewhere on the page or where the focus is on the brand's symbol.

Vertical

The vertical variants of the Logo component are designed to be used in situations where vertical space is limited. The vertical variants allow the brand's icon or symbol and name to be displayed in a compact and easily readable format, without taking up too much vertical space.

Implementation

Additional info

Menu

Displays a list of options when clicked, allowing users to select from a hidden set of choices.

Interactive Demo



Usage

The use of a menu component is an effective strategy in user interface design for organizing options neatly. This interactive element is triggered when a user clicks over a specific button or link, revealing a list of related choices.

This technique is commonly employed on websites to group content categories, enabling users to access additional information or perform specific actions without overwhelming the screen with a large number of permanently visible links. Help to simplify and organize options is crucial for enhancing the user experience.

Anatomy

The Dropdown Menu component contains the following possibilities:

1. **Trigger:** wraps the item that will open the menu. The trigger can be a button or an icon.
2. **Item:** describes the action or option the user may select. The text should be short, concise and easy to understand.
Item can displays an icon and a checkbox (both are optional).
Icon: provides a visual representation of the action.
Checkbox: allows the user to select one or more options simultaneously.
3. **Divider** (optional): visually separate a items group (section).
4. **Destructive item** (optional): if you include a destructive action, we recommend placing it on the bottom. Only one destructive action is allowed.

Positioning

The separation between the trigger item and the menu is 8px.

Its placement depends on where the menu appears within the user interface. It can be aligned to the left or right, top or bottom to ensure the menu is clearly visible.

Additional info

Modals

Alert

An alert is a modal window that prompts the user for acknowledgment.



Usage

- Use alerts when you want to provide critical information to the user
- Use short, descriptive and easy-to-understand copywriting. Keep messages short enough to fit on one or two lines to prevent scrolling (80 characters maximum)
- Do not use the alert component to provide feedback about user actions. Use a **snackbar** instead

Anatomy

1. **Title** (optional)
2. **Description** (required)
3. **Confirm Action** (required)
4. **Cancel action** (optional)

This type of modal has only one button, if no button label is defined, the default is "Accept".

Native alerts

In native implementations the alert component is automatically transformed in the platform default.

Confirm

Confirm is a modal window that prompts the user to confirm or cancel an

action.



Usage

- Confirmations should be opened as a result of a user action or critical system event
- Use the confirmation component for uncommon destructive actions that can't be undone
- Use short, descriptive and easy-to-understand copywriting. Keep messages short enough to fit on one or two lines to prevent scrolling (80 characters maximum)
- Do not use a confirmation for routine actions

Anatomy

1. **Dismiss** (optional)
2. **Title** (optional)
3. **Description** (required)
4. **Confirm action** (required) (Confirm text is also required)
5. **Cancel action** (required) (Cancel text is optional)

This type of modal has only two actions, if no action labels are defined, the defaults are "Cancel" and "Accept". If the "Cancel" copy it's too similar to the alternative action ("Abandon" for instance), consider using "Not now" to avoid confusion.

Native confirmations

In native implementations the confirmation component is automatically transformed in the platform default.

Dialog

A dialog is a modal window that allows a higher degree of customization than the alert and confirmation modal dialog variants.



Usage

- Use dialogs when the modal's task is more complex than accepting or cancelling
- Use dialogs when any type of rich content is needed to complete the modal's tasks

Anatomy

1. **Dismiss** (required)
2. **Icon** (optional)
3. **Title** (optional)
4. **Subtitle** (optional)
5. **Description** (required)
6. **Slot** (optional)
7. **Primary action** (optional) (Action label is optional)
8. **Secondary action** (optional) (Action label is optional)
9. **Link action** (optional) (Action label is optional)

When asking for user's data and there is a cancel action and a dismiss, it's important to distinguish between both to prevent the user from losing progress or data.

Dialog actions

Dialogs can contain up to three actions: primary, secondary and link. These actions follow the structure defined for the **button layout**.

Drawer

A drawer is a modal panel that slides in from the right on desktop or the bottom on mobile, used for auxiliary tasks, permanent settings, or additional information that complements the main flow.



Usage

- Use a drawer for subtasks or auxiliary flows that are not part of the principal flow (e.g., editing preferences, managing subtasks).
- Use drawers for permanent settings or configurations accessible across the app (e.g., account settings, application preferences).
- Use drawers for supplementary or additional information that supports but doesn't interfere with the primary task (e.g., help documentation, reference data).

Consider using a sheet or a dialog instead if:

- The actions are simple or contextual and directly related to the primary flow (e.g., confirming actions, selecting options).

Anatomy

1. **Dismiss** (optional)
2. **Title** (optional)
3. **Subtitle** (optional)
4. **Description** (required)
5. **Slot** (optional)
6. **Primary action** (optional)
7. **Secondary action** (optional)
8. **Link action** (optional)

Accessibility

- It is possible to pass an accessibility label to the dismiss action to better contextualize the action being performed. By default, it is set to "Close."

- The element that triggers the drawer should indicate that a modal panel has been opened.
- It is possible to change the hierarchy of the title semantics; by default, it is a heading level 2.

Implementation

Additional info

Mosaic

Layout for visually striking compositions and disrupting screen vertical rhythm



The mosaic component is a versatile tool for visual composition in a grid format, crafted to blend diverse elements like cards, images, and videos. Its core function lies in presenting information attractively and dynamically, breaking the visual monotony of screens. Often used as entry points, this component delivers a visually striking layout that captivates users, offering an engaging and enjoyable interactive experience.

Horizontal Mosaic



Vertical Mosaic



Implementation

Additional info

Navigation bars

The navigation bar displays information and actions related to the screen.

Its main uses are to place navigation items and actions, and to show the brand by including its logo or color.

Types:

- Android Navigation
- iOS Navigation
- Web Navigation

Android

Top app bar

This is the standard navigation bar in Android, and it is a native component.

For more information, you can consult Google's [Material Design guidelines](#).

Large Navigation Bar

This is a Mística component. We will use it instead of the large variant of Android's top app bar.

In this navigation bar, which is of a higher priority than other navigation bars, the title gains prominence. We will use it for the main tabs of an application, but not in secondary or tertiary levels.

Anatomy:

- **Title:** The title appears to the right of the navigation. (Obligatory)
- **Actions:** The actions are located in the top right part of the component. (Optional)

Interaction: Scrolling turns it into a native small top app bar.

Custom tabs

Custom tabs provide more control over the web experience, allowing seamless transitions between native and web views,

- Use Custom Tabs if the app directs users to URLs outside your domain.

iOS

Navigation Bar

This is the standard navigation bar in iOS and it is a native component.

For more information, you can consult Apple's [Human Interface Guidelines](#).

Large Navigation Bar

This is a Mística component. We will use this navigation bar to replace the large title variant of the native iOS navigation bar.

In this navigation bar, which is of a higher priority than other navigation bars, the title gains prominence. We will use it for the main tabs of an application, but not in secondary or tertiary levels. It doesn't include a navigation icon.

Anatomy:

- **Title:** The title appears to the right of the navigation (required).
- **Actions:** Contextual actions are located in the top right part of the component (optional).

Interaction: Scrolling turns it into a native iOS navigation bar with standard title.

Web

Unlike native navigation bars, the website has a desktop version and a mobile version.

Types:

- Main Navigation Bar
- Navigation Bar
- Funnel Navigation Bar

Main Navigation Bar

This component is built to support the primary navigation bars of public and private websites.

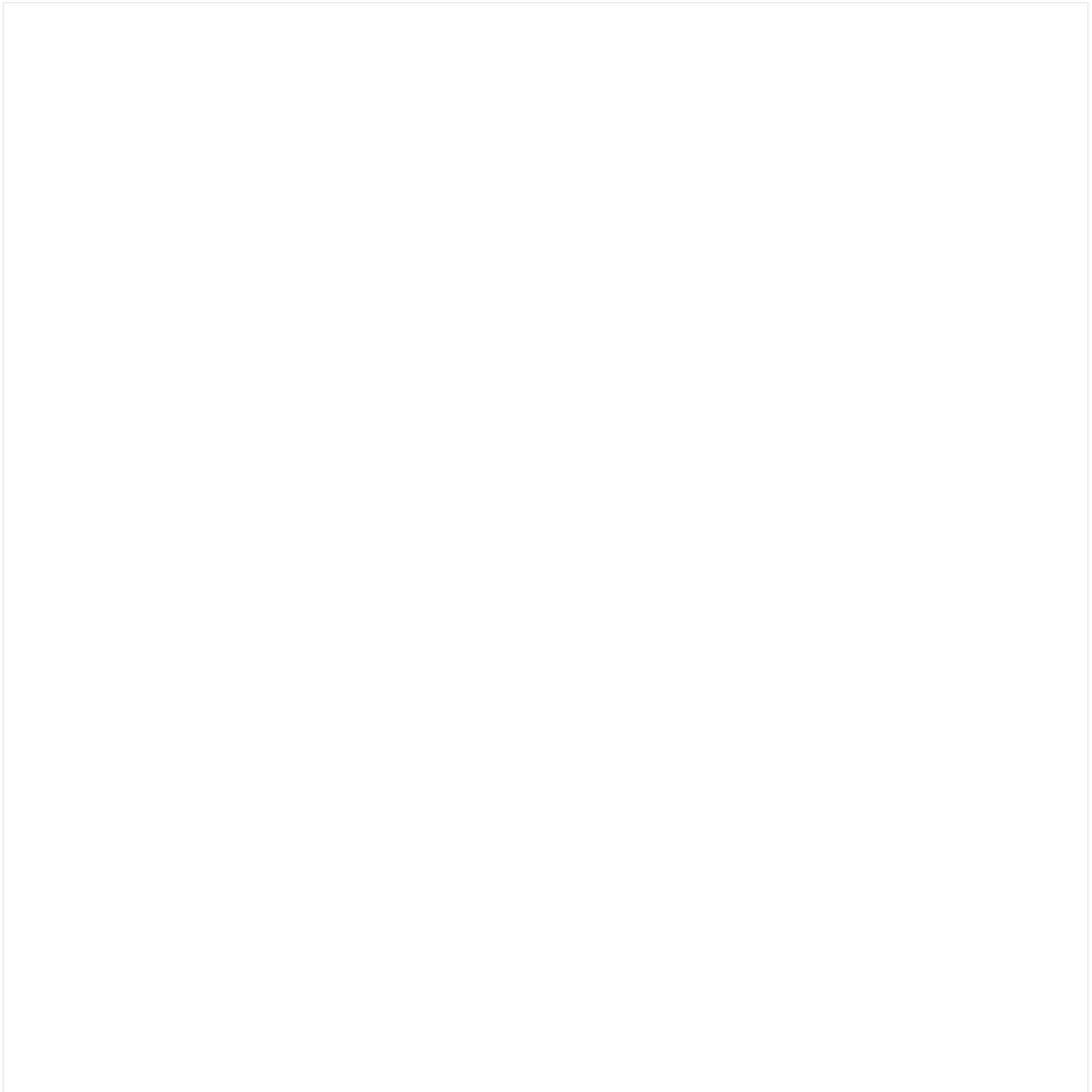
Desktop version



Anatomy:

- **Logo:** The logo is located to the left of the component (optional).
- **Main navigation bar items:** These are located after the logo (required).
- **Secondary navigation bar items:** These are located to the right of the navigation bar. It offers the possibility to use icons or icons plus text by default. However, personalized fragments may also be placed in this area (optional).
- **Menu:** It is a menu that expands when interacting with navigation items and contains second-level navigation items.

Mobile version



Anatomy:

- **Hamburger menu:** Located on the far left, it contains the first-level navigation items of the desktop version (mandatory).
- **Logo:** The logo is located next to the menu (optional).
- **Secondary navigation items:** These are located on the right side of the navigation bar. By default, these items can be represented with icons. Alternatively, custom fragments can be included in this area or within the hamburger menu (optional).

Menu

Once expanded, you'll see a menu with the first-level navigation items. The menu is closed using the usual close icon, which is located in the same position as the menu icon.

Navigation items allow users to access a second-level submenu. On the second level, the item can either display a menu or navigate, showing a title at the top with an entry point on the right labeled "View All."

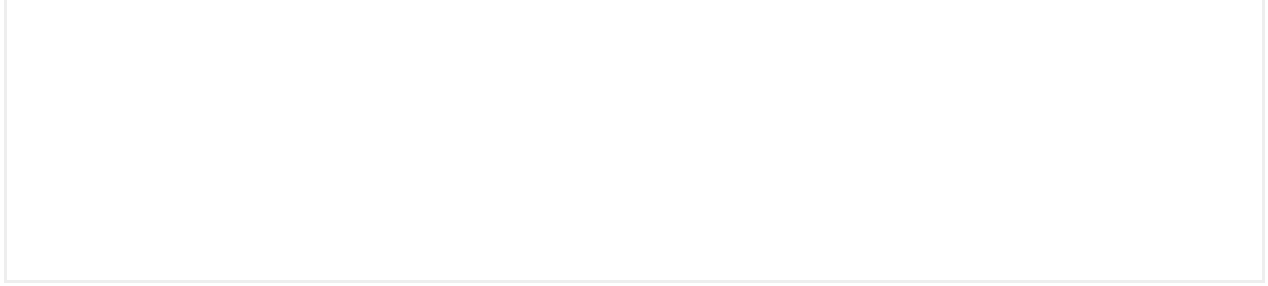
Both menu levels support custom content via the slot for added flexibility.

Navigation Bar

This navigation bar is similar to native navigation bars. Its main use is for processes that take place within a dialogue.

Desktop

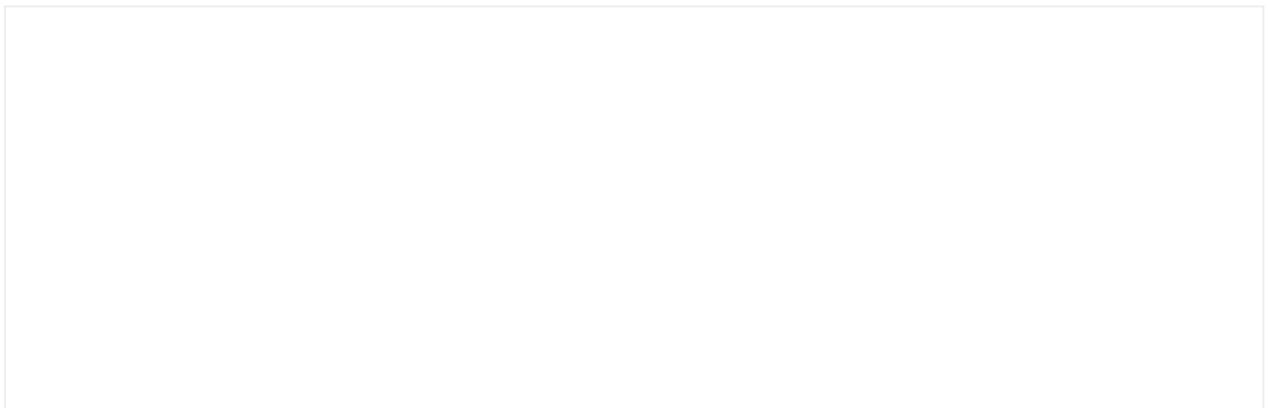




Anatomy:

- **Back:** The navigation to go back is located on the far left-hand side (optional).
- **Title:** The page title is located after the back icon (optional).
- **Actions:** The actions are located to the right of the navigation bar, and are represented with an icon (optional).
- **Close:** The close icon is located on the far right-hand side of the actions area (required).

Mobile

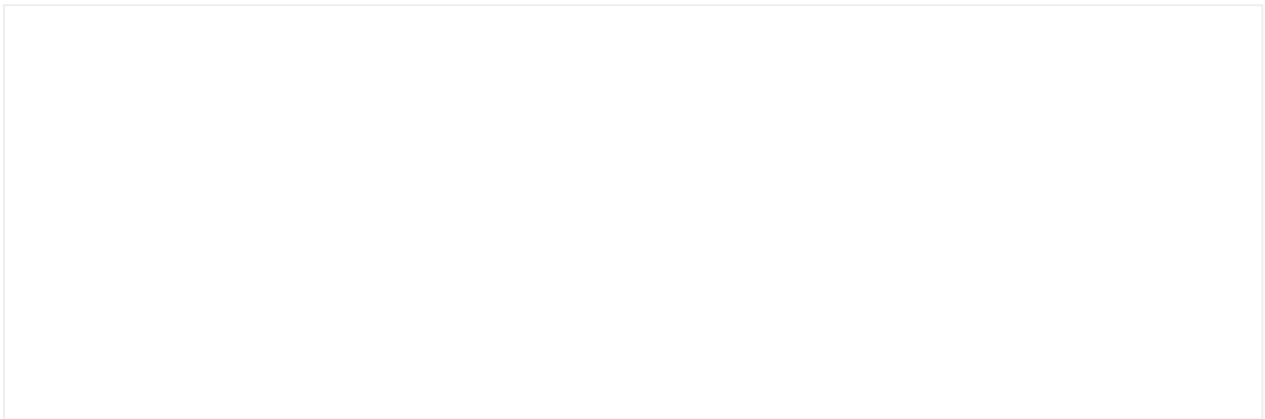


The anatomy is the same as in the desktop version. However, unlike the desktop version, it is located at the top of the device in full screen mode.

Funnel Navigation Bar

We will use this navigation bar for full screen funnels and transactional processes.

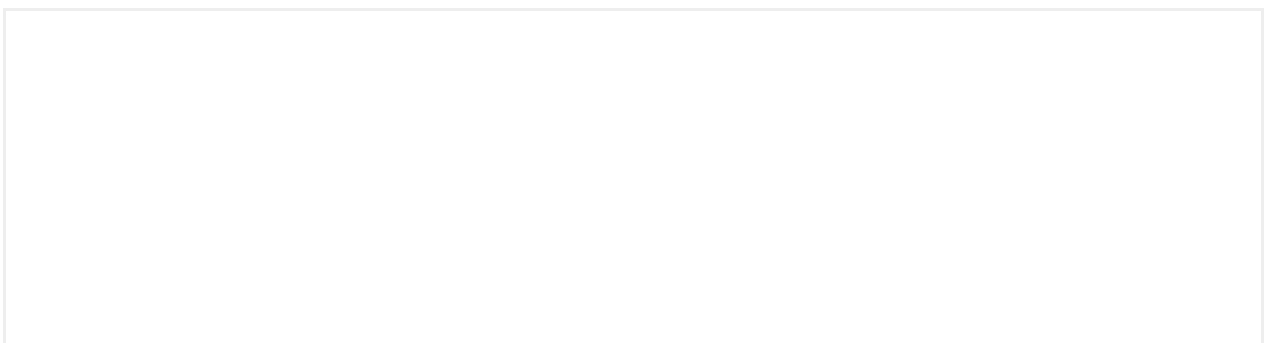
Desktop



Anatomy:

- **Logo:** The logo is located on the left-hand side (required).
- **Secondary navigation items:** These are located to the right of the navigation bar. They consist of an icon or an icon plus text (optional).

Mobile





The anatomy is the same as in the mobile version.

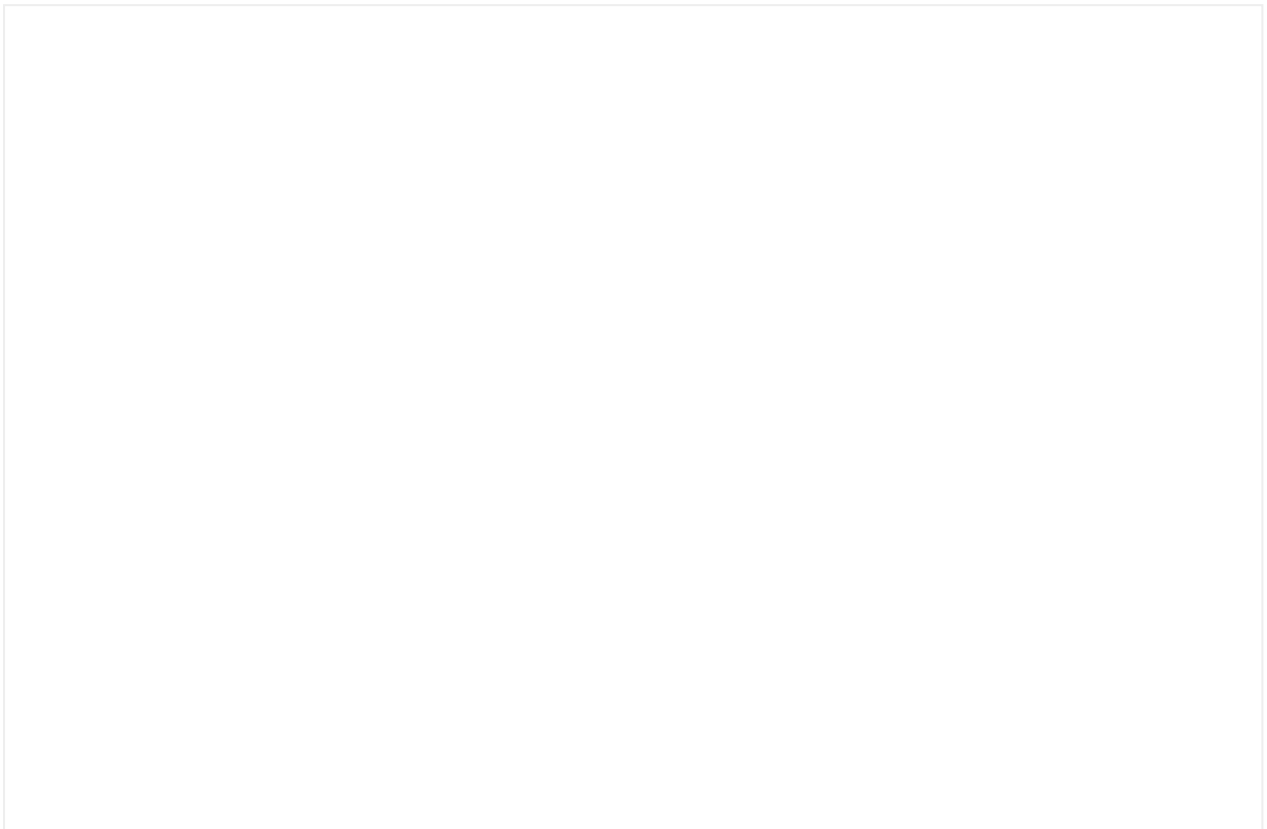
In this case, we will get rid of the text in the secondary navigation items.

Implementation

Additional info

Popover

Interactive demo



Contextual Guide

We define contextual guides as messages around a specific component of the interface, with the intention of giving users more information about their use or simply drawing their attention to them.

Copywriting in popovers and tooltips should be specially short and concise. Popovers are not used for promotional messages, use only with descriptive and functional content.

Usage

Popovers can be triggered automatically or by user interaction and they always refer to specific elements of the interface, pointing to them with their arrow (pointing up or down depending on the position of the element they refer to).

Users should always be able to close a popover tapping a close icon inside it.

More than one popover can never be shown in the same screen

Slot

Slots inside the component allow the customization of the component content.

Their main function is to offer flexibility and scalability for a component to be adapted to the specific requirements of each product.

Popover slots in native

Popover slots only work in web implementation, the native component only allows asset, title and description.

Implementation

Additional info

Progress bar

Provide feedback about progress of a task or process.

Determined



Usage

The progress bar can be used in different contexts:

- To display the progress status where uploading or downloading information is known.
- As a Stepper, where the number of steps can change during the flow (indeterminate stepper).

Some cases where the progress bar is best avoided:

- In an upload where the percentage or estimated time until completion is not known. In this case, it is best to use a **Spinner**.
- To represent data or a progress within a determined range, use the **meter** instead.
- For the pre-loading of a module. Skeletons are used for these cases (to be added to Mística).
- As an interactive element. In this case, it is best to use a Slider (to be added to Mística).

Stepped



Stepped progress bars help the users to track the progress through a set of predefined milestones

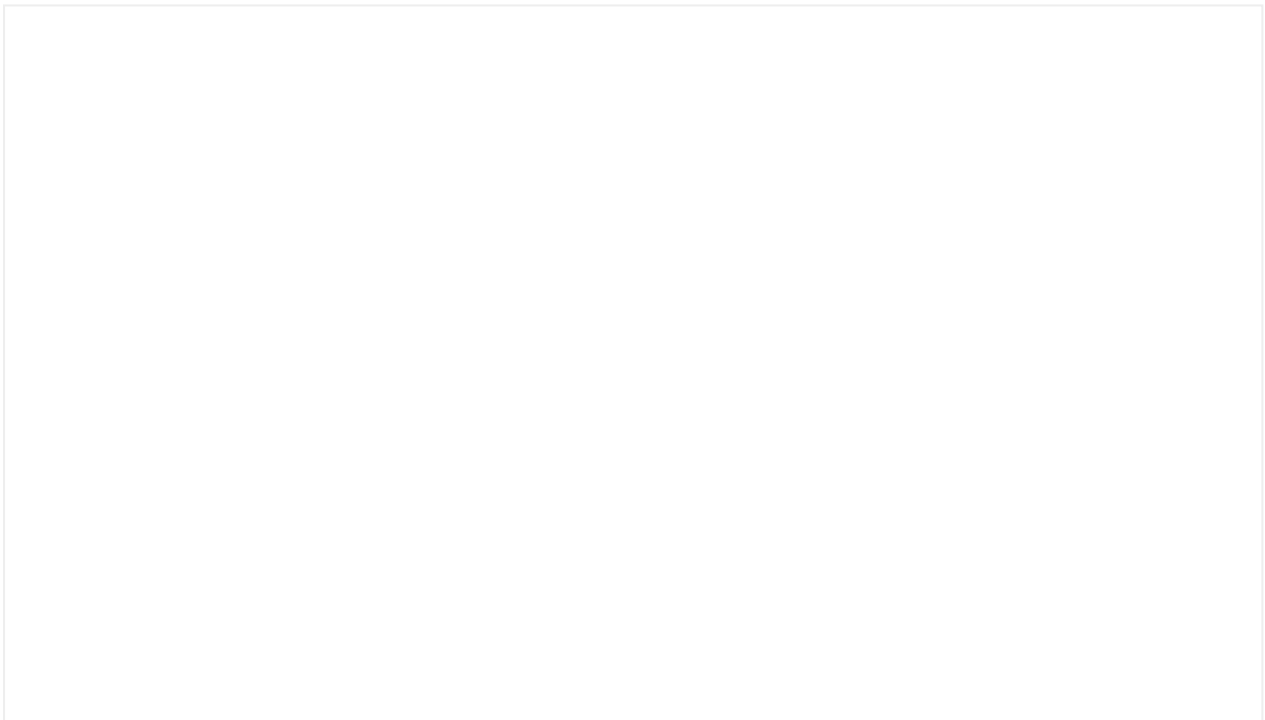
Usage

The progress bar stepped can be used in different contexts:

- To display the progress status where a list of clear milestones is present.
- As a simplified Stepper, where the number of steps is determinate.

Dynamic color

You can control the color of the progress bar, this is an example of how can you create a progress bar with dynamic colors. strength meter using a progress stepped:



Accessibility

Implementation

Additional info

Radio button

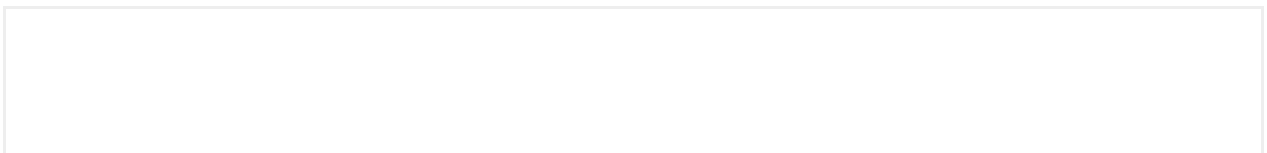


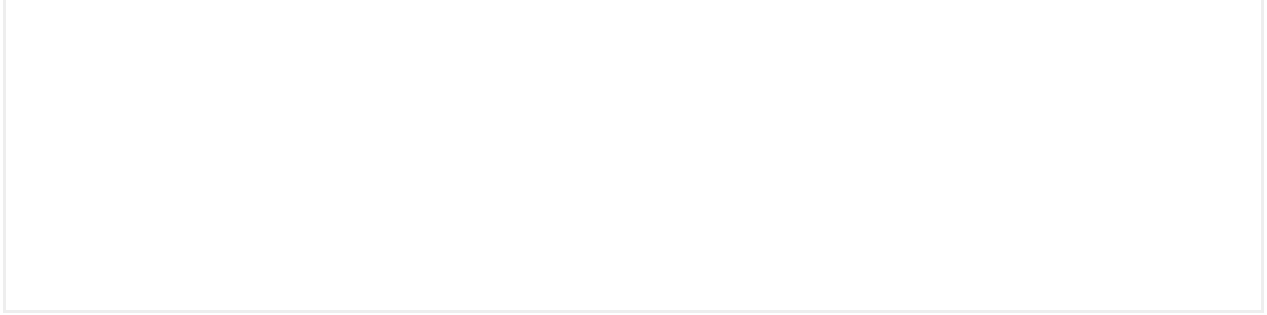
We use these if we have a list of mutually exclusive elements and the user must select just one. In other words, if an item is selected, another item in the list that was previously selected will automatically be deselected.

We will use radio buttons in lists with a minimum of two items and a maximum of seven. For longer lists, we advise using a drop-down.

Custom radio

You can also use a radio button with a custom design to create a little pieces of components to cover your needs.





Implementation

Additional info

Rating

This component allows users to rate their satisfaction with the quality of a product, service or experience.



The minimum number of items is 2 and the maximum is 5.

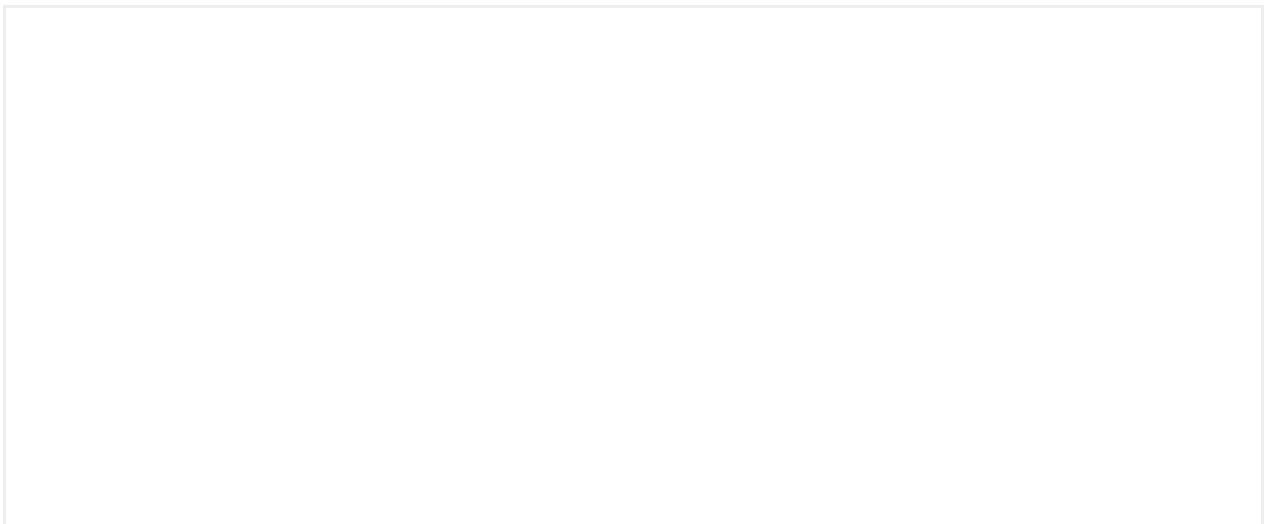
You can use Mística icons or custom icons but you will always need an outline icon and a fill icon. The colour of the icons is also customisable.

Usage

- Use the rating component for capturing qualitative or quantitative feedback with a clear scale.
- Use the rating for binary choices (e.g., like/dislike)
- When precise numerical input is required use a **number field** instead.
- For complex feedback that requires detailed explanations opt for a text area or survey form.

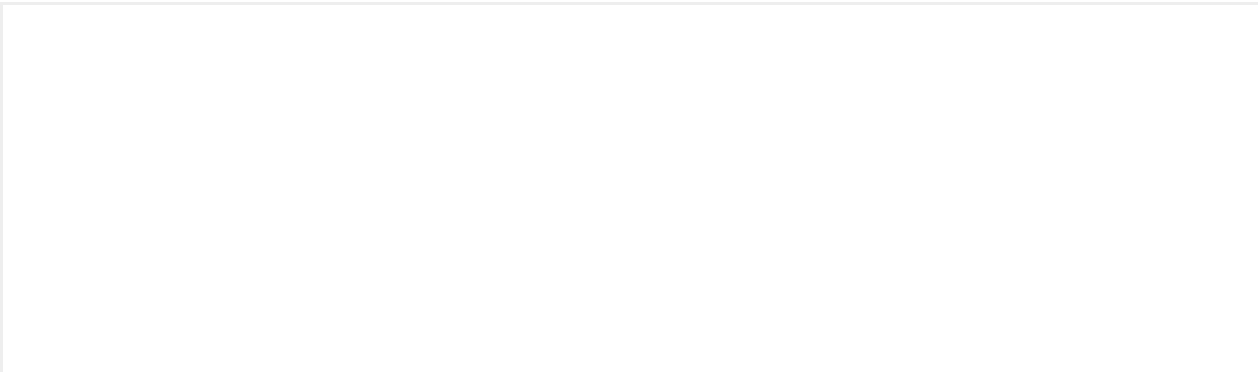
The rating has two variants:

- **Rating quantitative:** The rating scale consists of a single value in different degrees.
- **Rating qualitative:** The rating scale consists of different qualitative values.





Info Rating



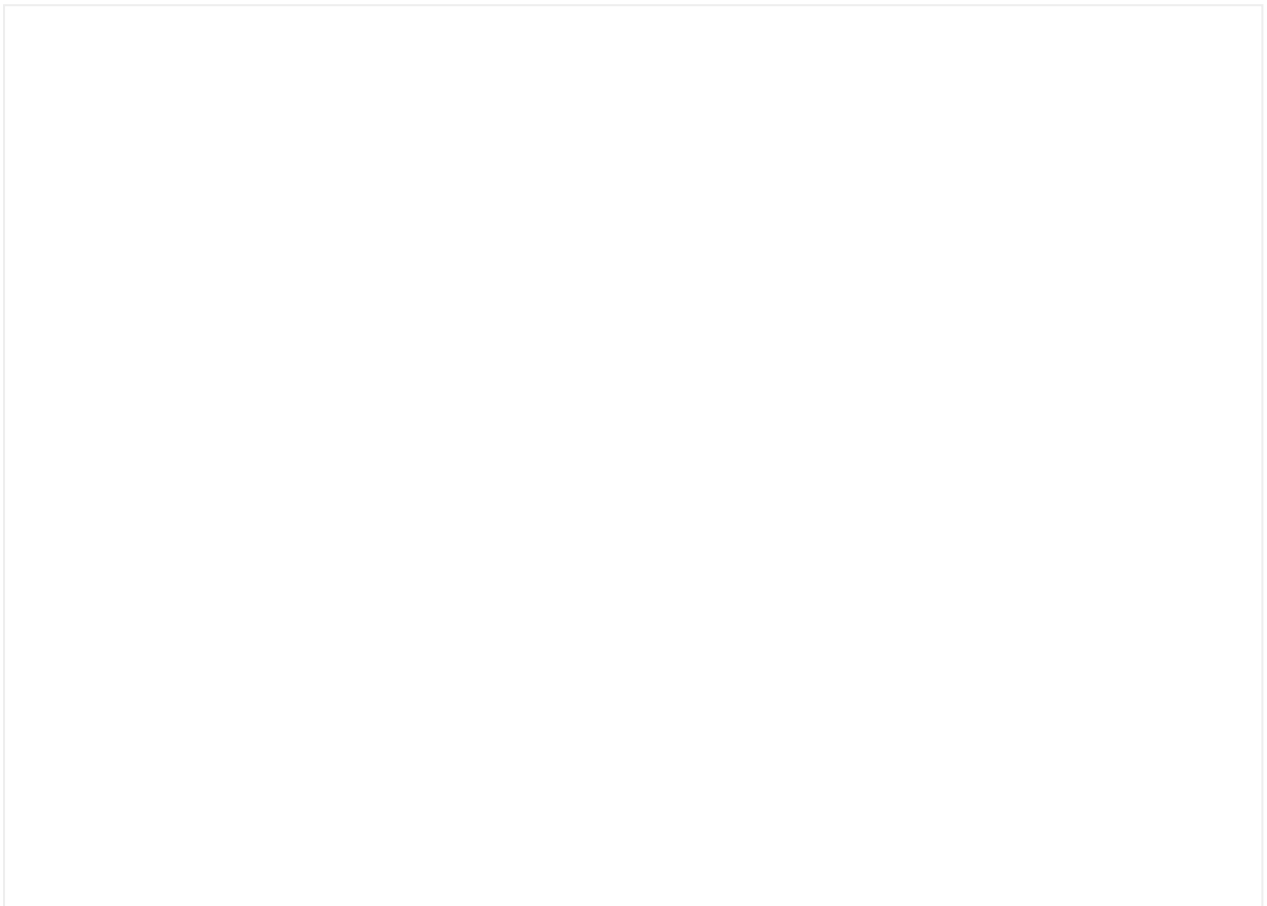


Accessibility

Select

Allow users to make a single selection between multiple options.

Interactive demo



Usage

- Use a select when a choice of more than 2 options is presented. If there are less than 3, we recommend the usage of a **radio group**.
- Keep select options as straightforward as possible.
- Use the native select when most of your experience is focused in mobile devices.

For a better understanding of input usage in forms read our [forms documentation](#)

Anatomy

- **Label**
- **Value**
- **Dialog**
- **Helper text** (Optional)

Sizing

Container width

The default **width** of the container varies depending on the viewport size:

- Desktop: 328px
- Mobile: 100%

Dialog width

When the option text exceeds the select width, the dialog width will grow to fill the available space before breaking into two lines.



Accessibility

Implementation

Additional info

Sheet



Typology

A sheet is a modal container that allows users to access to content details or perform contextual actions.

We have 6 different types of sheets:

- Single selection
- Informative
- Single filter
- Multiple filter
- Action list
- Actions
- Custom (only for 100% native or web products)

When to use

- For quick contextual selections
- For additional relative content
- For simple filtering options

When not to use

- For selections of more than 8 items
- For critical selections that may require user confirmation
- For single selection actions inside forms. Consider using a select instead

Anatomy

The sheet is divided into 2 sections, header and content, and contains the following elements:

1. **Handle**
2. **Title** (optional)
3. **Subtitle** (optional)
4. **Description** (optional)
5. **Interactive content** (required, depends on the sheet typology)
6. **Actions** (optional, depends on the sheet typology)

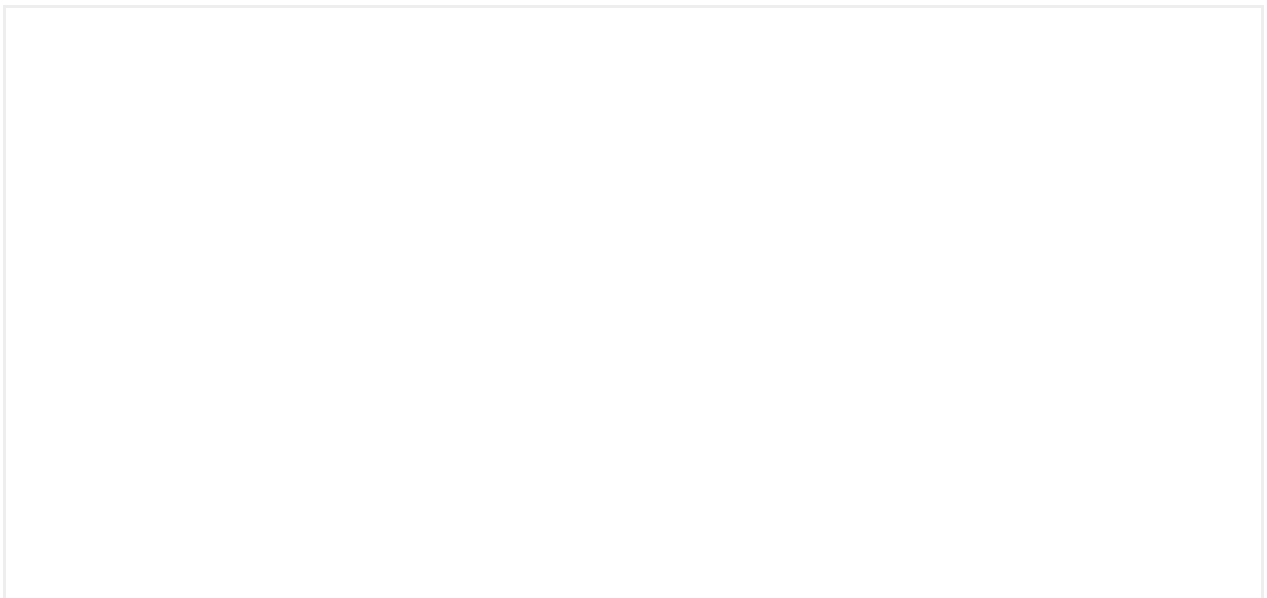
Size and position

- The maximum height of the sheet is 70% of the screen
- The sheet covers the entire screen width
- The sheet appears fixed to the bottom

Subtitle and description

- They are meant to provide additional information about the required selection
- It is recommended to keep them direct and short
- When needed, they can fall into multiple lines

Single selection





Although the row component offers multiple configurations, the single selection sheet rows are restricted to the following scenarios:

- Can contain a title or a title and a description
- Can have the following assets:

Informative

Use informative sheets to provide additional relative content to the information displayed on the page, in addition to the default title, subtitle, and description, informative sheets can contain lists.

Each list item is formed by:

- List item label or a list item label and an item description
- The following assets:

Single filter

Use single filter sheets to provide filtering options for the information displayed on the page.

This sheet typology contains **choice chips** that allow the user to select one option at a time. This is useful when you want the user to **choose only one of the possible options**.

Multiple filter

Use multiple filter sheets to provide filtering options for the information displayed on the page.

This sheet typology contains **filter chips** that allow the user to **select multiple filter options** simultaneously. Instead of choosing just one option, the user can select multiple chips to filter items based on different criteria. This sheet requires the user to confirm the filtering through the button.

Action list

An Action List sheet displays a structured list of actions.

Use them for contextual actions of related content.

- The actions list can include one destructive action
- Each action row can include an icon

Actions

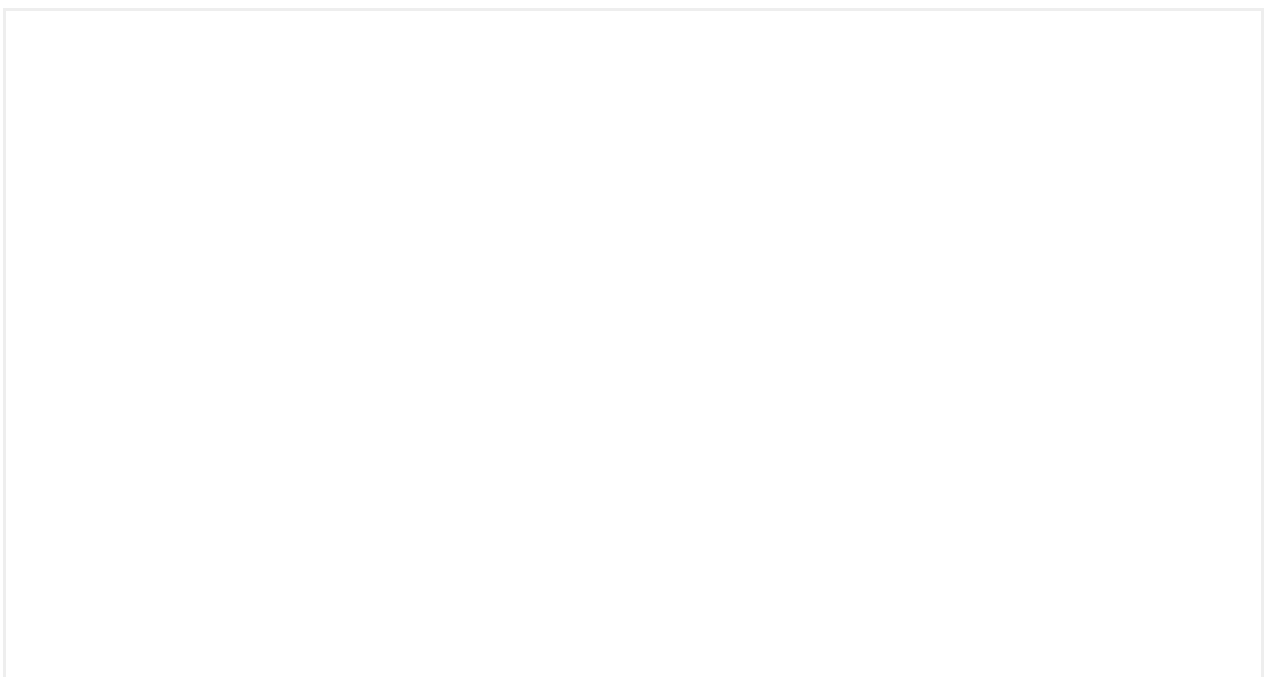
An Actions sheet displays a set of contextual actions. These actions are presented as buttons and are usually quick response options or secondary actions related to the current screen or task.

When use Actions or Action List

While an Actions sheet focuses on presenting a set of quick actions as buttons, an Actions List sheet provides a more detailed view of the actions. The choice between the two depends on the context and specific needs of the user interface and the actions to be presented.

Custom sheet

The custom sheet is actually the origin of this component, a presentation mode where the designer can freely design it. The typology of the sheet arises from the need for those hybrid products to make use of this component through a technical limitation, which is the Bridge, it is the way to communicate between native and web parts.





Behavior

Close and dismiss

The sheet will close when the user taps on one of the option items.

Sheets can be dismissed by performing the following actions:

- Drag down from the header or, in case there is no scroll, from the content area
- Tap outside the sheet

Scroll

When the content overflows over the content area it becomes scrollable, leaving the title and handle fixed to the top of the sheet.

Web (desktop)

To display the information on desktop devices properly, the sheet follows the dialog

component anatomy underneath. The behavior also differs from native and responsive implementations.

Anatomy

The sheet is divided into 3 sections: header, content and actions

Apart from the elements previously described, desktop presents:

1. **Dismiss button** (required)
2. **Action button** (only required for single selection type)

Size and position

- The maximum height of the sheet is 64px less than the viewport height
- The sheet has a maximum width of 680px
- The sheet appears centered on the screen

Behavior

Close and dismiss

The sheet will close only when the user confirms the selection previously done.

Sheets can be dismissed by performing the following actions:

- Using the top-right close button
- Through a button defined to perform that action

Scroll

When the content overflows over the content area it becomes scrollable, leaving the title and handle fixed to the top of the sheet.

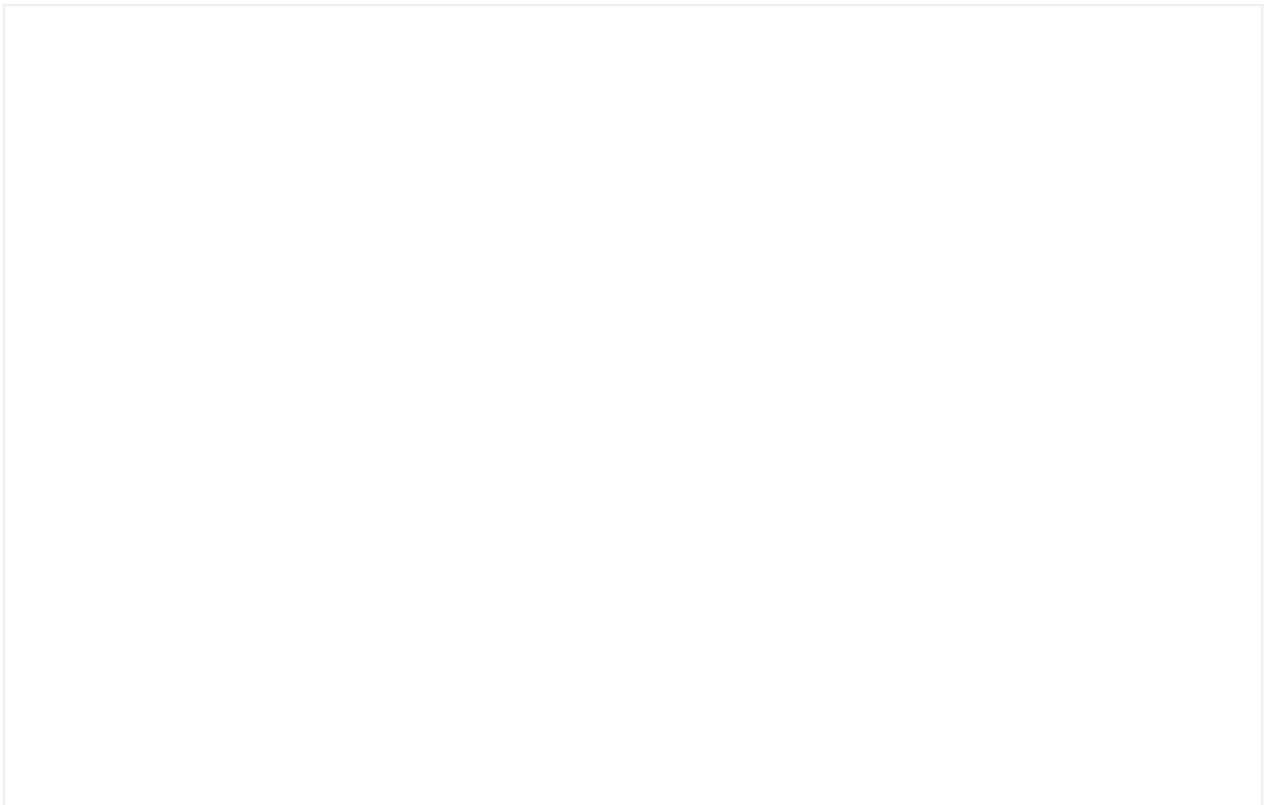
Accessibility

Implementation

Additional info

Skeletons

Skeleton gives users a low fidelity representation of the content in the process of loading. This improves the user's perceived loading time.



Usage

When to use

- When display content takes an extended amount of time to appear on-screen.
- When loading data for the first time.
- When the content being loaded is predictable.

When not to use

- Don't use skeleton loading if the content can be loaded very quickly. This can cause a flickering sensation.
- To loads triggered by user action. Instead, use the **Spinner** component.
- If you need to load data in the background, use **Loading bar** component.

Predefined types

Mística has five skeletons key listed below:

Skeleton Text

Skeleton text should be used where text elements like headings, paragraphs, or labels will be rendered.

Skeleton Line

Skeleton text should be used where text elements like Title.

Skeleton Row

Skeleton row can be used to represent components like lists (also in its boxed variant).

Skeleton Circle

It is a more atomic type of skeleton to create compositions that do not fit with the rest of the skeleton types.

Skeleton Rectangle

You can use this type for more visual elements such as images, videos, charts...

These five types enable to creation any more complex combinations based on what will be loaded in the area.



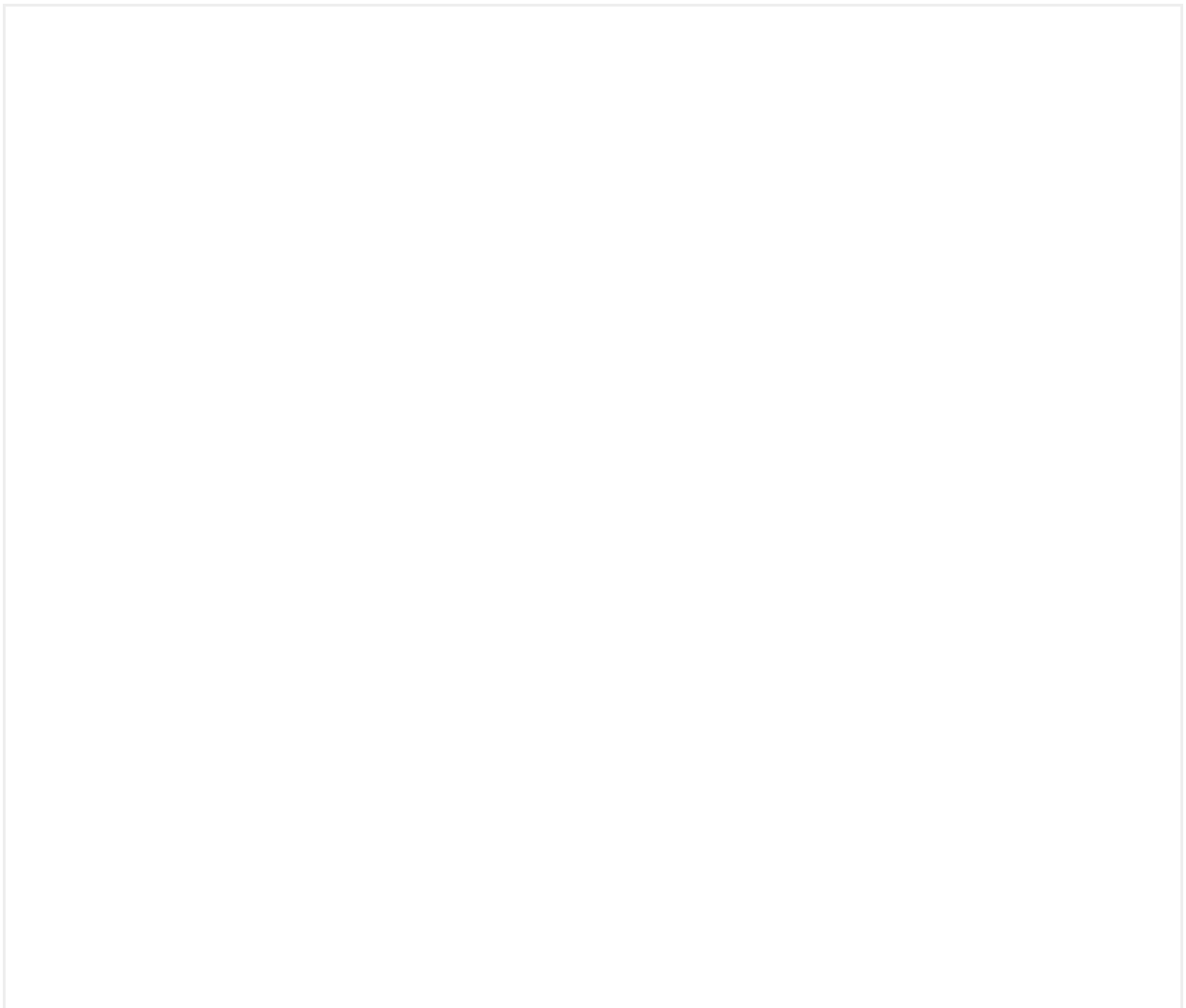
Behavior

The loading behavior is open to the definition of each product and can be incremental (the real content will replace the skeleton as soon as it is available) or it will be shown all at once.

When the information has been loaded, the real content will replace the skeleton component.

Animation

Skeleton have Pulse Animation Effect to further decrease perceived duration time and a Fade Animation transition when loading ends and content appears.

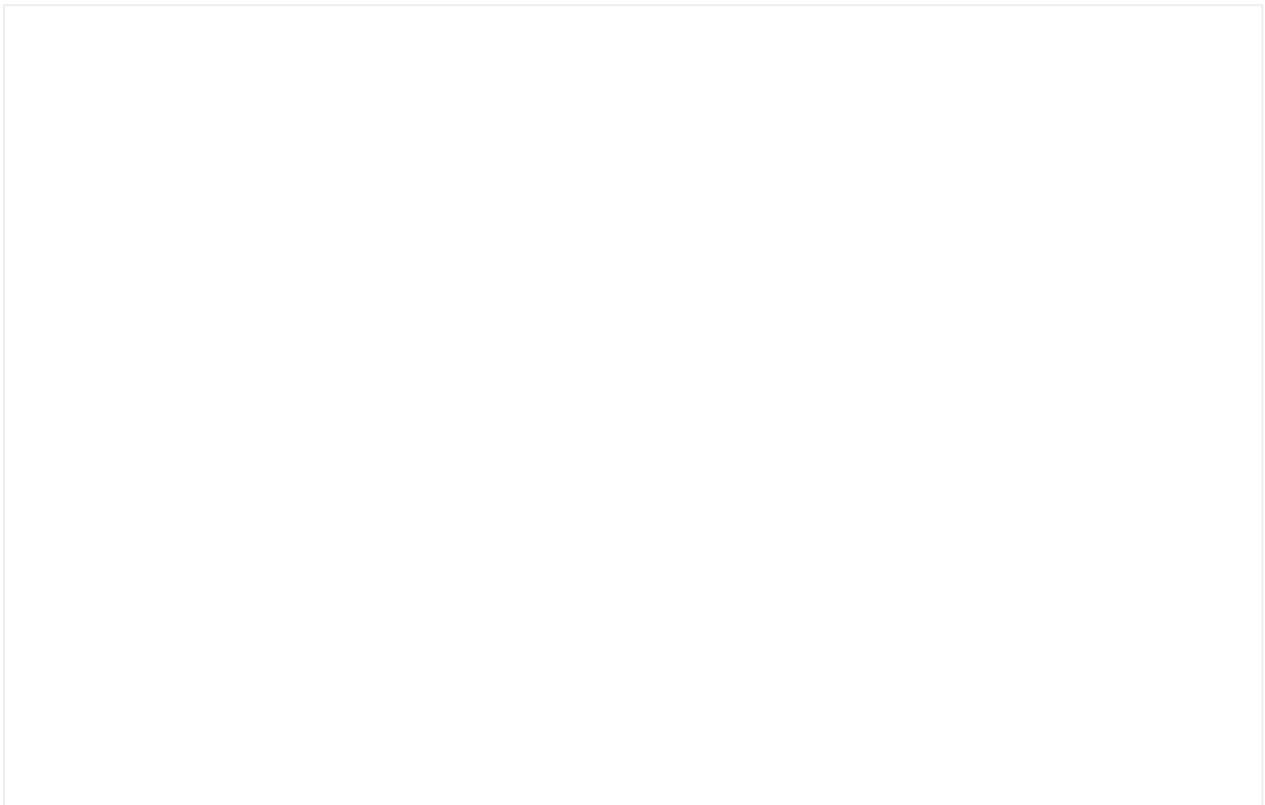


Implementation

Additional info

Skip link

A skip link allows keyboard and screen reader users to bypass repeated blocks of content and jump directly to other content of the page.



Usage

When to use

- When a page contains repeated elements at the top (navigation bars, headers, menus) that could force keyboard users to tab through many links before reaching the main content.
- To provide quick access to specific sections of the site, such as a footer or search panel.

When not to use

- On very simple pages with little or no repeated navigation where skipping isn't necessary.
- When focus order is already minimal and predictable.

Behavior

- The skip link should be the first interactive element on the page.
- It remains hidden visually until it receives keyboard focus.
- On activation, it moves focus directly to the designated landmark (commonly the <main> content area).
- Multiple skip links can be provided if needed (e.g., "Skip to search," "Skip to footer"), but keep them minimal to avoid clutter.

Accessibility

Implementation

Additional info

Slider

Allows users to choose a value by sliding a control horizontally



Anatomy

The basic structure of the slider consists of:

- A track that displays the selected amount and the remaining amount with a color difference.
- A handle that allows the user to interact with the component by sliding it left or right.

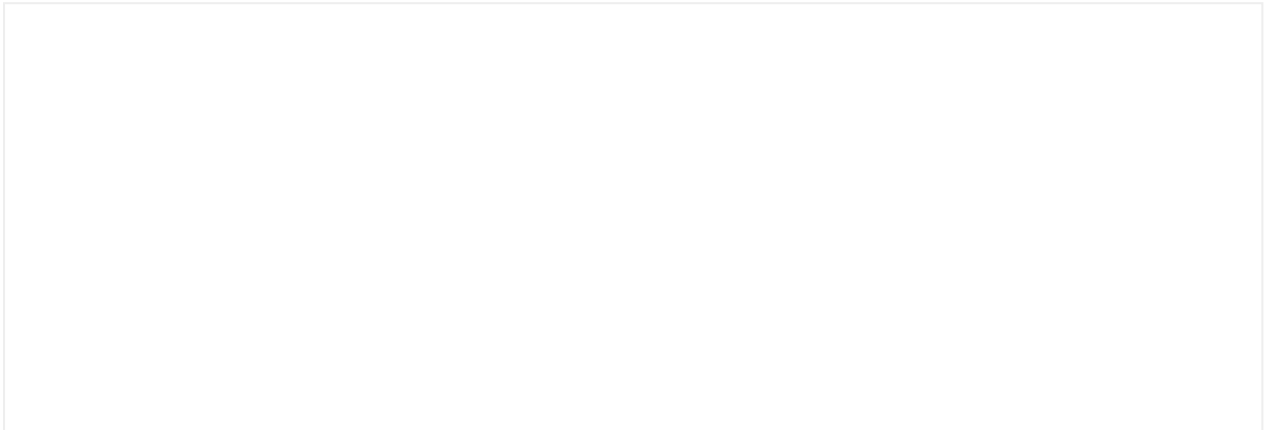
Our slider also provides the option of using a tooltip to display information about the slider value.

Alternatively, you can also retrieve the slider value and use it in text outside of the component.

Custom values

The Slider accommodates a range of values that extend beyond the conventional 'steps' logic.

Accessibility



Snackbar

Snackbars provide messages about app processes at the bottom of the screen. Also known Croutons in iOS.

Interactive demo



Usage

Success messages

For small and very specific actions where it's not possible to communicate success implicitly and where a success screen component may be unnecessary (for short processes or those not related with purchases, balance, upgrades, etc.) consider using a Snackbar (Crouton in iOS).

Please use this components sparingly, as we should communicate implicitly the success of any action or process whenever possible.

Error messages

iOS and Android guidelines recommend not using alerts and dialogs too frequently, as they should be reserved for high priority messages we want users to see no matter what. For errors of low importance.

We define low importance errors as those that don't deal with sensitive information or purchases (top up, upgrades, passwords...), for instance a language setting that couldn't be changed or a new group conversation name that couldn't be set.

System messages

We define system messages as those that communicate information that is related to the state of the device and are usually transversal to all sections of the product. for instance, messages related with the lack of connectivity belong to this category.

Anatomy

- **Message:** A message that provides the user contextual feedback.

- **Action** (Optional): The snackbar action will always dismiss the snackbar when pressed. Depending of the length of the action can be displayed inline with the text or at the bottom-right of the snackbar container.
- **Dismiss** (Optional)*: Dismiss is recommended for infinite durations when the action is not required. It can be also used when the duration is different from infinite.

Duration

- **Default**: The duration by default is 5s, if the snackbar has an action, 10s.
- **Infinite**: The duration can be configured as infinite.

Content

Please use short, descriptive and easy to understand copywriting. Avoid technical jargon and alarming language. Also try to keep messages short enough to fit on one or two lines and keep the component size small enough.

Positioning

The Snackbar is always placed at the center and at the bottom of the screen. On mobile, it is separated by 8px from the bottom, while on desktop it is separated by 40px.

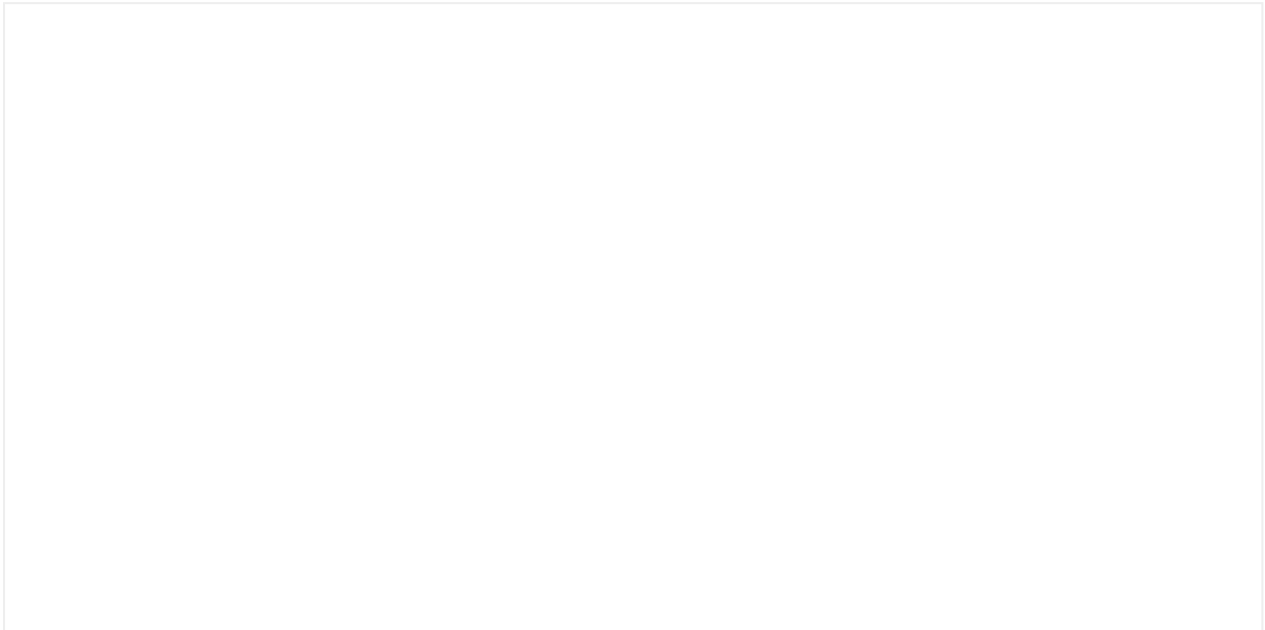
Accessibility

Implementation

Additional info

Spinner

Provide feedback to the user about their request being processed.



Usage

It is always preferred to show an error in the same page. This way, users can easily retry the action or edit something if they have to. In these cases, if the action to submit provokes a loading time, it should be indicated in the submit button with a spinner. You can see how to apply spinner in buttons [here](#).

Consider using the spinner with or without a title depending on the expected load time. If the expected load time is high enough (500 ms and on) for users to be able to read it, you should include a title with the spinner. This is critical for very high expected load times (3s and on), where you should inform users of the expected load time with as much precision as possible (for instance, use "a minute" instead of "a while").

Use a spinner when:

- The content to be loaded is unknown (e.g. third-party webviews).
- There is a partial load triggered by user request.

Don't use a spinner when:

- It is the initial content load. Use a skeleton instead.
- The load is not triggered by user action. Use a [loading bar](#) instead.

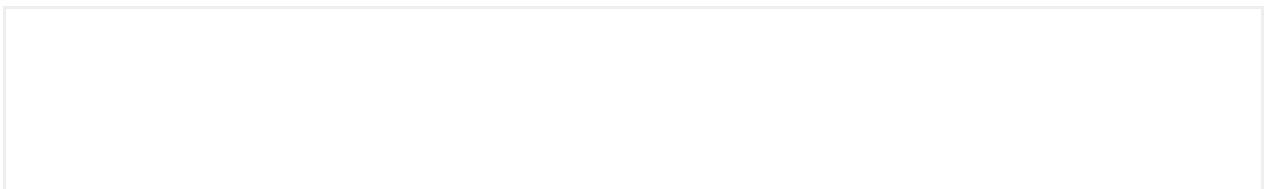
Accessibility

Stacking group

The StackingGroup is a container that aligns elements, allowing visual overlays and visual groupings



Usage





Accessibility

Stepper

Steppers guide users through complex processes to make them simple and intuitive. They reduce the users frustration and help them to complete tasks properly.



Accessibility

Implementation

Switch

Useful if we only have one option to activate.



Accessibility

Implementation

Additional info

Table

A table is an arrangement of data in rows and columns, or sometimes in a more complex structure.



Content type

We can differentiate two different types of cells in a table:

- **Header row cells:** You can only use text inside, no customization allowed.
- **Row cells:** You can use simple text inside or any custom elements.

Behavior

Responsive

The component gives you two possibilities to use in small screens:

- **Default:** the table behaves as in desktop and a horizontal scroll would appear to see the rest of the content.
- **Responsive:** The table automatically converts each row into independent modules (boxed or not) that will be placed vertically on the screen, with default text styles.

Column alignment

The column alignment can be changed between left and right, right aligned columns are useful when you need to display numeric data.

Custom height

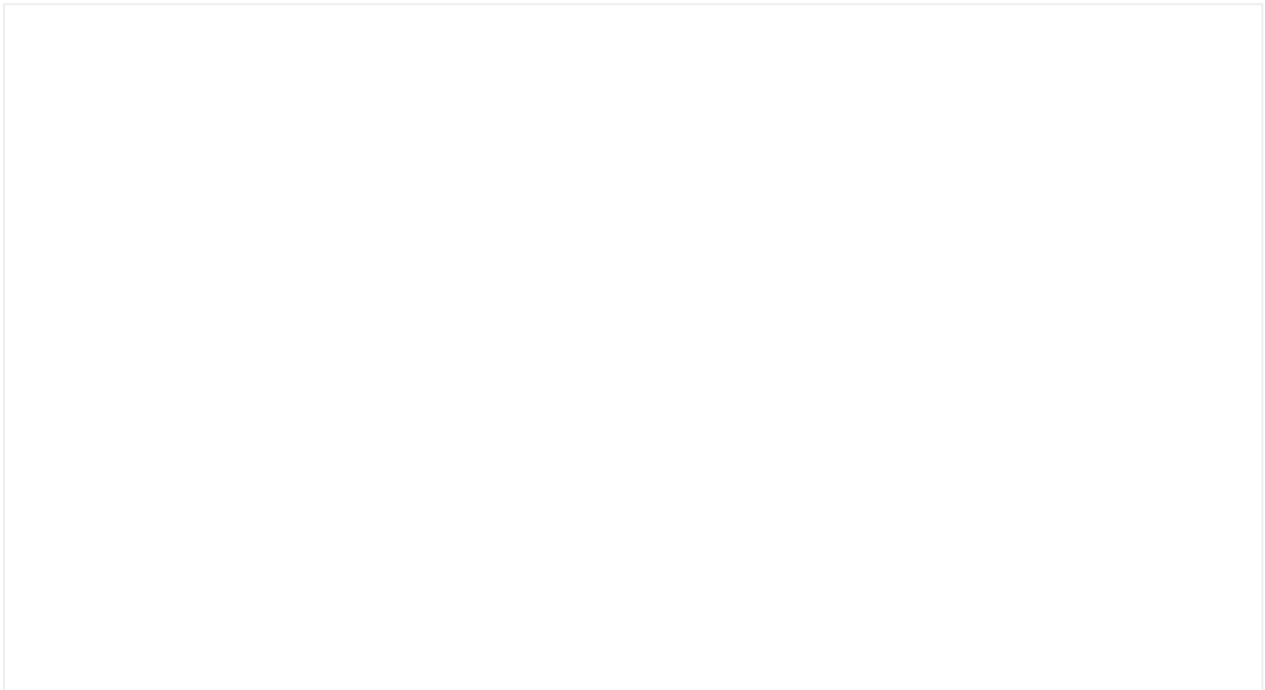
You can set a custom height for the table. When the content exceeds this height, a scroll bar will appear inside the table.

Accessibility

Tabs

Tabs allow us to organize and group related content to navigate quickly between sections inside them (same context or same page).

Interactive demo



Usage

The content presented in each section of a tab must keep the same type of hierarchy and, at the same time, that content must be different.

Usage recommendations:

- Use up to 6 tabs and a minimum of 2.
- Don't use complex texts, one or two words should be enough for this purpose.

Additional info

Tab bars



Typology

The TabBar allows browsing between the main sections of an application.

We will always use the TabBar with **Icon + Label**, as displayed in the following image.

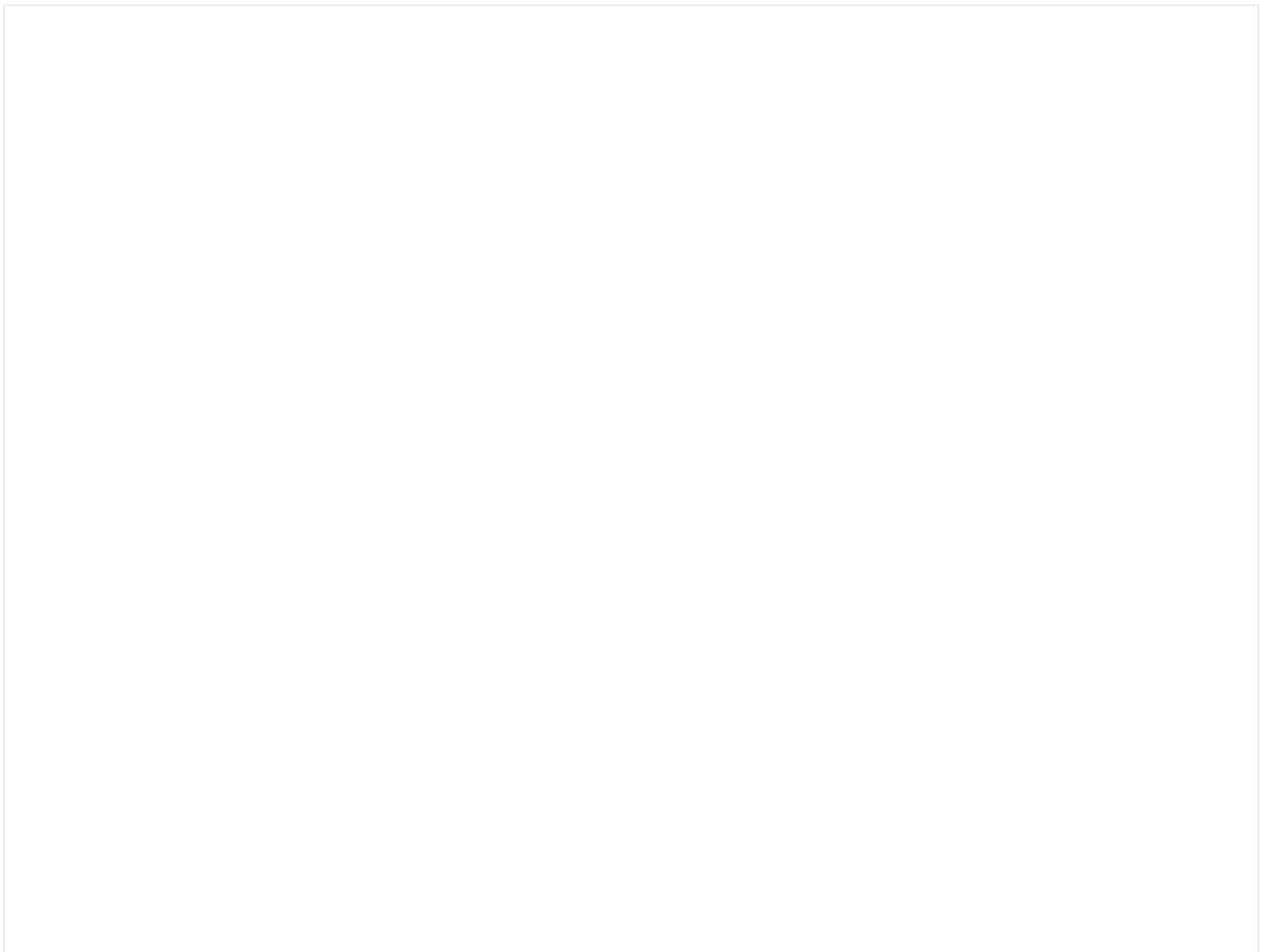
- Minimum 2 tabs
- Maximum 5 tabs

Additional info

Tag

Tags are compact elements with color codes to manage states.

Interactive demo



Usage

We use the tag components to label, categorize or organize information and components.

Try to use short labels for easy scanning of the various tag states. In addition to the states, all typologies support a **badge** if you need to provide additional information associated with the tag.

The categorization is established as follows:

- **Active**

Use brand color, for this reason is the best choice to use this component for neutral cases. Also used to set in progress states.

- **Inactive**

Commonly used for showing deleted, inactive or closed cases.

- **Promo**

Exclusively to highlight promotional content

- **Info**

For informative content

- **Success**

Commonly used for success and completed states

- **Warning**

Commonly used for showing alert state or pending processes

- **Error**

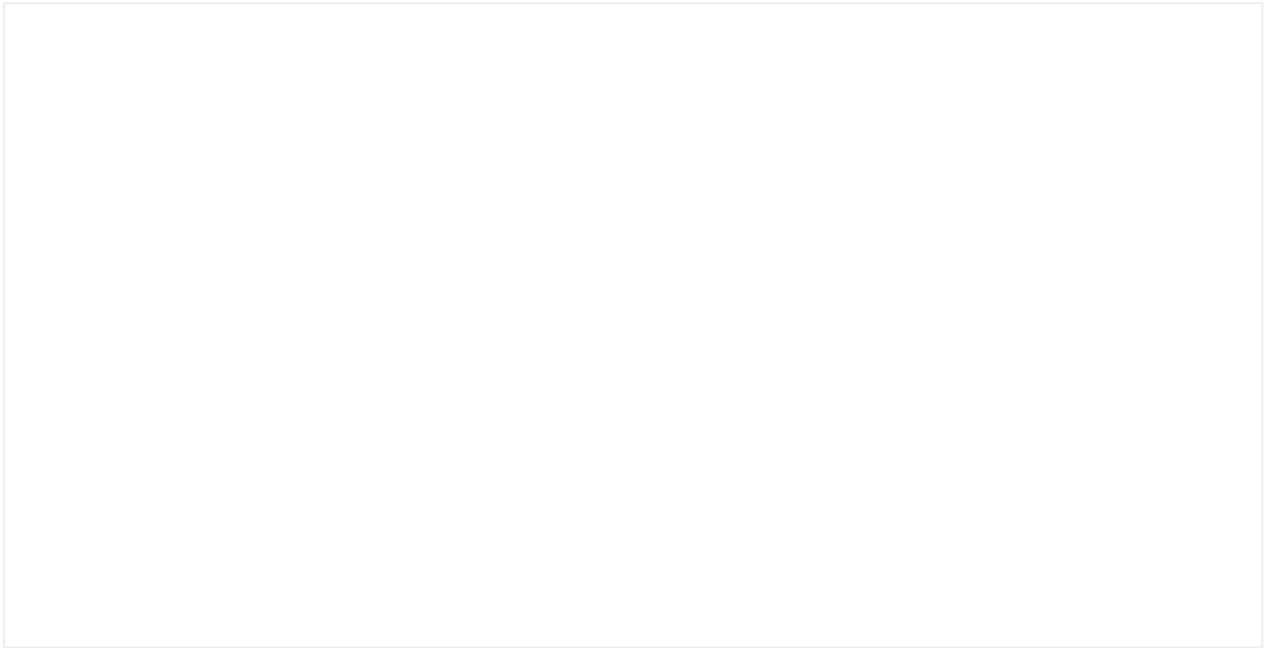
Commonly used for error, danger, or emergency cases where the user needs to make an important action

Implementation

Additional info

TextLink

Use text link (or hyperlink) to create inline linkable text.



Usage

You can use this element to:

- Navigate to other pages
- Link phone numbers or email.
- As an anchor link

Accessibility

Implementation

Additional info

Timeline

The Timeline component is used to represent events in a chronological timeline. It can be used to visually and organizedly display processes, stories, or temporal flows.

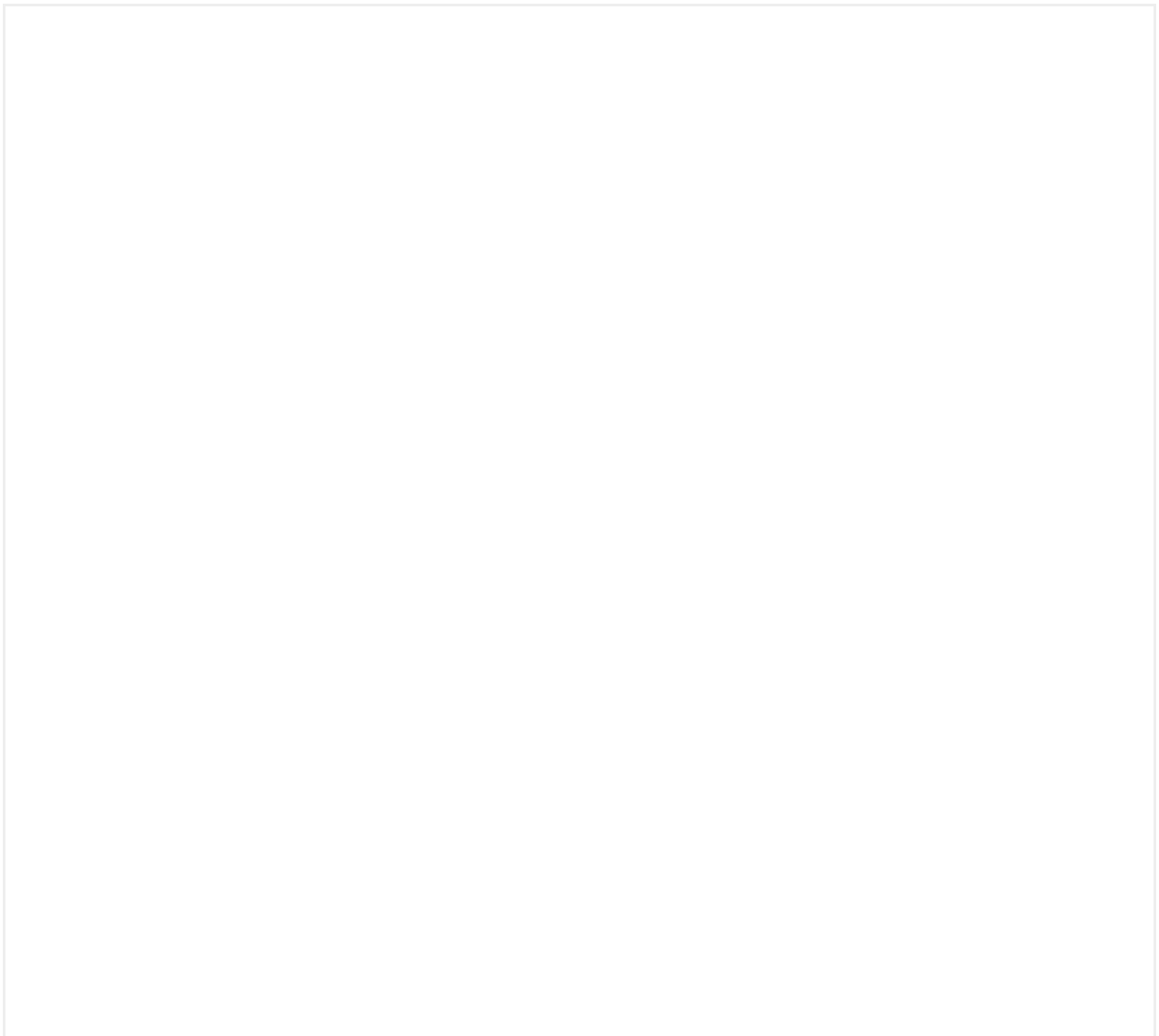


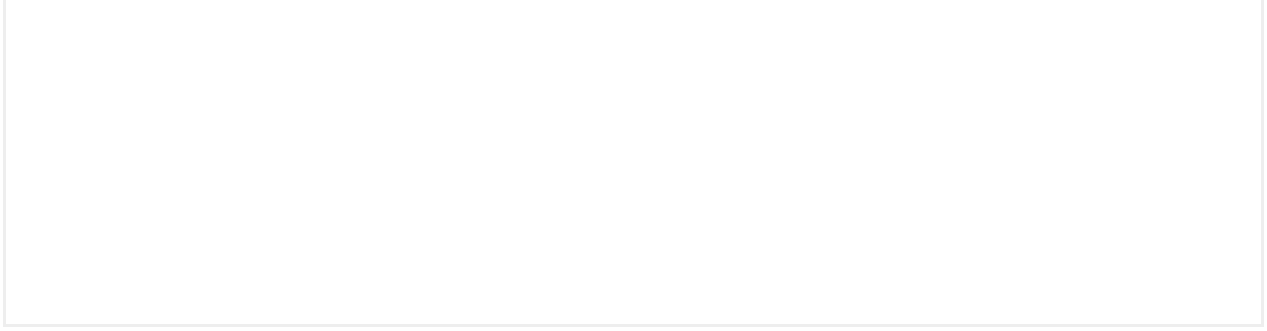
Usage

- Use the timeline component to describe a series of predefined or user-generated events that have a temporal relationship.
- Don't use a timeline to guide a user through a step flow use the **stepper** or **progress stepped** components instead.

Anatomy

Behaviour





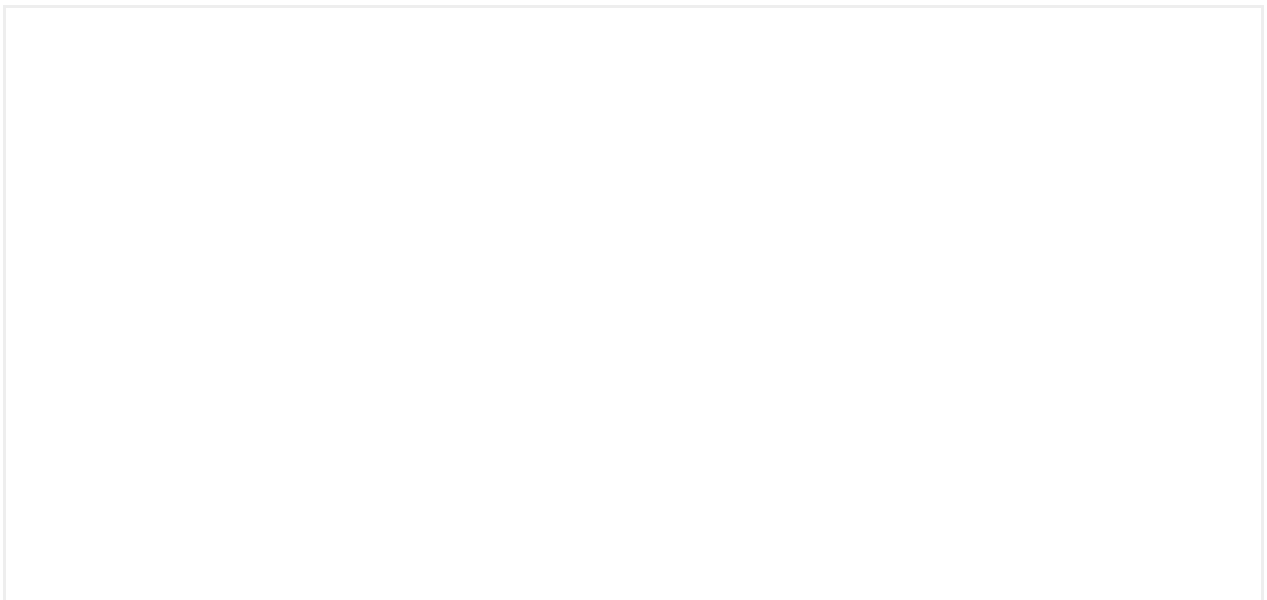
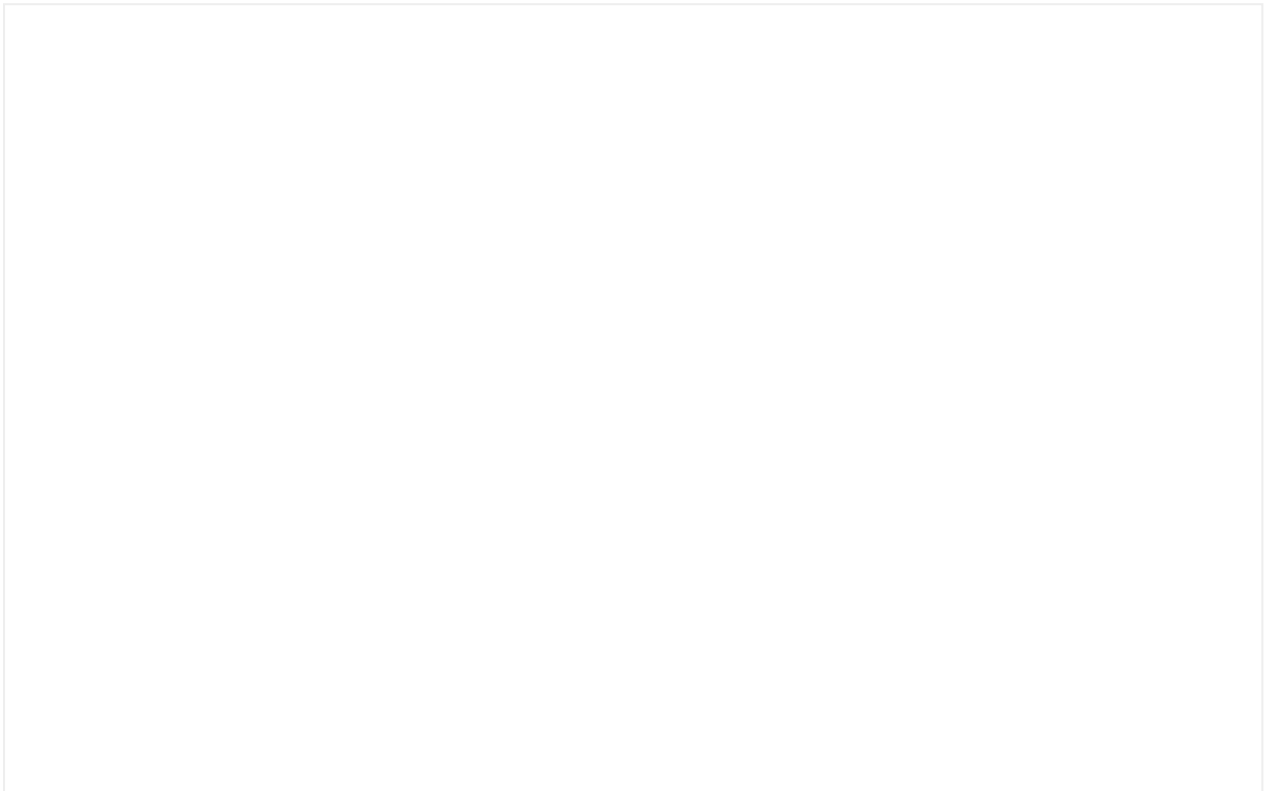
Accessibility

Implementation

Additional info

Timer

It generates a sense of urgency and is mainly used to indicate the time remaining before a promotion ends, an event starts, or the time left to complete an action.



Text timer

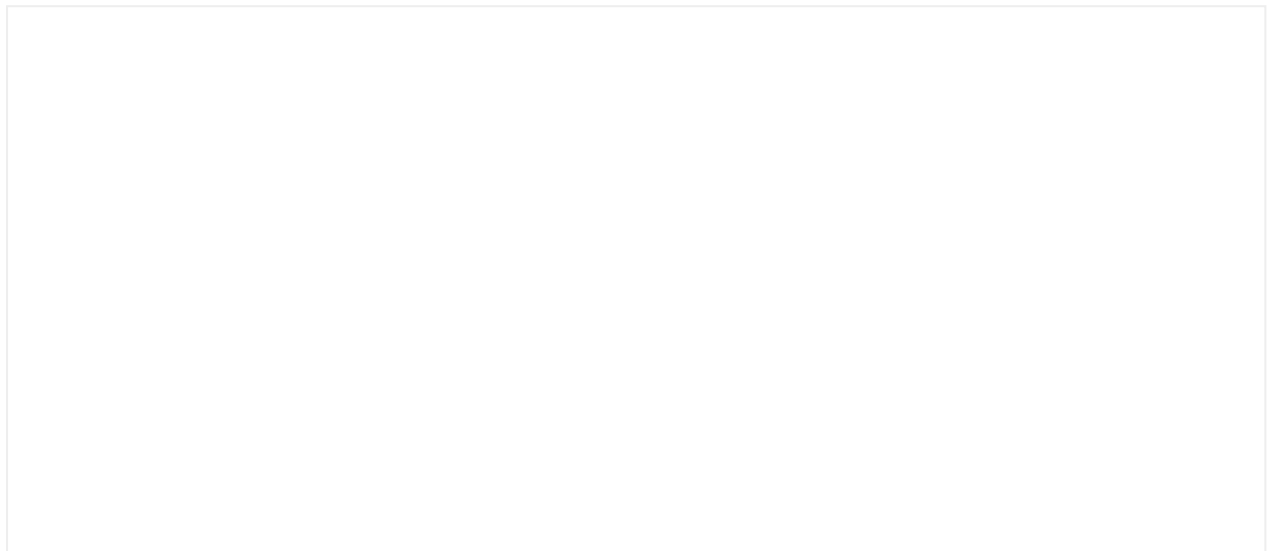
There are 3 types of text timers:

- **Without label:** The format is the same as a digital clock, it can display hours, minutes, and seconds.
- **Short label:** Includes the symbol of the time unit and can display up to days.
- **Long label:** Includes the full time unit and can also display up to days.



Customization

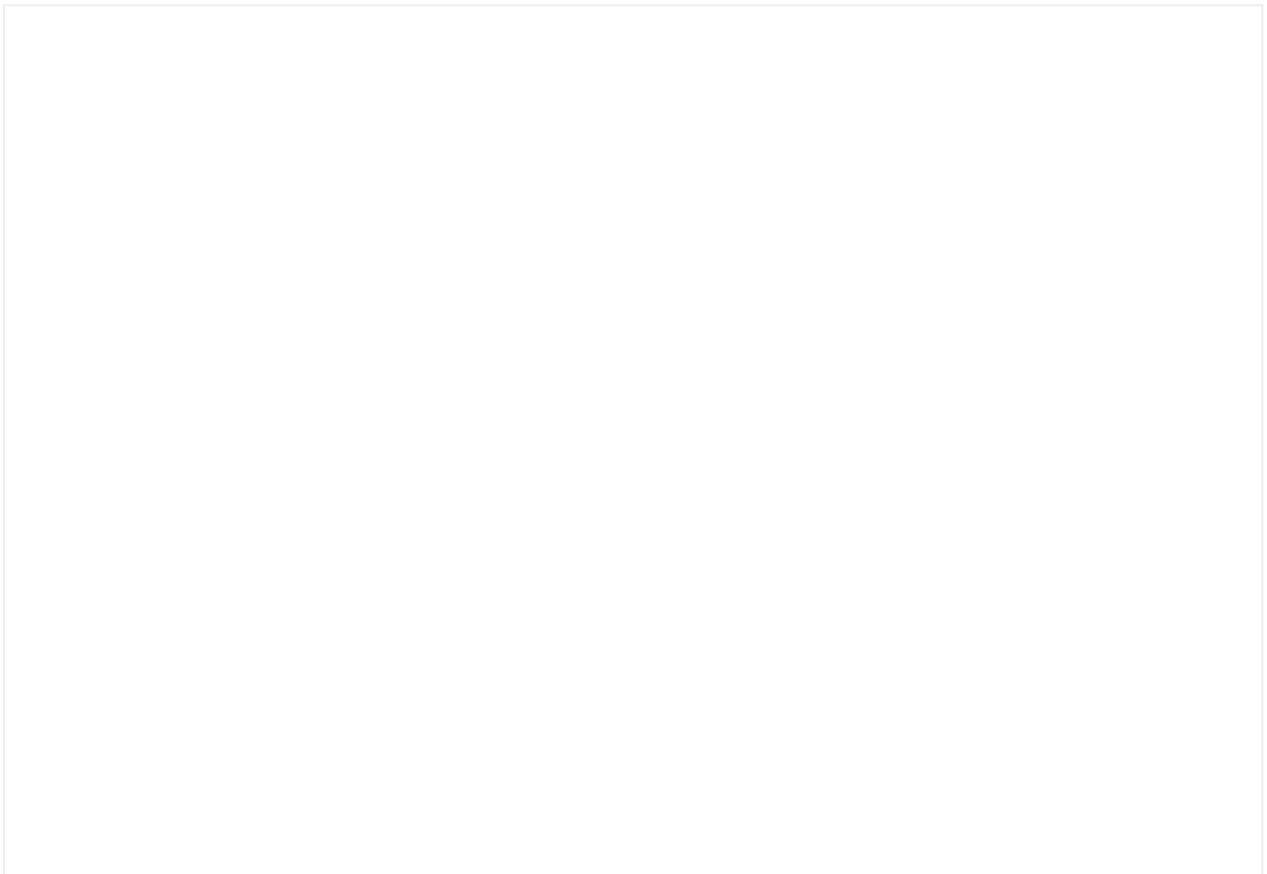
Allows the behavior of a timer to be introduced in any context and to customize the size, style, and color of the text.



Accessibility

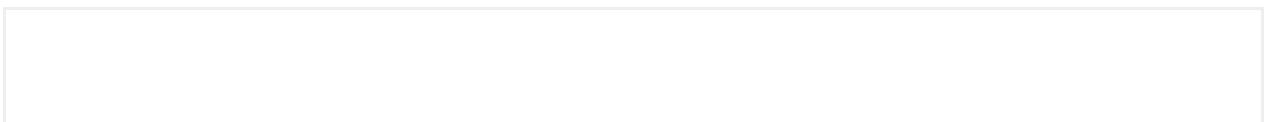
Title

It allows you to create organizations that make it easier for the user to give context to content blocks.



Title1

Title1 is used to separate more functional content blocks. For example, in a setting you might want to differentiate the type of the elements that comprise it without needing to generate large differentiations between these groups.



Title2

Title2 is designed to compartmentalize content within a section by creating a clear visual hierarchy between blocks of related information. It is ideal for dividing subsections within broader content areas, such as splitting different features, options, or steps in a process.

Title3

Title4

Title4 is used as a page heading in cases no other components can assume that role.

Navigation Link

An associated link can be added to the title. We recommend its use in order to expand the entire content of the section.

For example, for a carousel of elements in which highlighted elements are shown, we can use the title with a link that says "View all" to navigate to another list-type page where all the elements are shown.

Semantic hierarchy

Title component can change their tag to represent their level in the page hierarchy. Screen readers and assistive technologies rely on these tags to identify the page contents and structure. Changing the tag of the component doesn't affect the visual representation.

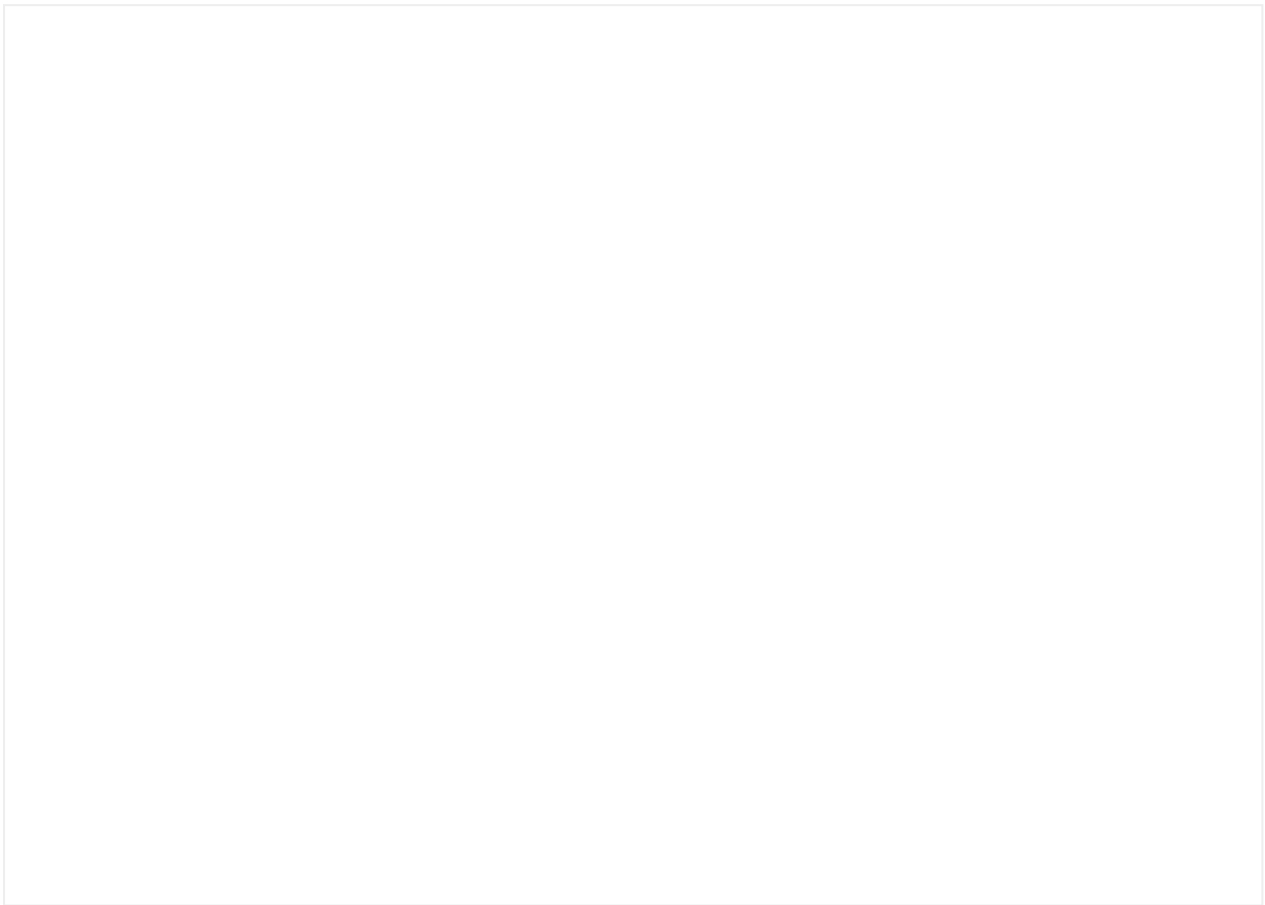
To read more about this topic: <https://webaim.org/techniques/semanticstructure/>

Implementation

Additional info

Tooltip

Display simple informative text when users hover over, focus on, or tap an element.



Usage

Mainly used in desktop screen

Don't use tooltips for information that is vital to task completion.

Users shouldn't need to find a tooltip in order to complete their tasks. Tooltips are best when they provide additional explanations for a form field unfamiliar to some users or reasoning for what may seem like an unusual request.

Provide brief and helpful content inside the tooltip.

Tooltips with obvious or redundant text are not beneficial to users. If you can't think of particularly helpful content, don't offer a tooltip.

Slot

Slots inside the component allow the customization of the component content.

Their main function is to offer flexibility and scalability for a component to be adapted to the specific requirements of each product.

Accessibility

Implementation

Additional info

Disabled states

To avoid the need to create customized properties, colors or logics for the 'disabled' status of each component, Mística proposes a transversal solution for any element to be disabled.

Any component to which a 'disabled' status is applied will appear with **50% opacity**.



Empty states

Empty States are used when there is no information to show to the user.

They are most commonly seen the first time a user interacts with a product or page, but can be used when data has been deleted or is unavailable.

Some examples of the ways in which Empty State screens can be used include:

The first time a new feature is used

If we had a "music playlists" feature, we would help the user by encouraging them to create their first playlist by adding the first song.

In a shopping cart when the user hasn't added anything

We would always try to offer the user an alternative when they find themselves on an empty screen.

A search that does not return any results

This would be a good time to help the user refine their search.

Anatomy

- Image or Icon
- Title (required)
- Description
- Actions

Empty State variants

Large image

It is constructed with a large image which covers the width of the screen on a mobile phone and the height of the component on a desktop. It has a lot of emotional and branded content, and it is perfect for the first times a product is used and for presenting its core functionalities.



Small image

It is constructed with a smaller image (112px high) for those moments when we don't need to have a high visual/emotional impact.

This type supports 16:9, 4:3 and 1:1 aspect ratios.



Icon

We can use icons inside an empty state. The icon is set to a size of 64x64, always using the "light" weight for the icon. It is intended to be used on more operational screens where the information given to the user is more functional than emotional.

[Learn more about the use of icons](#)



Empty state screens can also be used to show the empty state of a particular area by embedding it within the content of a screen.

This type supports 16:9, 4:3 and 1:1 aspect ratios.

Empty State Card



Implementation

Additional info

Feedbacks

Communicate to users when something is success, error or just informative.

Success Feedback Screen

By default, try to communicate the success of an action or process implicitly in the user interface without using a specific component. Ideally the result of that action or flow should be clearly visible in the very own interface:

Example: creating a support query through a multi-step process results in the redirection to the detail screen of that query just created.

Example: completing a "send balance" process from a conversation results in a message being inserted into it.

For long multi-steps processes or those where any kind of purchase is involved (plan upgrades, top up's, etc.) a success screen should be used. This is a specific screen that explicitly states the success of the process and is able to provide extra information or multiple actions to do next.

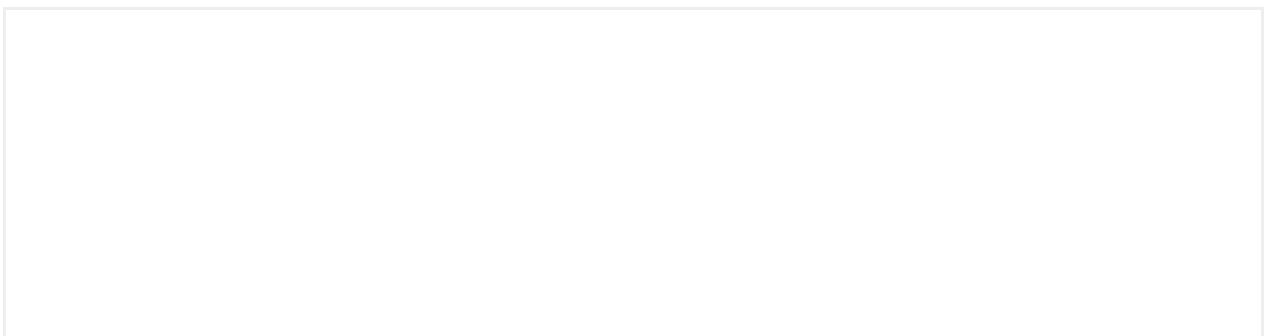
We define success scenarios as those where we need to communicate users that an action or process has been completed without any errors.

Use a maximum of two actions to guide users to whatever is best next based on the flow they just completed. Always make the most likely one prominent by using a primary button and display the others as a link to make a clear hierarchy.

Desktop success feedbacks can use image to step up a positive message



Desktop version



Error Feedback Screen

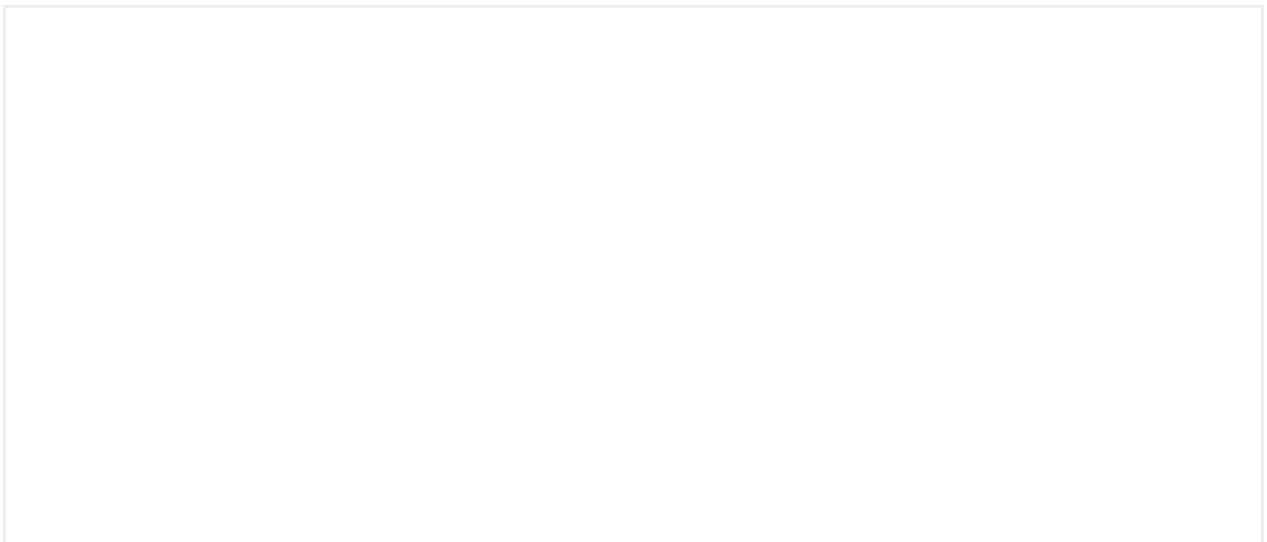
However, even though showing the error in the same screen should be preferred, technically it may not always be possible. In these cases consider using an error screen. This may also be useful in cases where we want to provide users extensive information and/or multiple actions, which may be difficult to accommodate in an alert or dialog.

Use a maximum of two actions to guide users to whatever is best next based on the flow they just completed. Always make the most likely one prominent by using a primary button and format the rest as links to provide contrast.

If the process or action that triggered the error can be retried it may be a good idea using a "Retry" button as the primary action, especially if the error is something that may be temporary and the chances of success when retrying are high.

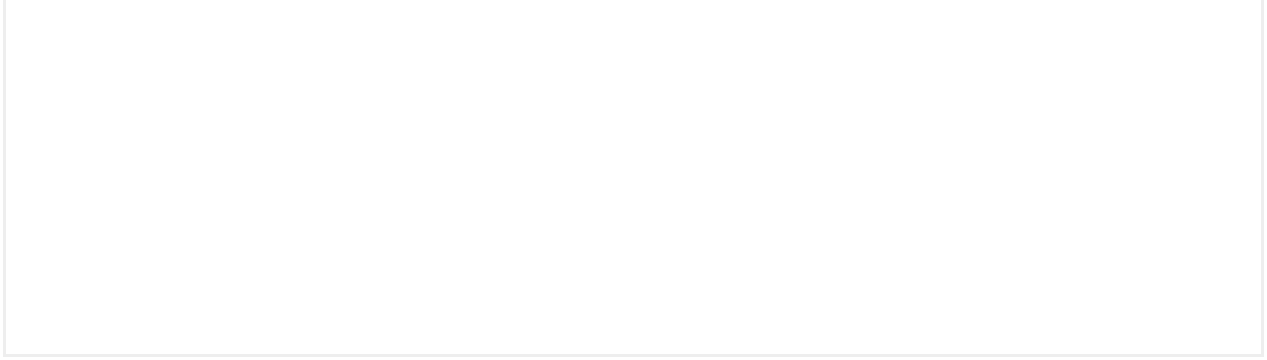


Desktop version



Info Feedback Screen

Use a maximum of two actions to guide users. Always make the most likely one prominent by using a primary button and display the others as a link to make a clear hierarchy.



Desktop version



Success Feedback

This component is different from the Success Feedback Screen. Unlike the Success Feedback Screen, this component does not cover 100% of the screen and allows adding more components below it.

Implementation

Additional info

Forms

Good Practices

A form is a conversation. And just like all conversations, this needs to be structured in a logical way, between two parties: in this case, between the user and our app. With the aim of building forms that facilitate coherent communication through our product, making this process as easy as possible for users, we've compiled several tips that can help us improve usability.

Request only the necessary information: We recommend deleting the fields whose information can be: collected in another way; collected later; or simply deleted. Remember that short forms increase the conversion ratio.

Use Title1 to group together related fields, helping to provide more context and facilitate interface scanning.

Follow a logical and predictable order: For example, we'll first request the credit card number, followed by the expiration date and lastly, the CVV number.

Structure

When a user accesses a form, we must, insofar as possible, avoid showing a large number of fields that require filling out, since this can frustrate the user when completing the form.

While it may be difficult to ascertain whether or not a form is very long, the type of flow or context can help us when structuring the different steps. We at Teléfonica

CX recommend designing forms with a maximum of 3 fields per step, as long as these fields are of the same type.

Multi-step forms

This technique can be used to distribute form fields across more than one screen, incorporating the use of a progress bar or a stepper to ensure that users are aware of their status at all times. When using this feature, each screen should show groupings of fields that are of the same type e.g. city, state and post code.

Layout of form elements

To organise the fields within a form, we have a modular layout that allows us to modify their distribution in accordance with our requirements. The layout can assume the form of a single column, which will be the default option, that stacks the fields vertically; two columns, allowing us to horizontally group related fields; or a combination of both layouts throughout the form.

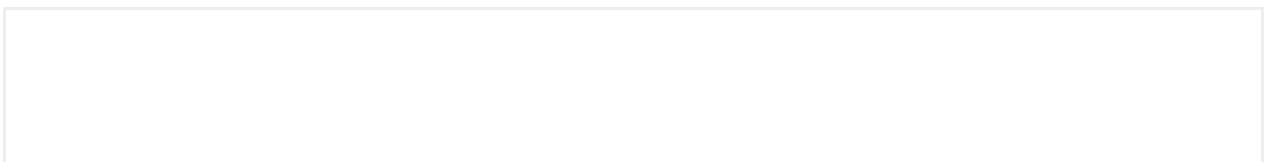
There is a component called "DoubleField" to make this possible in a easy way.

One-column layout

This layout allows us to stack the fields vertically, which makes it easier for users to scan through and complete a form.

Two-column layout

If we need to distribute multiple fields in the same row, we'll use this two-column layout. We'll restrict its use for related fields, which require the user to enter a little bit of information. Remember that the fields in the two-column format are not as wide as those in the one-column format; as such, they serve as reference or visual orientation for the user in fields where 1 to 4 characters must be entered.





Behavior

Auto-focus

When a user enters a form, it will not auto-focus on any of the fields.

Auto-jump

Auto-jump is the feature that allows the user to jump to the next field after filling out the information required by an input field.

This behavior only applies to fields that have a character restriction for a specific format. For example, a credit card number, a telephone number, a date, a selector, a post code, etc.

The auto-jump feature always moves forwards. In other words, if the first field is not filled in and the user goes directly to the second field, there would be an auto-jump to the third field, but never an auto-jump from the second to the first, nor from the third to the first.

Behavior in case of error

If there is format error, auto-jump does not occur. Instead, the error message would appear.

Behavior in the last field with auto-jump

We have two scenarios

1. When the last field has auto-jump and there are no other fields that precede it. In this case, the keyboard will collapse and disappear.
2. When the last field has auto-jump and there are other fields that precede it. In this case, the focus would jump to the next enabled field. From this point on, the user will need to manually jump to the next field, or close the keyboard.

Scroll

To provide the user with the full context of the form or the process to be started, and so that they don't have problems viewing it on a screen with a conversational header and keyboard that can be positioned above the lower fields (reducing the focus of the screen), we will set a behavior for the scroll based on the 3 following states:

State 0

When entering a form, there will be no auto-focus on the first field. Rather, the user will need to proactively tap on the first field, or any other field.

State 1

When the user taps on one of the fields, the scroll positions the selected field when the header is hidden away, allowing users to view more information.

State 2

If there is no more content at the bottom of the screen, the scroll will remain static when the user focuses on the remaining fields. If we had a very long form, the scroll would continue to move down, allowing the user to see the following information at all times.

Errors

Forms have a specific format to give users feedback on their completion and possible errors.

Errors are always specific to the input they refer to, telling users with as much as precision as possible the information they need to complete it successfully. Please use concise but helpful copywriting for these messages and avoid being vague or too general.

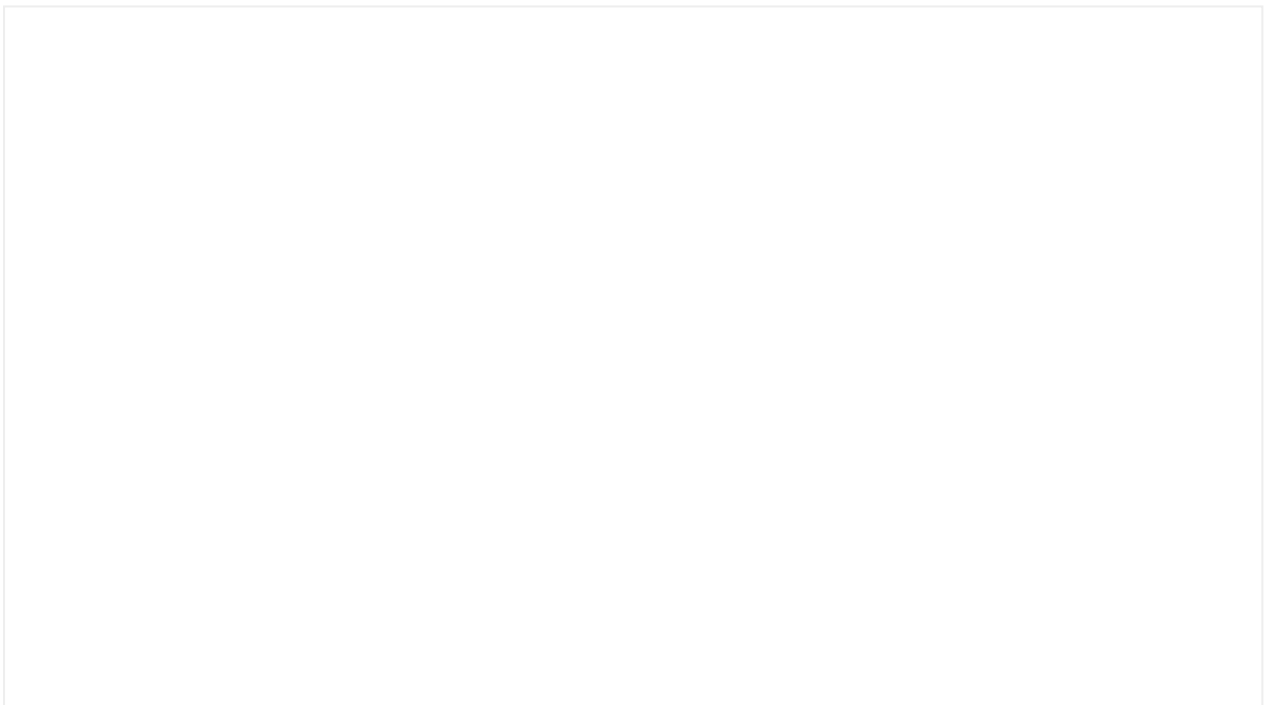
In the specific case of the date field with limited date range, the error should be always shown when the user select a non valid/permitted range.

Also take into account that form errors related with login are an exception to this rule. They shouldn't point out a specific form element to avoid any possible unauthorized access. Show these errors with a general message in an alert instead.

These error messages are persistent until users retry submitting the form.

Errors in Terms & conditions checkboxes in forms

Error in forms with Terms & Condition checkboxes always shows alert/dialog component when user doesn't mark this checkbox.



Actions in forms

Actions allow users to submit a completed form.

Positioning of the buttons on forms

The buttons shall remain fixed to the bottom of the screen. When the keyboard is displayed, it will appear over the buttons.

Keyboard actions

To facilitate navigation through a form, users can use the actions that are available in the keyboard toolbar.

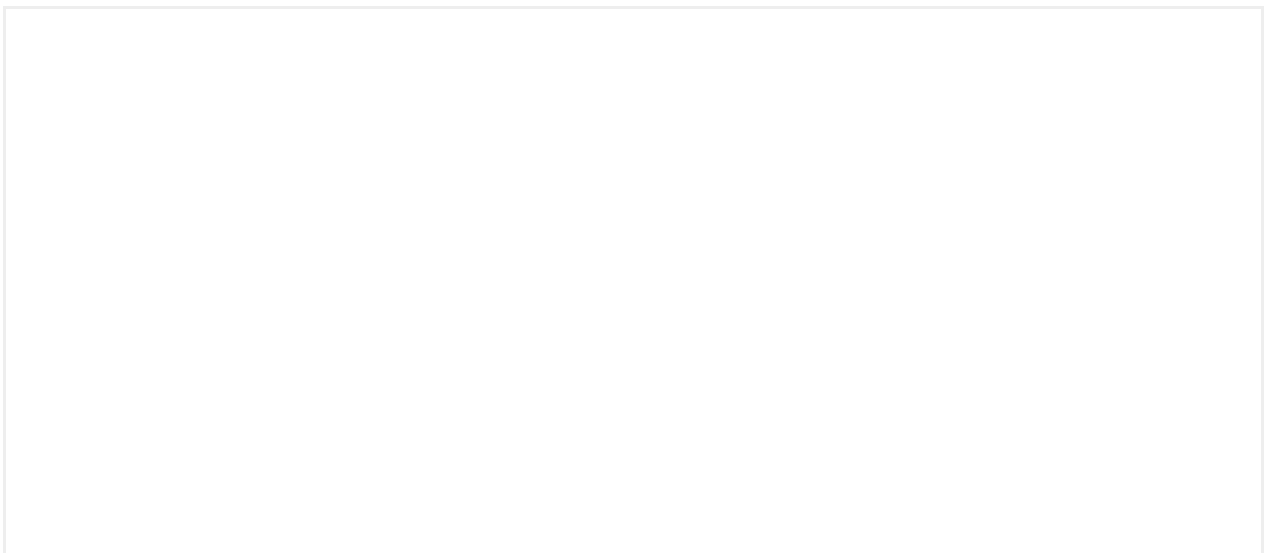
Alphanumeric, numeric and picker keyboards

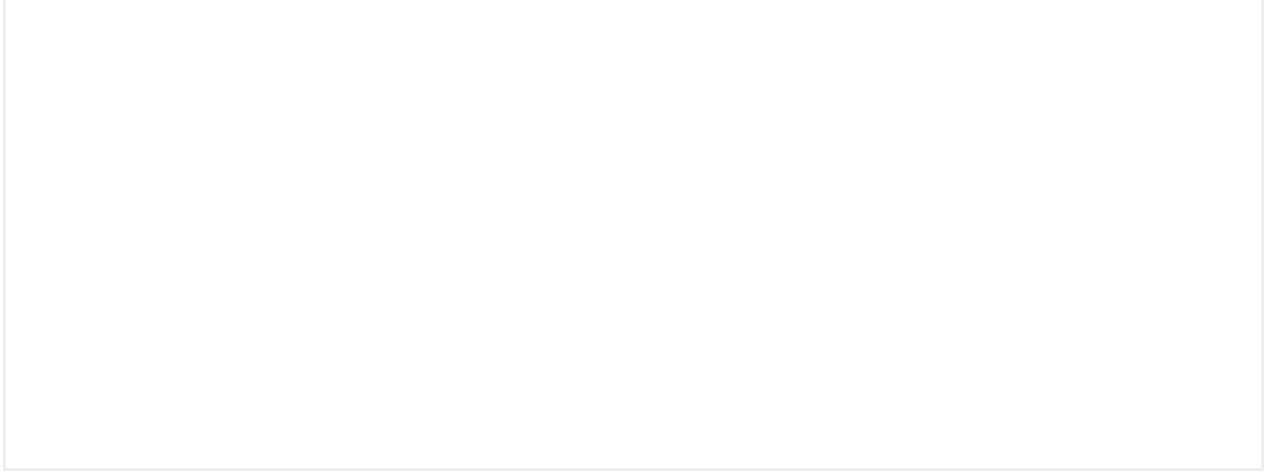
In a form, the lower right part of alphanumeric, numeric and picker keyboards have a function that allows the user to move to the next field (Next), before marking the action as completed (Done) once they have reached the final field. Once the user has tapped Done, the keyboard will disappear. Then, it will be the action of the CTA that allows the process to continue.

By tapping anywhere on the screen (as long as it is not an input field), the keyboard will disappear.

Related content

[Find more information about buttons](#)





Implementation

Additional info

Loading screens

Lets users know content or process is in progress and the entire page needs to be blocked until the data is retrieved.

Loading Screen



Anatomy

Both components contains the following elements:

- **Spinner or brand loading animation**
- **Title** (Optional): give the user about what the system is doing, such as "Uploading data ...".
- **Description** (Optional): use it to add more especific information about the background process.

Brand Loading Screen

Animation

This typology has been designed to make wait times feel shorter through the visual support of animation.

In addition, each brand can include its own custom animation and integrate it into its skin, making the wait shorter and the experience more distinctive.

Usage

Unlike the loading screen, the Brand Loading Screen could be used for loads where more waiting time is required.

Accessibility

Implementation

Additional info

How to contribute

If you're looking for something that Mística doesn't have, you have an idea or you notice something could be improved, you can create a discussion.

Create your discussion in GitHub

If you want, you can also create a discussion in GitHub explaining your need. Follow [this template](#) to create your first discussion

Create a discussion

** HIDDEN **

Contact us through Teams

Contributors like you help to make Mística great. Feel free to tell us about your idea or need through Teams. We look forward to hearing from you!

Contact us via Teams

** HIDDEN **

Requirements to contribute

We work closely with teams to define what we include. In this way, we ensure that Mística DS delivers the greatest value to those who use it. We therefore **place a high value on each proposal that can be included in Mística delivering value across different products**. We call this shared value. To achieve this, your proposal must:

- Use Mística
- Respond to a **global need**.
- **Be scalable** to other products or already be used by several of them.
- **Be correctly defined** and scalable to all media, including Desktop.

This is the process you need to follow to contribute to Mística:

What should I do if my proposal does not offer shared value?

If your proposal cannot establish shared value for two or more products, **consider using the existing pieces in Mística DS to create the proposal within your project.** Think that Mística can provide a wide set of styles and primitives which **you can use to build your "extended component" and solve your problem.**

If we detect that more teams can benefit from your proposal in the future, we will start the process to include it as part of Mística.

Thank you for your interest in making Mística great! ✨

Libraries management

Existen diferentes formas de consumir Mística por un producto.

- Mística library (caso ideal)
- Mixed libraries (Mística + Extended team library)

Usando el 100% de Mística

El equipo se debe adaptar a lo que provee Mística

Mixed libraries

Existen diferentes procesos en los que un equipo puede contribuir con Mística

- Directamente a Mística
- Migrando a Mística un componente de una extended library
-

Join our community

FAQs

Changelog

Discover about new features, bug fixes, important changes and what the team has planned for Mística's future. You can see them documented by version dates.

[Go to changelog](#)

Get all the changes and more in your inbox



[Subscribe to mística newsletter!](#)

News and articles

Discover our latest news and learn about Mística.

- **Mística, el sistema de diseño digital de Telefónica que usa componentes tecnológicos reutilizables**
- **Mística, el sistema de diseño que se adapta a los cambios**

We're writing new articles and generating content for the community. At the end of each quarter, we publish a new edition of our newsletter where we share all the latest updates on Mística, including improvements, new components, and practical tips on accessibility and design.

Subscribe to get full updates!

 [Subscribe to Mística Newsletter!](#)

[Visit Wrapped '23](#)

[Visit Wrapped '22](#)

Glossary

Definition of the key terms and concepts that appear in Mística documentation.

Atom

Element from Mística that in combination with other elements (from mística or custom) solve needs not covered by the system.

Atomic constructions

Elements that are built using a combination of atoms or custom elements.

Design token

Information associated with a name, at minimum a name/value pair.

In Mística, we use two words to talk about design tokens.

- Constant
- Variable

Constant is the value that does not change between brands. Constant is equivalent to "token". In the example below, color-text-primary would be the constant.

Variable is the value that changes between brands. In the example below, #000000 would be the variable. This value can be an hexadecimal color or the name of another variable like movistarBlue (that inside store the hex value)

Extended component

These components are built using Mística atoms and styles to meet very specific needs of a team. Due to their characteristics, they are stored in their own separate

library (extended library), not in the core library.

Extended library

This is a separate library created to store the extended components.

Primitive

Low-level, highly-flexible components that are meant to be reused for building higher specificity components or atomic constructions.

Skin

A brand list of values applied to the theme design tokens.

Slot

A slot is a flexible space enabled in some components for the inclusion of customized content. [More info.](#)